



UNIVERSITY OF CAPE TOWN

STA5004W

MSC STATISTICS & DATA SCIENCE MINOR DISSERTATION

Optimization and Regularization Under Arbitrary Objectives

Author: Jared N. Lakhani

Supervisor: Dr. Etienne A.D. Pienaar

12 February 2026

Project Repository

Access the project files and associated Beamer presentation for this report at:

 github.com/LKHJAR001/Optimization-and-Regularization-Under-Arbitrary-Objectives.

A.I Declaration

The author has used ChatGPT, as permitted by the supervisor, solely for language-related assistance, including grammar refinement, spell-checking, wording improvements, and LaTeX formatting of equations and text. The author takes full responsibility for the final content of this submission.

Abstract

This study investigates the limitations of applying Markov Chain Monte Carlo (MCMC) methods to arbitrary objective functions, focusing on a two-block MCMC framework which alternates between Metropolis-Hastings and Gibbs sampling. While such approaches are often considered advantageous for enabling data-driven regularization, we show that their performance critically depends on the sharpness of the employed likelihood form. By introducing a sharpness parameter and exploring alternative likelihood formulations proportional to the target objective function, we demonstrate how likelihood curvature governs both in-sample performance and the degree of regularization inferred by the training data. Empirical applications are conducted on reinforcement learning tasks: including a navigation problem and the game of tic-tac-toe. The study concludes with a separate analysis examining the implications of extreme likelihood sharpness on arbitrary objective functions stemming from the classic game of blackjack, where the first block of the two-block MCMC framework is replaced with an iterative optimization step. The resulting hybrid approach achieves performance nearly identical to the original MCMC framework, indicating that excessive likelihood sharpness effectively collapses posterior mass onto a single dominant mode.

Acknowledgements

I sincerely thank my supervisor, Dr. Pienaar, for their support and guidance throughout this dissertation.

Contents

1	Introduction	1
2	Literature Review	4
2.1	Overparameterization and regularization	4
2.2	Optimization beyond supervised learning	6
2.3	Regularization in reinforcement learning	7
2.4	MCMC for optimization	8
2.5	Two-block MCMC methods	9
3	Technical Preliminaries	11
3.1	Metropolis-Hastings	11
3.1.1	σ_θ^2 : A parameter with a hyperprior	12
3.2	Adaptive Metropolis-Hastings	13
3.3	Adaptive scaling	14
4	The Objective & Regularization	16
5	The Likelihood	18
5.1	Likelihood dominance & implications	19
6	Experimental Problems with Arbitrary Objectives	20
6.1	A navigation problem	20
6.1.1	Encoding	21
6.1.2	The game state	22
6.1.3	Control: Drone movement	22
6.1.4	An arbitrary objective	24
6.1.5	Likelihoods	24
6.1.6	Results: Effects of regularization	31
6.1.7	Results: MCMC	36
6.2	The Tic-Tac-Toe problem	38
6.2.1	Encoding	39
6.2.2	The game's outcome	39
6.2.3	Control	40
6.2.4	The arbitrary objective	41
6.2.5	Results: Effects of regularization	42
6.2.6	Results: MCMC	44
6.3	The Blackjack Problem I: Controlling Player Decisions	49
6.3.1	Encoding	49
6.3.2	The hand's outcome	50
6.3.3	Control: Player decision	50
6.3.4	The arbitrary objective	51
6.3.5	Results: Effects of regularization	52
6.4	The Blackjack Problem II: Controlling Bet Size	55
6.4.1	Control: Bet size	55

6.4.2	The arbitrary objective	56
6.4.3	Results: Effects of regularization	57
7	Hybrid Algorithms	61
7.1	Genetic algorithm hybrid	61
7.2	Gradient descent hybrid for classification	62
7.3	A toy example: 3-class classification	63
7.4	Hybrid for Blackjack Problem I, II and III	65
8	Conclusion	69
	References	71
	Appendices	74
	Appendix A Joint Metropolis-Hastings	74
	Appendix B Neural Network Architecture	74
	Appendix C Genetic Algorithm	75
	Appendix D Specifications	77
	Appendix E Random Search	78
	Appendix F The True Count	78
	Appendix G 3-Class Classification Particle Data	79
	Appendix H Code Snippets	79

1 Introduction

In supervised learning regimes, modern machine learning models can be viewed as flexible classes of non-linear basis functions that are capable of approximating complex relationships between inputs and outputs (Vapnik, 1998; Hastie, Tibshirani, and Friedman, 2009). Neural networks, kernel methods, and other parametric architectures provide rich sets of possible input-output relationships within which a predictive mapping can be learned from data (Bishop, 2006; Shalev-Shwartz and Ben-David, 2014). Training such models typically involves selecting a particular functional form from this class and estimating its parameters by minimizing an objective function that quantifies the discrepancy between model predictions and observed outcomes (Vapnik, 1991). This objective is most commonly defined through a loss function, such as mean squared error or cross-entropy, and optimization is performed with respect to the model parameters using gradient-based methods.

However, the expressive power that makes modern machine learning models attractive also introduces significant statistical challenges. In particular, many commonly used models are overparameterized, meaning they contain far more parameters than are strictly required to interpolate the training data (Hastie et al., 2009). While this overparameterization enables near-perfect in-sample fit, it simultaneously increases the risk of poor generalization to unseen data. As a result, controlling model complexity becomes essential. Regularization provides a principled mechanism for addressing this issue by augmenting the objective function with a penalty that discourages overly complex parameter configurations (Bishop, 2006). By constraining the effective capacity of the model, regularization enforces a trade-off between fitting the training data and achieving robust out-of-sample performance.

In supervised learning settings, regularization is naturally integrated into gradient-based optimization frameworks, and its effects are well understood both from statistical learning theory and from a Bayesian perspective (Bishop, 2006; Hastie et al., 2009). However, many learning problems of practical interest fall outside the supervised learning paradigm: in particular, reinforcement learning tasks, where agents learn by interacting with an environment and receive feedback in the form of rewards, rather than labeled input-output pairs (Sutton and Barto, 2018). In such settings, the objective function may be noisy, discontinuous, or non-differentiable with respect to the model parameters. This motivates the notion of an arbitrary objective: an objective that need not be differentiable with respect to the environment or directly tied to observed data. It only requires that the objective function assigns high values to desirable behaviour and low values to undesirable behaviour. These arbitrary objectives render conventional gradient-based optimization techniques inapplicable or unreliable, motivating the use of alternative optimization strategies.

One class of approaches capable of handling such arbitrary objectives is based on stochastic search and sampling methods. Evolutionary algorithms, such as genetic algorithms, explore the parameter space through population-based updates without relying on gradient information (Holland, 1975). Similarly, Markov Chain Monte Carlo (MCMC) methods can be adapted for optimization by reformulating objective functions as probability distributions and sampling from said distributions, as in simulated annealing (SA) (Kirkpatrick, Gelatt, and Vecchi, 1983; Geyer and Thompson, 1995). In particular, SA reformulates a cost function $C(x)$ into a Boltzmann-like distribution, $\pi_\beta(x) \propto \exp(-\beta C(x))$, where $\beta = 1/T$ denotes the inverse temperature. At high temperature (small β), the Markov chain explores the state space broadly, whereas at low temperature (large β) it increasingly concentrates around global minima of the

cost function.

Among sampling-based approaches, MCMC methods are often presented as appealing due to their flexibility and their supposed ability to allow regularization to be inferred from data (Gelman, 2006; Robert and Casella, 2004; Bishop, 2006). However, longstanding discussions in the MCMC and Bayesian computation literature emphasize that such interpretations rely critically on the probabilistic meaning of the target distribution and the manner in which likelihoods are specified or scaled (Geyer and Thompson, 1995; Robert and Casella, 2004). In particular, following the perspective articulated by Neal (2001), when MCMC targets are not grounded in a well-defined generative model, changes to likelihood sharpness or scaling modify the distribution being sampled, making it unclear whether the procedure should be interpreted as posterior inference or as a form of stochastic optimization.

This study extends the discussion by examining the shortcomings of applying MCMC to arbitrary objectives, with particular emphasis on two-block MCMC (alternating Metropolis-Hastings (MH) and Gibbs sampling), a method often presented as advantageous for allowing the training set to infer regularization. Rather than simply exponentiating an arbitrary objective as in SA, the study investigates alternative likelihood formulations deliberately shaped to remain proportional to the arbitrary objective function. This shifts MCMC from a sampling paradigm towards a mode-seeking algorithm, paralleling the perspective of SA. In addition to altering the likelihood specification, the study explicitly modulates the sharpness of the likelihood form through a sharpness parameter. The analysis highlights that the sharpness of the likelihood form plays a decisive role in both determining in-sample performance and the strength of regularization inferred by the training set.

To illustrate these points, the study applies two-block MCMC to reinforcement learning tasks: specifically a navigation problem and tic-tac-toe. Beforehand, the study investigates the role of regularization on out-of-sample performance for solutions obtained from a genetic algorithm (GA), and additionally compares these results to solutions obtained by random search (RS).

The study concludes by illustrating the implications of increased likelihood sharpness, demonstrating this through reinforcement learning tasks associated with blackjack. Here, the study simplifies the two-block MCMC by replacing the first block with an iterative optimization procedure and compares this hybrid approach to the original scheme. The resulting near-identical performance demonstrates that increasing likelihood sharpness ultimately collapses posterior mass onto the dominant mode, echoing the observation of Kirkpatrick et al. (1983) that lower temperatures (that is, higher likelihood sharpness β) increasingly concentrate samples around the global minima of the cost function.

Furthermore, this study makes three primary contributions. First, it provides a conceptual clarification of how MCMC behaves when applied to arbitrary objectives, showing that such procedures should often be interpreted as stochastic optimization methods rather than as Bayesian posterior inference. Second, it demonstrates empirically that the sharpness of the likelihood formulation acts as an implicit regularization control, and directly influences both in-sample and out-of-sample performance. Thirdly, the study shows that increasing likelihood sharpness causes two-block MCMC to collapse toward deterministic, mode-seeking behaviour, echoing the mechanism underlying SA. Together, these findings clarify what MCMC-based procedures are effectively accomplishing in arbitrary objective settings. Additionally, the study

also offers practical guidance on how to implement two-block MCMC effectively in these settings, where the study highlights the importance of adaptive MCMC procedures to improve mixing and stabilizing performance.

2 Literature Review

2.1 Overparameterization and regularization

Neural networks, along with various other machine learning architectures, constitute a class of overparameterized models: models with more parameters than training samples. As a consequence of this overparameterization, such models exhibit a propensity to achieve near-perfect fit on the training data yet require the incorporation of regularization techniques to ensure satisfactory generalization to previously unseen data. This concept of overparameterization can be formally understood through the Vapnik-Chervonenkis (VC) Generalization Bound, which relates in-sample and out-of-sample errors. With probability $1 - \delta$ we have: $E_{out}(g) \leq E_{in}(g) + \sqrt{\frac{8}{N} \ln \left(\frac{4(2N)^{d_{VC}} + 1}{\delta} \right)}$ ¹, wherein the VC dimension d_{VC} serves as a measure of model complexity. Specifically, an increase in the complexity of the hypothesis space corresponds to a higher VC dimension, which in turn, induces an expansion of the generalization gap thereby increasing the risk of overfitting, as stated in Vapnik (1991).

Regularization, as formally expounded in Hastie et al. (2009), is a fundamental technique in statistical learning that introduces a penalty term to the model’s objective function, thereby discouraging excessive model complexity. One widely adopted regularization approach is L2 regularization, also known as ridge regularization, wherein the squared magnitude of the model coefficients (or weights) is penalized. Unlike L1 regularization, which promotes sparsity by driving certain coefficients exactly to zero, L2 regularization instead constrains the magnitude of all parameters, thereby ensuring a controlled reduction in model complexity without entirely eliminating any particular parameter. In the context of fitting a single-output neural network, if we wanted to minimize the mean square error objective of $\sum_{i=1}^N (y_i - a_1(i)^L)^2$ ², we would include the constraint $\sum_{j,k,l} (w_{kj}^l)^2 \leq \tau$ ³, where τ would act as a means to constrain the parameters of the model to be closer to zero, thus lessening model complexity. Furthermore, the model fitting procedure becomes a constrained optimization problem for which we can re-write the penalized objective as:

$$\sum_{i=1}^N (y_i - a_1(i)^L)^2 + \nu \sum_{j,k,l} (w_{kj}^l)^2, \quad (1)$$

whereby increasing our Lagrangian multiplier ν would be equivalent to decreasing τ , thereby enforcing a stricter penalty on the model parameters.

Given that regularization introduces an additional penalty term to the objective function, the resulting optimization problem remains differentiable with respect to the model parameters, thereby necessitating the use of gradient-based optimization techniques such as gradient descent. Specifically, gradient descent iteratively updates the model parameters in the direction of the negative gradient of the objective function, ensuring convergence to an optimal solution under appropriate step-size selection and convexity conditions, as described in Amari (1993). Furthermore, the choice of regularization strength, governed by the parameter ν , critically influences the trade-off between model complexity and generalization. As such, determining an optimal value for ν requires an empirical approach, typically employing a validation

¹ E_{out} and E_{in} denote the out-of-sample and in-sample errors respectively, g denotes the final chosen hypothesis function, N denotes the number of samples and δ denotes the tolerance.

² Here, y_i denotes the i^{th} outcome of the N observations, where $a_1(i)^L$ represents the sole node of the i^{th} observation in the output layer L . Full neural net architecture is described in Appendix B.

³ Here, w_{kj}^l denotes the kj -th weight linking the k -th node in layer $l - 1$ and the j -th node in layer l .

set to assess model performance under varying degrees of regularization. This validation set approach, entails training the model with multiple candidate values of ν and subsequently evaluating its predictive performance on held-out validation data. The value of ν that minimizes the validation error is then selected, ensuring that the model is sufficiently complex to replicate nuances of the underlying pattern in the data, but also not too complex such that the model simply “recalls” what has been seen in the data.

From a Bayesian perspective, regularization can be interpreted as placing prior distributions on the parameter space of an overparameterized model. Specifically, L2 regularization corresponds to assuming a normal (Gaussian) prior on the parameters.

Proof. We assume our observed response $y_i | \mathbf{w}, \mathbf{a}(i)^0 \sim \mathcal{N}(a_1(i)^L, \sigma_y^2)$ and are independent from all other N observations. That is, we assume our observations follow a Gaussian distribution with mean equal to the sole node of the i^{th} observation in the output layer L , with some variance σ_y^2 . Here, the vector $\mathbf{w}$ denotes the R number of weights and biases of a single-output neural network, with vector of inputs $\mathbf{a}(i)^0$ denoting the input nodes (on the 0^{th} layer) for the i^{th} observation.

Furthermore, we assume $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}_R)$. That is, the vector of R independent weights and biases is multivariate Gaussian distributed with mean of $\mathbf{0}$ with some covariance matrix $\sigma_w^2 \mathbf{I}_R$.

Using Bayes rule we have:

$$\begin{aligned} p(\mathbf{w} | \mathcal{D}) &= \frac{p(\mathcal{D} | \mathbf{w})p(\mathbf{w})}{p(\mathcal{D})} \\ &\propto p(\mathcal{D} | \mathbf{w})p(\mathbf{w}) \\ &\propto \left[\prod_{i=1}^N \mathcal{N}(y_i; \mathbf{a}(i)^1, \sigma_y^2) \right] \mathcal{N}(\mathbf{w}; \mathbf{0}, \sigma_w^2 \mathbf{I}) \\ &\propto \prod_{i=1}^N \mathcal{N}(y_i; \mathbf{a}(i)^1, \sigma_y^2) \prod_{i=1}^R \mathcal{N}(w_i; 0, \sigma_w^2), \end{aligned}$$

where, given a dataset $\mathcal{D}$, $p(\mathcal{D} | \mathbf{w})$ is the likelihood, representing how well the parameters explain the observed data; $p(\mathbf{w})$ is the prior, encoding our beliefs about the parameters before seeing the data; and $p(\mathbf{w} | \mathcal{D})$ is the parameter posterior, the updated distribution of the parameters after observing the data.

Now taking the negative log probability of the parameter posterior:

$$\begin{aligned} -\log[p(\mathbf{w} | \mathcal{D})] &\propto -\sum_{i=1}^N \log [\mathcal{N}(y_i; \mathbf{a}(i)^1, \sigma_y^2)] - \sum_{i=1}^R \log [\mathcal{N}(w_i; 0, \sigma_w^2)] \\ &\propto \frac{1}{2\sigma_y^2} \sum_{i=1}^N (y_i - a_1(i)^L)^2 + \frac{1}{2\sigma_w^2} \sum_{i=1}^R w_i^2 \\ &\propto \sum_{i=1}^N (y_i - a_1(i)^L)^2 + \nu \sum_{i=1}^R w_i^2, \end{aligned}$$

whereby ν controls the strength of the regularization as in the L2 penalized loss function in Equation 1. □

Clearly, minimizing the L2 penalized loss function is equivalent to maximizing the posterior distribution of the parameters under a Gaussian prior, which corresponds to Maximum a Posteriori (MAP) estimation.

2.2 Optimization beyond supervised learning

In the context of reinforcement learning (RL), an agent learns to perform tasks by interacting with an environment, making sequential decisions, and receiving feedback in the form of rewards, as described in Painter and Brunskill (2018). The objective function, in this case, is to maximize the cumulative reward of the agent within the environment. Unlike supervised learning, gradients cannot be computed directly with respect to labeled data, as RL models are not trained on explicit input-output pairs. As a result, the objective is generally not expressed as a differentiable loss function evaluated on a fixed dataset, and gradients cannot be obtained by straightforward backpropagation through data. This motivates the notion of an arbitrary objective: an objective that need not be differentiable with respect to the environment or directly tied to observed data. It only requires that the objective function assigns high values to desirable behaviour and low values to undesirable behaviour.

Although reinforcement learning objectives are generally arbitrary and not derived from likelihood-based models, there exist special cases in which such objectives can be expressed as expectations with respect to a parameterized probability distribution. When an objective takes the form⁴

$$J(\theta) = \mathbb{E}_{x \sim p_\theta(x)}[f(x)],$$

gradient-based optimization becomes feasible. In particular, the likelihood-ratio identity yields

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim p_\theta(x)}[f(x) \nabla_\theta \log p_\theta(x)],$$

allowing gradients of the expected objective to be computed without requiring differentiability of the reward function. This result enables the use of gradient-based optimization methods even when the underlying objective is non-supervised, noisy, or non-differentiable with respect to the environment.

When arbitrary objectives cannot be expressed in a form amenable to gradient estimation, rendering standard gradient-based optimization inapplicable, alternative optimization methods capable of solving arbitrary objectives must be employed. Genetic algorithms (GAs) are a prominent class of such methods. They simulate the evolutionary process by initiating a population of random candidate solutions and employing selection mechanisms to iteratively recombine and mutate these candidates. This process yields new populations with potentially enhanced fitness (objective function values) relative to previous populations. Now given sufficient iterations, this process systematically favours traits (parameter values) that should enhance performance in achieving the objective function. As a result, we obtain an evolutionary algorithm in which the population of solutions gradually evolves toward an optimal solution.

While GAs provide a biologically inspired mechanism for navigating complex search spaces, it is important to contextualize their performance against simpler baseline strategies. In particular, random search (RS) offers a natural point of comparison due to its algorithmic simplicity and lack of heuristic bias.

⁴Here, $f(x)$ denotes a reward function and $p_\theta(x)$ is a distribution over outcomes induced by parameters θ .

RS operates by sampling candidate solutions uniformly at random from the feasible domain, evaluating their objective values, and retaining the best-performing solution observed over a fixed number of trials. Despite its simplicity and lack of adaptive guidance, RS has demonstrated competitive performance in a range of settings, particularly when the objective landscape is noisy, discontinuous, or lacks exploitable structure as detailed in Bergstra and Bengio (2012). This makes RS an appropriate benchmark for assessing whether a GA is truly needed, or if a GA’s performance gains are only marginal relative to an unguided search.

2.3 Regularization in reinforcement learning

While regularization is well understood and formally grounded in supervised learning, its role in RL and other arbitrary-objective settings is considerably less explicit. In RL, learning proceeds through interaction with an environment rather than from fixed input-output pairs, and the objective is typically defined as the maximization of cumulative reward. As a result, standard notions of generalization and overfitting are less directly applicable, and regularization is often introduced implicitly or heuristically rather than derived from a probabilistic framework.

One class of approaches introduces regularization explicitly at the parameter level. For example, weight decay (e.g., L2 regularization) is commonly applied to arbitrary objectives as a means of controlling model complexity. In this setting, the regularization parameter λ is not inferred from the data, but is instead treated as a fixed hyperparameter chosen externally by the practitioner. Typical selection procedures include heuristic tuning, grid search, or adherence to values used in prior studies as expounded in Mnih et al. (2016) and Henderson et al. (2018). As noted by Henderson et al. (2018), empirical performance in deep reinforcement learning can be highly sensitive to such implementation choices, including the strength of L2 regularization.

In cases where the arbitrary objective, stemming from a particular RL task, can be expressed in a form amenable to gradient estimation⁵, entropy regularization is commonly used to encourage stochastic policies and promote exploration as stated in Williams (1991) and Mnih et al. (2016). Entropy regularization augments the reinforcement learning objective $J(\theta)$ by adding an expected entropy term⁶,

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda \mathbb{E}_s[\mathcal{H}(\pi_\theta(\cdot | s))],$$

where $\lambda > 0$ controls the strength of the regularization. This modification biases learning toward policies that retain randomness, discouraging premature convergence to deterministic behavior and smoothing the optimization landscape.

⁵Entropy regularization only becomes operational when gradient information is available, as its effect relies on computing how the entropy of the policy changes with respect to the parameters θ . Without access to $\nabla_\theta \mathcal{H}(\pi_\theta(\cdot | s))$, the regularization term cannot provide directional guidance for optimization and thus cannot be meaningfully enforced.

⁶Let $\pi_\theta(a | s)$ denote a parameterized policy, which assigns a probability to each action a given a state s , where θ represents the policy parameters. The entropy of the policy at state s is defined as

$$\mathcal{H}(\pi_\theta(\cdot | s)) = - \sum_a \pi_\theta(a | s) \log \pi_\theta(a | s),$$

which quantifies the uncertainty of the action distribution.

Closely related are trust-region methods, which impose a Kullback-Leibler (KL) divergence penalty⁷ between successive policy updates, effectively regularizing the learning process by restricting abrupt changes in policy space. In particular, Trust Region Policy Optimization (TRPO) enforces an explicit constraint on the KL divergence between the old and updated policies, ensuring that each update remains within a local trust region and preventing overly aggressive changes in policy space as conducted in Schulman, Levine, Abbeel, Jordan, and Moritz (2015). Proximal Policy Optimization (PPO) adopts a computationally simpler approximation to this idea, replacing the hard KL constraint with a clipped surrogate objective or an adaptive KL penalty that discourages large deviations from the previous policy as carried out in Schulman, Wolski, Dhariwal, Radford, and Klimov (2017). In both cases, the KL divergence acts as a regularizer that stabilizes learning by restricting abrupt policy updates. While these techniques improve stability and empirical performance, the strength of regularization is again, externally specified and not inferred from the data.

A second strand of work introduces regularization implicitly through randomness in the learning process rather than through explicit penalty terms. In this setting, stochasticity is used to stabilize learning and prevent overly confident policies. For example, Gal and Ghahramani (2016) show that dropout acts as a form of regularization by randomly disabling neurons during training. By temporarily ignoring all weights connected to the dropped neurons, dropout prevents the network from relying too heavily on any single set of parameters, thereby helping to control overfitting. Another example of stochastic regularization is provided by Fortunato et al. (2018), who add small random perturbations directly to the neural network parameters, producing a slightly modified but consistent policy that improves exploration.

Additionally, during genetic algorithm optimization, stochastic regularization is enforced by evaluating candidate solutions under randomized environments: whereby the environment configuration is resampled at each iteration to promote robustness and reduce overfitting to any single environment realization, as demonstrated in Tobin, Fong, Ray, Schneider, Zaremba, and Abbeel (2017) and Peng, Andrychowicz, Zaremba, and Abbeel (2018). This variability discourages solutions that exploit specific environment realizations and instead favours parameter configurations that perform robustly across perturbations, thereby acting as an implicit form of regularization, as discussed in Hansen, Zhou, Al-Shedivat, Walker, Finn, and Levine (2021). In all these implicit approaches, regularization arises as a side effect of stochasticity rather than being explicitly imposed or inferred from data.

2.4 MCMC for optimization

In the framework of Bayesian Neural Networks, the parameters of the neural network are treated as random variables rather than fixed values. This means they follow a probability distribution, reflecting our uncertainty about their true values. Given a dataset $\mathcal{D}$, our goal is to infer the posterior distribution over the parameters θ , which is given by Bayes' theorem: $p(\theta | \mathcal{D}) \propto p(\mathcal{D} | \theta)p(\theta)$. Since the parameter posterior is typically intractable due to high-dimensional parameter spaces, that is, it either has a complex or unknown form, as stated in Dobson and Barnett (2018), MCMC methods may be employed to approximate it. MCMC generates samples from the parameter posterior by constructing a Markov chain whose stationary distribution is the true posterior. In this way, the utilization of MCMC aims to provide

⁷The KL divergence is

$$\text{KL}(\pi_{\theta_{\text{old}}}(\cdot | s) \| \pi_{\theta}(\cdot | s)) = \sum_a \pi_{\theta_{\text{old}}}(a | s) \log \frac{\pi_{\theta_{\text{old}}}(a | s)}{\pi_{\theta}(a | s)}.$$

the entire posterior distribution $p(\boldsymbol{\theta} \mid \mathcal{D})$ and not just the MAP estimate; corresponding to $\operatorname{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D})$ which is the mode of the posterior. According to Dobson and Barnett (2018), Markov chains simplify complex problems since the next sample in the chain depends solely on the previous sample: $p(\boldsymbol{\theta}^{(j)} = \mathbf{a} \mid \boldsymbol{\theta}^{(j-1)}, \boldsymbol{\theta}^{(j-2)}, \dots, \boldsymbol{\theta}^{(0)}) = p(\boldsymbol{\theta}^{(j)} = \mathbf{a} \mid \boldsymbol{\theta}^{(j-1)})$.

One of the earliest approaches to applying MCMC to arbitrary objective functions was achieved through SA, where one reformulates optimization as sampling from a distribution where good solutions have high probability. That is, the method reformulates a cost function $C(x)$ into a Boltzmann-like distribution, $\pi_{\beta}(x) \propto \exp(-\beta C(x))$, where $\beta = 1/T$ denotes the inverse temperature. At high temperature (small β), the Markov chain explores the state space broadly, whereas at low temperature (large β) it increasingly concentrates around global minima of the cost function. The algorithm proceeds by running a standard MH sampler targeting π_{β} and gradually increasing β according to a cooling schedule. In the limit as $T \rightarrow 0$, the chain places its mass on the global minima of $C(x)$. This approach was popularized in physics and combinatorial optimization by Kirkpatrick et al. (1983) and rigorously studied in the context of Bayesian image analysis by Geman and Geman (1984), who also established theoretical convergence guarantees under logarithmic cooling.

Subsequent work generalized SA by embedding temperature schedules more explicitly within MCMC frameworks to improve exploration of complex, multimodal objective landscapes. Parallel tempering, also known as Metropolis-coupled MCMC, runs multiple Markov chains in parallel at different temperatures, each targeting a tempered version of the objective-induced distribution, and periodically proposes swaps between chains, as in Geyer (1991). This mechanism allows high-temperature chains to explore broadly while low-temperature chains concentrate probability mass near optimal solutions, thereby mitigating premature convergence to local optima. Closely related approaches include simulated tempering, which treats the temperature itself as a stochastic variable within the Markov chain, as carried out by Marinari and Parisi (1992). Additionally, annealed MCMC schemes formalize temperature schedules into the sampling procedure, causing the Markov chain to progressively focus on high-quality solutions, as conducted by Geyer and Thompson (1995). Collectively, these methods retain the core idea of optimization via sampling from a sequence of tempered distributions whose mass progressively concentrates on optimal or near-optimal solutions as the temperature is lowered.

2.5 Two-block MCMC methods

Two-block MCMC methods are widely used in hierarchical Bayesian models to jointly infer model parameters and regularization-related hyperparameters from data. In this framework, the unknowns are partitioned into two blocks: typically the primary model parameters and a set of latent variables or hyperparameters governing prior strength, noise variance, or smoothness. Conditional on one block, the other can often be sampled efficiently, leading to alternating Gibbs or Metropolis-Hastings updates. This approach has been extensively applied in Bayesian regression and inverse problems, where regularization parameters such as variance components are treated as random variables and inferred from the data rather than fixed *a priori* (Gelman, 2006; Robert and Casella, 2007). For example, Hierarchical Gaussian models commonly use two-block MCMC to alternate between sampling regression coefficients and their associated precision parameters, allowing the data to determine the effective strength of shrinkage, as described in Banerjee, Carlin, and Gelfand (2014). Similar two-block MCMC schemes also appear in Bayesian variable selection, where latent inclusion indicators and regression coefficients are sampled in

separate blocks, allowing model complexity, measured by the number of active predictors, to be determined by the data rather than fixed *a priori*, as proposed by George and McCulloch (1997).

Two-block MCMC methods have also played a central role in Bayesian image analysis and inverse problems, where regularization is essential to counteract ill-posedness. In these settings, one block typically consists of the latent signal or image, while the second block contains hyperparameters controlling smoothness or noise precision. Geman and Geman (1984) popularized this approach in Bayesian image restoration, using alternating updates to infer both the image and its regularization strength. Subsequent work in inverse problems has employed similar schemes to infer spatial smoothness and noise levels directly from data, as in Kaipio and Somersalo (2005). Importantly, several authors have noted that as likelihoods become increasingly sharp, two-block MCMC transitions from genuine posterior sampling toward mode-seeking behaviour, closely resembling SA, as explained by Neal (2001). This observation highlights that while two-block MCMC is often framed as fully Bayesian inference, its practical behavior, and the degree to which regularization is truly inferred, depends critically on the relative concentration of the likelihood and prior.

3 Technical Preliminaries

This section marks the beginning of the methodological framework employed in this study. In particular, we detail one of the fundamental MCMC algorithms: the Metropolis-Hastings (MH) algorithm as illustrated by Hastings (1970) and Metropolis, Rosenbluth, Rosenbluth, Teller, and Teller (1953). The section also presents the adaptive MCMC procedures employed in this study which improve chain mixing and overall sampling efficiency.

3.1 Metropolis-Hastings

Given the current state of $\boldsymbol{\theta} \in \mathbb{R}^S$, that is $\boldsymbol{\theta}^{(j)}$, the MH algorithm proposes a new value $\boldsymbol{\theta}^*$ obtained from $\boldsymbol{\theta}^* = \boldsymbol{\theta}^{(j)} + \mathbf{Q}$. Subsequently, $\boldsymbol{\theta}^*$ is accepted as the new value in the Markov chain under the following acceptance criterion:

$$\boldsymbol{\theta}^{(j+1)} = \begin{cases} \boldsymbol{\theta}^*, & \text{if } U < \alpha, \\ \boldsymbol{\theta}^{(j)}, & \text{otherwise.} \end{cases} \quad (2)$$

Now the vector $\mathbf{Q}$ denotes drawn values from a proposal density (usually $\mathbf{Q} \sim \mathcal{N}(\mathbf{0}, \sigma_Q^2 \mathbf{I}_S)$), and U is a drawn value from a uniform distribution between 0 and 1, that is $U \sim \mathcal{U}(0, 1)$. Furthermore, α is the acceptance probability given by:

$$\begin{aligned} \alpha &= \min \left(\frac{p(\boldsymbol{\theta}^* | \mathcal{D})}{p(\boldsymbol{\theta}^{(j)} | \mathcal{D})} \cdot \frac{Q(\boldsymbol{\theta}^{(j)} | \boldsymbol{\theta}^*)}{Q(\boldsymbol{\theta}^* | \boldsymbol{\theta}^{(j)})}, 1 \right) \\ &= \min \left(\frac{p(\mathcal{D} | \boldsymbol{\theta}^*)p(\boldsymbol{\theta}^*)}{p(\mathcal{D} | \boldsymbol{\theta}^{(j)})p(\boldsymbol{\theta}^{(j)})} \cdot \frac{Q(\boldsymbol{\theta}^{(j)} | \boldsymbol{\theta}^*)}{Q(\boldsymbol{\theta}^* | \boldsymbol{\theta}^{(j)})}, 1 \right). \end{aligned}$$

If the proposal density is symmetric, then α simplifies to

$$\alpha = \min \left(\frac{p(\mathcal{D} | \boldsymbol{\theta}^*)p(\boldsymbol{\theta}^*)}{p(\mathcal{D} | \boldsymbol{\theta}^{(j)})p(\boldsymbol{\theta}^{(j)})}, 1 \right). \quad (3)$$

For the present study we impose symmetry by assuming Gaussian priors for our parameters $\boldsymbol{\theta} \in \mathbb{R}^S$, that is, $\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \mathbf{I}_S)$ (as elaborated in Section 4). Here, the likelihood ratio $\frac{p(\mathcal{D} | \boldsymbol{\theta}^*)}{p(\mathcal{D} | \boldsymbol{\theta}^{(j)})}$ reflects how much more likely the proposed parameter $\boldsymbol{\theta}^*$ is, compared to the current parameter $\boldsymbol{\theta}^{(j)}$, in explaining the observed data $\mathcal{D}$. Since the likelihood function quantifies the plausibility of the data given $\boldsymbol{\theta}$, the MH algorithm accepts proposed moves with higher likelihoods more readily, while still allowing occasional transitions to lower-likelihood regions to ensure proper exploration of the posterior $p(\boldsymbol{\theta} | \mathcal{D})$. Additionally, the prior ratio $\frac{p(\boldsymbol{\theta}^*)}{p(\boldsymbol{\theta}^{(j)})}$ contributes to exploration by favouring moves toward regions of the parameter space that are more consistent with prior beliefs, especially when the likelihood offers little guidance such as being relatively flat. Furthermore, one may view the the prior ratio as a means to ensure acceptance is not solely driven by proposals which increase the likelihood. In terms of terminology used later, we refer to a likelihood-driven sampler as one in which the likelihood ratio dominates the prior ratio in the acceptance probability expression of Equation 3.

3.1.1 σ_θ^2 : A parameter with a hyperprior

We now assume the variance of the prior of θ , σ_θ^2 , to not be fixed, but rather having its own distribution. Instead of sampling the entire parameter vector $\Lambda = [\theta', \sigma_\theta^2]' \in \mathbb{R}^{S+1}$ in a joint MH framework (as illustrated in Appendix A), we split the parameters into groups (or blocks) and sample each block conditionally on the others, leveraging their conditional distributions. Hence, under the two-block MCMC framework, the algorithm effectively samples from the joint posterior $p(\theta, \sigma_\theta^2 | \mathcal{D})$ by alternately drawing from the conditionals $p(\theta | \sigma_\theta^2, \mathcal{D})$ and $p(\sigma_\theta^2 | \mathcal{D}, \theta)$. To reiterate, we know that since $p(\theta | \sigma_\theta^2, \mathcal{D}) = \frac{p(\theta, \sigma_\theta^2 | \mathcal{D})}{p(\sigma_\theta^2 | \mathcal{D})}$, when sampling θ from $p(\theta | \sigma_\theta^2, \mathcal{D})$, σ_θ^2 is treated as fixed. This is because it is conditioned on the value sampled in the previous step of the sampler, which results in $p(\theta | \sigma_\theta^2, \mathcal{D}) \propto p(\theta, \sigma_\theta^2 | \mathcal{D})$. The same logic may be used to conclude $p(\sigma_\theta^2 | \theta, \mathcal{D}) \propto p(\theta, \sigma_\theta^2 | \mathcal{D})$, since when sampling from $p(\sigma_\theta^2 | \theta, \mathcal{D})$, θ is treated as fixed. The two-block MCMC framework can simplify the sampling process, particularly when the joint proposal distribution is complex or high-dimensional. Additionally, it can improve mixing in cases where there is strong posterior correlation between θ and σ_θ^2 , which can hinder the efficiency of joint updates. In this context, adequate mixing refers to the sampler's ability to explore the parameter space effectively: typically indicated by reduced autocorrelation between successive samples (for example, between $\theta^{(j)}$ and $\theta^{(j+1)}$ for all j), thereby promoting more reliable convergence to the stationary distribution. We outline the algorithm for the two alternating blocks as follows:

Block 1: Sampling $\theta | \sigma_\theta^2, \mathcal{D}$ We assume the prior $\theta | \sigma_\theta^2 \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \mathbf{I}_S)$ and a symmetric proposal density $\mathbf{Q}_\theta \sim \mathcal{N}(\theta^{(j)}, \sigma_{Q_\theta}^2 \mathbf{I}_S)$. Hence, the acceptance probability for the first block is:

$$\begin{aligned} \alpha_\theta &= \min \left(\frac{p(\theta^* | (\sigma_\theta^2)^{(j)}, \mathcal{D})}{p(\theta^{(j)} | (\sigma_\theta^2)^{(j)}, \mathcal{D})} \cdot \frac{Q(\theta^{(j)} | \theta^*)}{Q(\theta^* | \theta^{(j)})}, 1 \right) \\ &= \min \left(\frac{p(\mathcal{D} | \theta^*, (\sigma_\theta^2)^{(j)}) p(\theta^* | (\sigma_\theta^2)^{(j)})}{p(\mathcal{D} | \theta^{(j)}, (\sigma_\theta^2)^{(j)}) p(\theta^{(j)} | (\sigma_\theta^2)^{(j)})} \cdot \frac{Q(\theta^{(j)} | \theta^*)}{Q(\theta^* | \theta^{(j)})}, 1 \right) && \text{Bayes Theorem} \\ &= \min \left(\frac{p(\mathcal{D} | \theta^*) p(\theta^* | (\sigma_\theta^2)^{(j)})}{p(\mathcal{D} | \theta^{(j)}) p(\theta^{(j)} | (\sigma_\theta^2)^{(j)})}, 1 \right) && \text{Likelihood not dependent on } \sigma_\theta^2 \\ &= \min \left(\frac{p(\mathcal{D} | \theta^*) \cdot \frac{1}{\sqrt{(2\pi(\sigma_\theta^2)^{(j)})^S}} \exp\left(-\frac{1}{2(\sigma_\theta^2)^{(j)}} \|\theta^*\|^2\right)}{p(\mathcal{D} | \theta^{(j)}) \cdot \frac{1}{\sqrt{(2\pi(\sigma_\theta^2)^{(j)})^S}} \exp\left(-\frac{1}{2(\sigma_\theta^2)^{(j)}} \|\theta^{(j)}\|^2\right)}, 1 \right). \end{aligned}$$

Hence, taking the log, we obtain:

$$\log(\alpha_\theta) = \min \left(\log(p(\mathcal{D} | \theta^*)) - \frac{1}{2(\sigma_\theta^2)^{(j)}} \|\theta^*\|^2 - \log(p(\mathcal{D} | \theta^{(j)})) + \frac{1}{2(\sigma_\theta^2)^{(j)}} \|\theta^{(j)}\|^2, 0 \right), \quad (4)$$

after which, using Equation 2 to ascertain $\theta^{(j+1)}$.

Block 2: Sampling $\sigma_\theta^2 | \theta, \mathcal{D}$ We assume the hyperprior $\sigma_\theta^2 \sim \text{Inv-Gamma}(a, b)$. Since σ_θ^2 does not appear in the likelihood, the data $\mathcal{D}$ provides no additional information about σ_θ^2 beyond what θ

already does. Hence, conditioning on $\mathcal{D}$ does not change the distribution of $\sigma_\theta^2 \mid \theta$:

$$\begin{aligned}
p(\sigma_\theta^2 \mid \theta, \mathcal{D}) &\propto p(\mathcal{D} \mid \theta, \sigma_\theta^2) \cdot p(\sigma_\theta^2 \mid \theta) \\
&= p(\mathcal{D} \mid \theta) \cdot p(\theta \mid \sigma_\theta^2) \cdot p(\sigma_\theta^2) && \text{Likelihood independent of } \sigma_\theta^2 \\
&\propto p(\theta \mid \sigma_\theta^2) \cdot p(\sigma_\theta^2) && \text{Likelihood constant for fixed } \theta \\
&\propto \frac{1}{\sqrt{(2\pi\sigma_\theta^2)^S}} \exp\left(-\frac{\|\theta\|^2}{2\sigma_\theta^2}\right) \cdot \frac{b^a}{\Gamma(a)} (\sigma_\theta^2)^{-(a+1)} \exp\left(-\frac{b}{\sigma_\theta^2}\right) \\
&\propto (\sigma_\theta^2)^{-(a+\frac{S}{2}+1)} \exp\left(-\frac{b + \frac{\|\theta\|^2}{2}}{\sigma_\theta^2}\right).
\end{aligned}$$

Hence $(\sigma_\theta^2)^{(j+1)} \mid \theta^{(j+1)}, \mathcal{D} \sim \text{Inv-Gamma}\left(a + \frac{S}{2}, b + \frac{\|\theta^{(j+1)}\|^2}{2}\right)$: a distribution from which we can sample directly. Therefore, the MH algorithm is not required for this block. We choose $a, b \approx 0$ in order to specify a nearly uninformative hyperprior of σ_θ^2 : meaning that the prior exerts minimal influence on the posterior $p(\sigma_\theta^2 \mid \theta)$, allowing the observed data to primarily determine the inference. This approach avoids imposing strong assumptions on σ_θ^2 and reflects prior ignorance about its scale.

To compute the MAP estimate $\hat{\theta}^{MAP} = \arg\max_{\theta} p(\theta \mid \mathcal{D})$, we merely identify the mode of the posterior distribution $p(\theta \mid \mathcal{D})$. Likewise, $\hat{\theta}_i^{MAP} = \arg\max_{\theta_i} p(\theta_i \mid \mathcal{D})$, which is merely the mode of the marginal posterior $p(\theta_i \mid \mathcal{D})$, which implicitly integrates out both the remaining components θ_{-i} and the dispersion parameter σ_θ^2 . In this context, each component, θ_i for $i = 1, \dots, S$, has a marginal posterior distribution given by $p(\theta_i \mid \mathcal{D}) = \int p(\theta, \sigma_\theta^2 \mid \mathcal{D}) d\theta_{-i} d\sigma_\theta^2$.

In hierarchical Bayesian modeling, assigning a prior distribution to σ_θ^2 allows the training data to inform the appropriate degree of regularization. Section 4 elucidates $\sigma_\theta^2 \propto \frac{1}{\nu}$, therefore smaller σ_θ^2 values imply stronger shrinkage toward zero, while larger values allow greater flexibility. By endowing σ_θ^2 with a hyperprior, we integrate over uncertainty in the regularization strength rather than fixing it arbitrarily. Consequently, the resulting MAP estimates of the parameters inherently reflect an optimal degree of regularization, effectively “baking in” regularization informed by the training data.

Thus, in a hierarchical Bayesian framework, the training data plays a dual role: it informs the marginal posterior distributions of the model parameters θ , $p(\theta_i \mid \mathcal{D})$ for $i = 1, \dots, S$, while simultaneously guiding the level of regularization through inference on the dispersion parameter σ_θ^2 .

3.2 Adaptive Metropolis-Hastings

In standard MH algorithms, the proposal distribution remains fixed throughout sampling. In contrast, adaptive MCMC methods dynamically tune aspects of the proposal distribution using information gathered during earlier iterations. This adaptive strategy aims to improve mixing by better matching the geometry of the stationary distribution, as expounded in Roberts, Gelman, and Gilks (1997).

We previously stated that the proposal distribution for $\theta \in \mathbb{R}^S$ at current iteration j is $\mathbf{Q}_\theta \sim \mathcal{N}(\theta^{(j)}, \Sigma_j)$ where $\Sigma_j = \sigma_{Q_\theta}^2 \mathbf{I}_S$: a multivariate normal random walk centered at the current state of θ , $\theta^{(j)}$, with isotropic covariance matrix $\sigma_{Q_\theta}^2 \mathbf{I}_S$. During the burn-in phase only, we adapt Σ_j based on previously accepted proposals of θ . By limiting adaptation to the burn-in period, we avoid violating

the diminishing adaptation and ergodicity conditions required for convergence to the correct stationary distribution in fully adaptive MCMC algorithms, as explained by Roberts and Rosenthal (2007). Additionally, we may still utilise Equation 3 to compute acceptance probabilities, α_θ , as our covariance matrices used will all be equivalent after the burn-in phase. That is $\Sigma_j = \Sigma_{j+1}$ for $j > \text{burn-in}$, giving rise to symmetric proposal densities.

Per Haario, Saksman, and Tamminen (2001), the covariance matrix at iteration j is given as:

$$\Sigma_j = \text{Cov} \left(f \left(\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(j)} \right) \right) + \epsilon \mathbf{I},$$

where Σ_j is the empirical covariance matrix of a function of the first j samples and $\epsilon \mathbf{I}_S$ is a small positive-definite matrix (for example, $\epsilon = 10^{-6}$) added to maintain numerical stability. The proposal then becomes $\mathbf{Q}_\theta \sim \mathcal{N}(\theta^{(j)}, s^2 \Sigma_j)$, where $s^2 = \frac{2.38}{S}$. Roberts et al. (1997) showed that for efficient exploration of high-dimensional target distributions, the acceptance rate should be around 0.234, and the optimal step size (variance) scales as $s^2 = \frac{2.38}{S}$.

To reduce the correlation between the proposals, we employ:

$$f \left(\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(j)} \right) = \begin{cases} \left(\theta^{(j-\delta\Delta)}, \theta^{(j-\delta\Delta+\delta)}, \dots, \theta^{(j-\delta\Delta+(\Delta-1)\delta)}, \theta^{(j)} \right), & \text{if } j > \delta\Delta, \\ \left(\theta^{(j-\lfloor \frac{j}{\delta} \rfloor \delta)}, \theta^{(j-\lfloor \frac{j}{\delta} \rfloor \delta + \delta)}, \dots, \theta^{(j-\lfloor \frac{j}{\delta} \rfloor \delta + (\lfloor \frac{j}{\delta} \rfloor - 1)\delta)}, \theta^{(j)} \right), & \text{if } \delta < j \leq \delta\Delta, \\ \left(\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(j)} \right), & \text{if } j \leq \delta, \end{cases} \quad (5)$$

with $\theta^{(1)}$ randomly drawn from the multivariate normal distribution $\mathcal{N}(\mathbf{0}_{S \times 1}, \sigma_{\text{init}}^2 \mathbf{I}_S)$. Here, δ serves as a stride parameter that ensures non-consecutive θ values are used in the empirical covariance calculation, thereby mitigating autocorrelation between proposals. The window size parameter Δ is purely curated to ease computation of the empirical covariances at each iteration (its inclusion is optional in which case one would omit the first case of Equation 5).

3.3 Adaptive scaling

Previously, it was noted that according to the optimal scaling theory of Roberts et al. (1997), the optimal proposal scale for high-dimensional target distributions is given by $s^2 = \frac{2.38}{S}$. However, in order to directly control the average acceptance rate α toward its theoretical optimum of approximately 0.234, we adopt an alternative strategy for scaling the covariance matrix Σ_j during sampling. Specifically, during the burn-in phase, the scaling factor s^2 is adaptively updated at each iteration j according to the observed acceptance rate α_j :

$$(s^2)^{(j+1)} = (s^2)^{(j)} \times \exp(\gamma_j \cdot (\alpha_j - 0.234)),$$

where γ_j is a sequence of diminishing adaptation rates. In accordance with the theoretical guarantees for ergodicity of adaptive MCMC methods established by Roberts and Rosenthal (2007), we choose $\gamma_j = \frac{1}{j^\kappa}$ for some $0.5 < \kappa < 1$. This choice satisfies the standard conditions $\sum_j \gamma_j = \infty$ and $\sum_j \gamma_j^2 < \infty$ as per Roberts and Rosenthal (2007) ensuring that the adaptation is both diminishing and stable. While these conditions are necessary for ensuring ergodicity in fully adaptive MCMC, in our implementation,

adaptation is restricted to the burn-in phase. This pragmatic restriction alleviates these theoretical concerns.

4 The Objective & Regularization

This section demonstrates the necessity for the likelihood function, $p(\mathcal{D} \mid \boldsymbol{\theta})$, to be proportional to (or a monotonically increasing function of) the objective function being maximized. Consider the arbitrary objective, for a given parameter configuration $\boldsymbol{\theta} \in \mathbb{R}^S$, denoted as $\text{argmax}_{\boldsymbol{\theta}} \text{Obj}(\boldsymbol{\theta})$. By including L2 regularization, the L2 penalized objective becomes:

$$\text{argmax}_{\boldsymbol{\theta}} \left(\text{Obj}(\boldsymbol{\theta}) - \nu \sum_{i=1}^S \theta_i^2 \right). \quad (6)$$

Under the assumption that the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ is monotonic increasing with respect to $\text{Obj}(\boldsymbol{\theta})$, and assuming Gaussian priors for our parameters $\boldsymbol{\theta} \in \mathbb{R}^S$, that is $\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \sigma_{\theta}^2 \mathbf{I}_S)$, using Bayes rule we have:

$$\begin{aligned} p(\boldsymbol{\theta} \mid \mathcal{D}) &= \frac{p(\mathcal{D} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})} \\ &\propto p(\mathcal{D} \mid \boldsymbol{\theta})p(\boldsymbol{\theta}) \\ &\propto p(\mathcal{D} \mid \boldsymbol{\theta})\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \sigma_{\theta}^2 \mathbf{I}) \\ &\propto p(\mathcal{D} \mid \boldsymbol{\theta}) \prod_{i=1}^S \mathcal{N}(\theta_i; 0, \sigma_{\theta}^2). \end{aligned}$$

Now taking the log probability of the parameter posterior:

$$\begin{aligned} \log[p(\boldsymbol{\theta} \mid \mathcal{D})] &\propto \log[p(\mathcal{D} \mid \boldsymbol{\theta})] + \sum_{i=1}^S \log[\mathcal{N}(\theta_i; 0, \sigma_{\theta}^2)] \\ &\propto \log[p(\mathcal{D} \mid \boldsymbol{\theta})] - \frac{1}{2\sigma_{\theta}^2} \sum_{i=1}^S \theta_i^2. \end{aligned}$$

Now since we assumed the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ to be proportional to $\text{Obj}(\boldsymbol{\theta})$, that is, $p(\mathcal{D} \mid \boldsymbol{\theta}) \propto \text{Obj}(\boldsymbol{\theta})$, we have:

$$\log[p(\boldsymbol{\theta} \mid \mathcal{D})] \propto \text{Obj}(\boldsymbol{\theta}) - \nu \sum_{i=1}^S \theta_i^2,$$

whereby $\nu \propto \frac{1}{\sigma_{\theta}^2}$ controls the strength of the regularization as in the L2 penalized objective in Equation 6. Clearly, $\text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D})$, is equivalent to the L2 penalized objective in Equation 6, that is, $\hat{\boldsymbol{\theta}}^{MAP} = \text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D}) = \text{argmax}_{\boldsymbol{\theta}} \left(\text{Obj}(\boldsymbol{\theta}) - \frac{1}{2\sigma_{\theta}^2} \sum_{i=1}^S \theta_i^2 \right)$. We may say that the L2 penalization constraint depends on fixed σ_{θ}^2 , the number of coefficients, S , and their magnitudes θ_i , written as function $f(\sigma_{\theta}^2, S, \theta_i)$.

However, within the context of the two-block MCMC scheme, since σ_{θ}^2 is sampled at each iteration, it is no longer the case that the MAP estimate $\hat{\boldsymbol{\theta}}^{MAP}$ is equivalent to $\text{argmax}_{\boldsymbol{\theta}} \left(\text{Obj}(\boldsymbol{\theta}) - \nu \sum_{i=1}^S \theta_i^2 \right)$ with $\nu \propto \frac{1}{\sigma_{\theta}^2}$, as this equivalence only holds when σ_{θ}^2 is fixed across all iterations. Rather, at each iteration j , we induce a shrinkage factor $\nu^{(j)} \propto \frac{1}{(\sigma_{\theta}^2)^{(j)}}$ on the proposal $\boldsymbol{\theta}^*$, as shown in Equation 4. Nevertheless, we may still characterize the amount of regularization associated with the MAP estimates by examining the marginal posterior $p(\sigma_{\theta}^2 \mid \mathcal{D})$. This distribution reflects the posterior uncertainty about the degree of shrinkage, that is, how much regularization the data supports. For example, we may utilise the mean of $p(\sigma_{\theta}^2 \mid \mathcal{D})$ to reflect an “effective ridge penalty” or have the posterior intervals to show uncertainty in the

amount of regularization inferred.

This two-block MCMC framework also results in our marginal prior for $\boldsymbol{\theta}$ to not be Gaussian as before. Since we assume priors: $\boldsymbol{\theta} \mid \sigma_\theta^2 \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \mathbf{I}_S)$ and $\sigma_\theta^2 \sim \text{Inv-Gamma}(a, b)$, we have:

$$\begin{aligned} p(\boldsymbol{\theta}) &= \int_0^\infty p(\boldsymbol{\theta}, \sigma_\theta^2) d\sigma_\theta^2 \\ &= \int_0^\infty p(\boldsymbol{\theta} \mid \sigma_\theta^2) p(\sigma_\theta^2) d\sigma_\theta^2 \\ &= \int_0^\infty (2\pi\sigma_\theta^2)^{-(\frac{S}{2})} \exp\left(-\frac{1}{2\sigma_\theta^2} \|\boldsymbol{\theta}\|^2\right) \frac{b^a}{\Gamma(a)} (\sigma_\theta^2)^{-(a+1)} \exp\left(-\frac{b}{\sigma_\theta^2}\right) d\sigma_\theta^2 \\ &= \frac{b^a}{\Gamma(a)} (2\pi)^{-(\frac{S}{2})} \int_0^\infty (\sigma_\theta^2)^{-(a+\frac{S}{2}+1)} \exp\left(-\frac{1}{2\sigma_\theta^2} (\|\boldsymbol{\theta}\|^2 + 2b)\right) d\sigma_\theta^2. \end{aligned}$$

Using $\int_0^\infty x^{-(\alpha+1)} \exp(-\frac{\beta}{x}) dx = \beta^{-\alpha} \Gamma(\alpha)$, we have the following:

$$\begin{aligned} p(\boldsymbol{\theta}) &= \frac{\Gamma(a + \frac{S}{2})}{\Gamma(a)} (2\pi)^{-(\frac{S}{2})} b^a \left(b + \frac{1}{2} \|\boldsymbol{\theta}\|^2\right)^{-(a+\frac{S}{2})} \\ &= \frac{\Gamma(\frac{2a+S}{2})}{\Gamma(\frac{2a}{2}) (2a)^{\frac{S}{2}} \pi^{\frac{S}{2}} (\frac{b}{a})^{\frac{S}{2}}} \left(1 + \frac{1}{2a} \boldsymbol{\theta}' \left(\frac{b}{a} \mathbf{I}_S\right)^{-1} \boldsymbol{\theta}\right)^{-(\frac{2a+S}{2})}. \end{aligned}$$

Hence, $\boldsymbol{\theta} \sim t_S(\mathbf{0}, \frac{b}{a} \mathbf{I}_S, 2a)$. That is, the marginal prior on $\boldsymbol{\theta}$ is multivariate Student- t with $2a$ degrees of freedom, location vector $\mathbf{0}$, and scale matrix $\frac{b}{a} \mathbf{I}_S$, rather than multivariate Gaussian. Accordingly:

$$\log[p(\boldsymbol{\theta} \mid \mathcal{D})] \propto \text{Obj}(\boldsymbol{\theta}) - \left(a + \frac{S}{2}\right) \log\left(b + \frac{1}{2} \sum_{i=1}^S \theta_i^2\right).$$

Hence $\hat{\boldsymbol{\theta}}^{MAP} = \text{argmax}_{\boldsymbol{\theta}} \left(\text{Obj}(\boldsymbol{\theta}) - \left(a + \frac{S}{2}\right) \log\left(b + \frac{1}{2} \sum_{i=1}^S \theta_i^2\right)\right)$. Constraining $\text{Obj}(\boldsymbol{\theta})$ with a penalty of the form $\log(f(\sum_{i=1}^S \theta_i^2))$, rather than the quadratic penalty $\sum_{i=1}^S \theta_i^2$ that arises under a Gaussian marginal prior, highlights the difference between the two. Both behave similarly for small coefficients θ_i , but the Student- t prior imposes much weaker shrinkage in the tails, applying almost no penalization to large coefficients relative to the Gaussian case. Previously, under a fixed Gaussian prior, the penalization constraint for L2 regularization was a function $f(\sigma_\theta^2, S, \theta_i)$. In the two-block MCMC framework, since σ_θ^2 itself is treated hierarchically via the hyperparameters (a, b) , the penalization constraint is now a function $f(a, b, S, \theta_i)$.

5 The Likelihood

We emphasize that, because our objectives are arbitrary, no concrete assumptions about the data-generating process can be made. In other words, there is no well-defined probabilistic model for the data. Conventionally, the likelihood is derived from the joint density of the data samples $\{x_i\}_{i=1}^n$ given the parameters θ : $p(\mathcal{D} \mid \theta) = f(x_1, \dots, x_n \mid \theta) = \prod_{i=1}^n f(x_i \mid \theta)$ (for x_i i.i.d.). In contrast, in our setting, we only have an objective function $\text{Obj}(\theta)$ that we aim to maximize. Hence, the study utilises likelihoods (more accurately described as pseudo-likelihoods, although we use the terms interchangeably throughout the study) which are purposely fabricated to be proportional to $\text{Obj}(\theta)$, as prescribed in Section 4, to ensure $\hat{\theta}^{MAP} = \text{argmax}_{\theta} \left(\text{Obj}(\theta) - f\left(\sum_{i=1}^S \theta_i^2\right) \right)$. Additionally, the likelihoods are tempered: that is, constructed to allow control over their sharpness with respect to $\text{Obj}(\theta)$. This tempering increases the sensitivity of the likelihood to changes in the objective function values, since $p(\mathcal{D} \mid \theta) \propto \text{Obj}(\theta)$. An increased sharpness may also be interpreted as a greater concentration of mass around the modal regions of their densities. Consequently, it translates into a more pronounced influence of the likelihood on the conditional posterior distribution $p(\theta \mid \sigma_{\theta}^2, \mathcal{D})$, since $p(\theta \mid \sigma_{\theta}^2, \mathcal{D}) \propto p(\mathcal{D} \mid \theta) \cdot p(\theta \mid \sigma_{\theta}^2)$. Within the context of the two-block MCMC framework described in Section 3.1.1, increased likelihood sharpness implies that the sampler becomes more likelihood-driven, thereby diminishing the influence of the prior ratio in exploring the conditional posterior.

In this light, the pseudo-likelihoods employed in the study are not intended to represent rigorous data-generating models. Instead, they are used within the MCMC framework, not to sample from a full posterior, but to facilitate optimization over the parameter space. The MCMC algorithm is thus repurposed as a mode-seeking procedure, targeting the mode of the conditional posterior $p(\theta \mid \sigma_{\theta}^2, \mathcal{D})$. That is to say, to concentrate samples around a dominant mode of $p(\theta \mid \sigma_{\theta}^2, \mathcal{D})$. It suffices that the proposal mechanism is guided by likelihood functions that monotonically increase with the objective of interest, biasing the sampling process toward regions of high posterior density. These regions necessarily correspond to high-likelihood (thus high-valued objective) areas since $p(\theta \mid \sigma_{\theta}^2, \mathcal{D}) \propto p(\mathcal{D} \mid \theta) \cdot p(\theta \mid \sigma_{\theta}^2)$. As such, the requirement for a fully specified likelihood linking the data to the model becomes less critical.

We emphasise now, for the purposes of convergence in MCMC, it is often more desirable for the sampler to be likelihood-driven rather than prior-driven. Equation 4 further illustrates this concept. In the limiting case of a negligible likelihood, it simplifies to:

$$\log(\alpha_{\theta}) = \min \left(\frac{1}{2(\sigma_{\theta}^2)^{(j)}} \left(\|\theta^{(j)}\|^2 - \|\theta^*\|^2 \right), 0 \right),$$

indicating that when the likelihood is approximately flat, that is, $\log(p(\mathcal{D} \mid \theta^*)) \approx \log(p(\mathcal{D} \mid \theta^{(j)}))$, proposals that reduce $\|\theta^*\|^2$ are more likely to be accepted, leading to an indefinite contraction of θ towards zero. Although a balanced contribution between prior and likelihood is theoretically desirable, the practical impossibility of pre-specifying this balance motivates us to ensure that the likelihood is sufficiently dominant in shaping the conditional posterior $p(\theta \mid \sigma_{\theta}^2, \mathcal{D})$.

5.1 Likelihood dominance & implications

In the context of the two-block MCMC scheme as in Section 3.1.1, where sampling is performed from the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}) \propto p(\mathcal{D} \mid \boldsymbol{\theta}) \cdot p(\boldsymbol{\theta} \mid \sigma_\theta^2)$, it is important to note that the mode of conditional $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$ need not coincide exactly with the mode of the marginal $p(\boldsymbol{\theta} \mid \mathcal{D})$. This is because the former is directly influenced by the specific value of σ_θ^2 . That is, σ_θ^2 directly determines the shape of the prior $p(\boldsymbol{\theta} \mid \sigma_\theta^2)$. We note that:

$$\begin{aligned} p(\boldsymbol{\theta} \mid \mathcal{D}) &= \int p(\boldsymbol{\theta}, \sigma_\theta^2 \mid \mathcal{D}) d\sigma_\theta^2 \\ &= \int p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}) \cdot p(\sigma_\theta^2 \mid \mathcal{D}) d\sigma_\theta^2. \end{aligned} \quad (7)$$

This expression implies that the marginal posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$ is a weighted average of the conditionals $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$, where the weights come from the marginal $p(\sigma_\theta^2 \mid \mathcal{D})$. In other words, to obtain $p(\boldsymbol{\theta} \mid \mathcal{D})$, one averages over the uncertainty in σ_θ^2 .

There are two main scenarios where the mode of the marginal posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$ will approximately coincide with the mode of the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$. First, if the marginal $p(\sigma_\theta^2 \mid \mathcal{D})$ has low variance (that is, it is sharply peaked around a single value), then the integral in Equation 7 is dominated by a narrow range of σ_θ^2 , effectively treating σ_θ^2 as nearly constant. Second, if the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ is highly informative (that is, dominates the conditional $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$), then it largely determines the shape of the conditional, making it sharply peaked in roughly the same region of $\boldsymbol{\theta}$ regardless of the specific value of σ_θ^2 . In this case, all the conditional posteriors in the integral of Equation 7 are peaked in the same region, and so the resulting marginal $p(\boldsymbol{\theta} \mid \mathcal{D})$ will also be peaked there.

Furthermore, given the assumption that the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ is sufficiently dominant, we have $\text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}) \approx \hat{\boldsymbol{\theta}}^{MAP} = \text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D})$. Additionally, it may be argued that through likelihood dominance, since $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}) \propto p(\mathcal{D} \mid \boldsymbol{\theta}) \cdot p(\boldsymbol{\theta} \mid \sigma_\theta^2)$, we may have $\text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}) \approx \text{argmax}_{\boldsymbol{\theta}} p(\mathcal{D} \mid \boldsymbol{\theta}) = \hat{\boldsymbol{\theta}}^{MLE}$. Furthermore, due to proportionality $p(\mathcal{D} \mid \boldsymbol{\theta}) \propto \text{Obj}(\boldsymbol{\theta})$, $\hat{\boldsymbol{\theta}}^{MLE} = \text{argmax}_{\boldsymbol{\theta}} \text{Obj}(\boldsymbol{\theta})$. Hence, altogether, we may argue that, given sufficient likelihood dominance, $\text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}) \approx \hat{\boldsymbol{\theta}}^{MAP} = \text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D}) \approx \hat{\boldsymbol{\theta}}^{MLE} = \text{argmax}_{\boldsymbol{\theta}} p(\mathcal{D} \mid \boldsymbol{\theta}) = \text{argmax}_{\boldsymbol{\theta}} \text{Obj}(\boldsymbol{\theta})$. That is, the mode of the conditional $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$ will necessarily lie in a region where $\text{Obj}(\boldsymbol{\theta})$ attains high values. This implies that, when the prior is severely undermined, two-block MCMC merely serves as an optimization technique to seek out $\text{argmax}_{\boldsymbol{\theta}} \text{Obj}(\boldsymbol{\theta})$, where negligible regularization is inferred to the MAP estimates.

6 Experimental Problems with Arbitrary Objectives

This section outlines the reinforcement learning problems with arbitrary objective functions used in the study. Each experimental setup includes a description of how the environment is encoded and how a neural network is employed as a means of control. Within each of the subsequent sections, the study examines the effects of regularization, comparing the performance of a GA and RS under varying pre-specified regularization strengths ν . Subsequently, the results obtained using two-block MCMC are presented, along with a discussion of caveats.

For the first experimental setup, a navigation problem, the study evaluates the impact of different pseudo-likelihood forms on MCMC performance: considering both in-sample and out-of-sample results, as well as the inferred regularization strength. In the second experimental setup, tic-tac-toe, the analysis focuses on how likelihood sharpness and the initial variance parameter, σ_{Init}^2 , influence MCMC performance. Finally, the third experimental setup of S-17 blackjack is used to investigate the performance of a hybrid algorithm relative to the original two-block MCMC approach employed throughout the study.

Furthermore, a neural network with two hidden layers and three nodes per layer was used (see Appendix D), with hyperbolic tangent activation functions in both layers. The network was intentionally kept small to control model complexity, since the aim of this study is to examine how regularization behaves and to assess the shortcomings of two-block MCMC, rather than to maximize predictive performance. The 3-3 structure provides enough nonlinearity to represent reinforcement learning policies while avoiding excessive flexibility that could confound the interpretation of inferred regularization. The hyperbolic tangent function was chosen because it is smooth, zero-centred, and bounded, which helps keep the learning dynamics stable and makes the effects of shrinkage-based regularization easier to interpret. As such, these architectural choices are meant to provide a simple and stable baseline that isolates the mechanisms under investigation.

6.1 A navigation problem

In what follows, the study employs a navigation problem to assess how different pseudo-likelihood formulations influence both in-sample and out-of-sample performance, as well as the level of regularization inferred. Specifically, we construct a navigation task in which a neural network serves as the control policy governing movement for a drone that needs to escape a “forest” of moving obstacles. This environment is adapted from a simpler configuration presented in Pienaar (2024) and is deliberately chosen to assess MCMC performance using a continuous environment (the actions of the drone agent occur in a bounded space which is encoded as continuous values).

Formally, consider T drones navigating within a two-dimensional circular arena. The arena is defined as the annular region bounded by an inner radius R_{inner} and an outer radius R_{outer} . Each drone aims to escape this arena within K steps, where each step has a fixed length δ . A drone is considered to have crashed if it comes within a distance R_{crash} of any of the J orbiting obstacles. Upon crashing, the drone ceases further navigation. An illustration of the navigation problem is displayed in Figure 1.

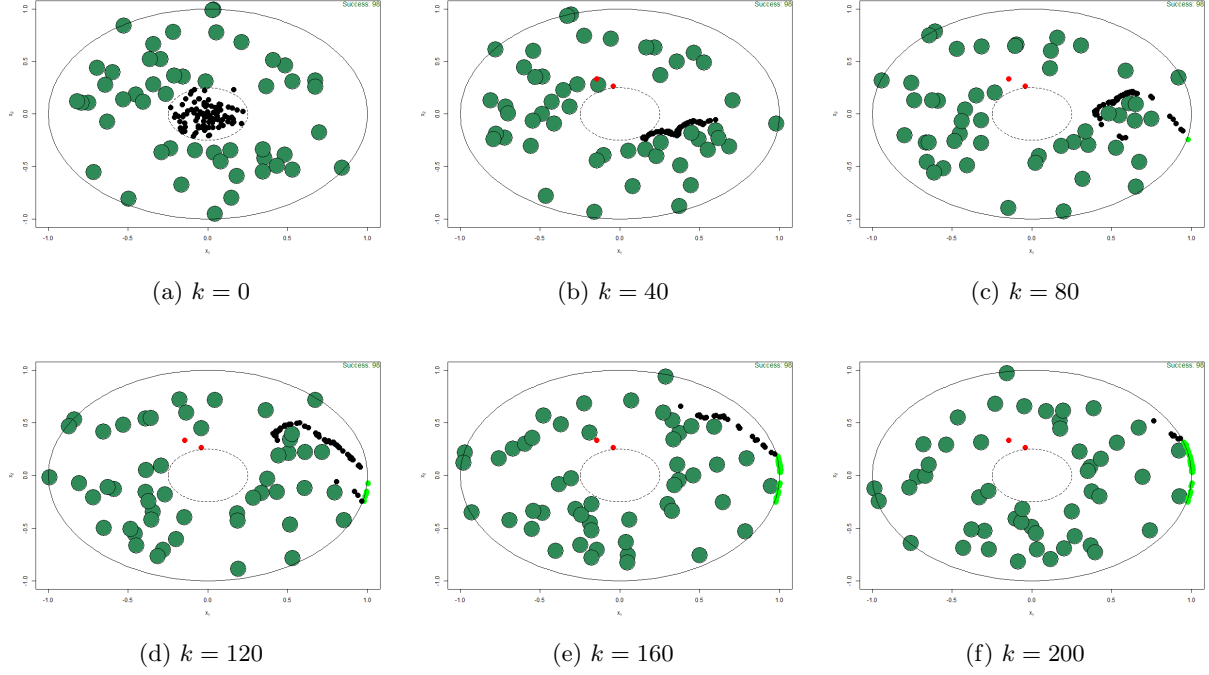


Figure 1: Illustration of the navigation problem: T drones (black dots) attempting to escape the annular arena while avoiding the J orbiting obstacles (larger green disks) (snapshots of $\hat{\theta}_{\nu=4 \times 10^{-6}}^{\text{GA},(\text{II})}$ evaluated on an arbitrary out-of-sample initialization).

6.1.1 Encoding

We represent the annular arena by the region $[R_{\text{inner}} \cos(\theta), R_{\text{outer}} \cos(\theta)] \times [R_{\text{inner}} \sin(\theta), R_{\text{outer}} \sin(\theta)] \in \mathbb{R}^2$ for $\theta \in [0, 2\pi)$. The J obstacles orbit the arena with unique angular frequencies, ω_j , and orbital radii, r_j where the j^{th} obstacle has a coordinate at the k^{th} iteration: $\mathbf{o}_j^{(k)} = (o_{j1}^{(k)}, o_{j2}^{(k)}) : j = 1, 2, \dots, J$ where $o_{j1}^{(k)} = r_j \cos(\omega_j \cdot k + \phi_j)$ and $o_{j2}^{(k)} = r_j \sin(\omega_j \cdot k + \phi_j)$ for angular frequencies, $\omega_j = 2\pi U(\frac{1}{P_{\text{upper}}}, \frac{1}{P_{\text{lower}}})$, phase shifts, $\phi_j = 2\pi U(0, 1)$ and orbital radii, $r_j = U(R_{\text{inner}} + R_{\text{crash}}, R_{\text{outer}})$. We draw the initial T drones at coordinates $\mathbf{x}_t^{(0)} = (x_{t1}^{(0)}, x_{t2}^{(0)}) : t = 1, 2, \dots, T$ where $x_{t1}^{(0)} = r_t \cos(\theta_t)$ and $x_{t2}^{(0)} = r_t \sin(\theta_t)$ for $\theta_t = 2\pi U(0, 1)$ and $r_t = U(0, R_{\text{inner}})$. Figure 2 provides a schematic illustration of the annular arena and associated notation.

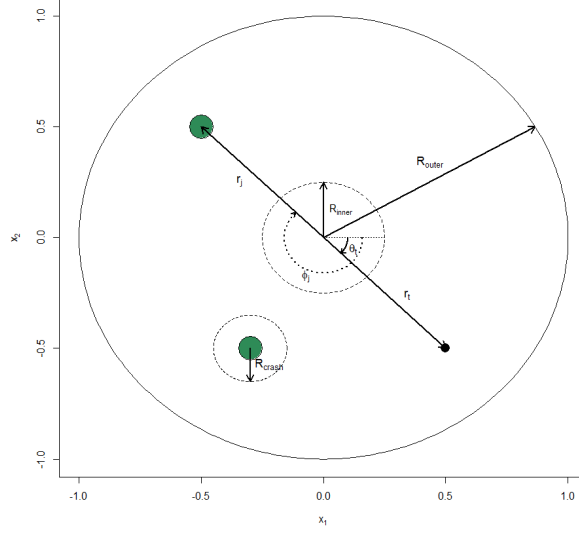


Figure 2: Diagram illustrating the mathematical notation of the navigation problem (noting an orbiting obstacle as a large green disk and drone as a black dot).

6.1.2 The game state

After each movement made by an obstacle, the game state must be evaluated to determine whether a collision with a drone has occurred. Similarly, following each drone movement, the game state must again be assessed to establish whether the drone has either succeeded or failed. A failure is defined as the drone either (i) coming within a distance R_{crash} of an obstacle, signifying a crash, or (ii) failing to reach the boundary of the circular arena within K permitted steps. A success is defined as the drone reaching the edge of the arena. Consequently, the game state is evaluated twice during each time step k : once after the obstacles move and once after the drones move. If neither a success nor failure condition is met, the drone proceeds to the next step. We encode this as:

$$s_t^k = \begin{cases} +1, & \text{if } \sqrt{x_{t1}^{(k)} + x_{t2}^{(k)}} \geq R_{outer}, \\ -1, & \text{if } \exists j \in \{1, 2, \dots, J\} \text{ such that } \|\mathbf{x}_t - \mathbf{o}_j\| \leq R_{crash}, \\ 0, & \text{otherwise.} \end{cases}$$

where s_t^k denotes the games status for drone t at step k while $k \leq K$. Now if $s_t^k \in \{-1, 1\}$ for $k \leq K$ then the navigation for that particular t^{th} drone ends. Otherwise, the game continues until $k > K$ and the status is recorded as -1 for that particular t^{th} drone.

6.1.3 Control: Drone movement

During each step, we must move each drone by changing $(x_{t1}^{(k)}, x_{t2}^{(k)})$ by at most δ for at most K steps. Hence, the game-updating equation follows:

$$\mathbf{x}_t^{(k+1)} = \mathbf{x}_t^{(k)} + \mathbf{ct}(\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, \boldsymbol{\theta})\delta_t,$$

where $\mathbf{ct}(\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, \boldsymbol{\theta}) \in [-1, 1]^2$ is some control vector for some parameter configuration $\boldsymbol{\theta} \in \mathbb{R}^S$. Now,

the interface between a model and the navigation is undergone through this control vector for which

$$\mathbf{ct} : (\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, \boldsymbol{\theta}) \rightarrow \mathbf{model} \left(\boldsymbol{\Omega} \left(\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, \phi(\boldsymbol{\theta}_1) \right), \boldsymbol{\theta}_2 \right) \xrightarrow{\sigma_L(\cdot)} \left(\hat{x}_{t1}^{(k)}, \hat{x}_{t2}^{(k)} \right) \in [-1, 1]^2,$$

where $\boldsymbol{\theta}_1 \subset \boldsymbol{\theta}, \boldsymbol{\theta}_2 \subseteq \boldsymbol{\theta}$ and $\boldsymbol{\theta}$ is fixed throughout the navigation period and all $t = 1, 2, \dots, T$ drones move according to this $\boldsymbol{\theta}$. Additionally, $(\hat{x}_{t1}^{(k)}, \hat{x}_{t2}^{(k)})$ represents the additions to the x and y coordinate of the t^{th} drone for the k^{th} step such that $\mathbf{x}_t^{(k+1)} = \mathbf{x}_t^{(k)} + \hat{\mathbf{x}}_t^{(k)} \delta_t$. Furthermore, when the t^{th} drone is deemed terminal, we set $\delta_t = 0$, resulting in no positional update, that is, $\mathbf{x}_t^{(k+1)} = \mathbf{x}_t^{(k)}$ for the k^{th} iteration in which terminality is detected. Otherwise, $\delta_t = \delta$. In other words, if the t^{th} drone has reached a terminal state, its coordinates $\mathbf{x}_t^{(k)}$ are still passed through to the control vector $\mathbf{ct}$. Now in the framework of using a neural network as our model, we define $\boldsymbol{\Omega} : \left(\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, \phi(\boldsymbol{\theta}_1) \right) \rightarrow \mathbf{a}(t)^0 \in \mathbb{R}^{d_0}$ which signifies the vector of input nodes for the t^{th} drone, where ϕ is a user-defined mapping. Additionally, $\boldsymbol{\theta}_2$ are the weights and biases of the neural network, $\mathbf{w} \in \mathbb{R}^R$, where $R \leq S$, and $\sigma_L(\cdot)$ represents the hyperbolic tangent activation function applied at the output layer, allowing movement of the t^{th} drone in all directions.

6.1.3.1 Feature engineering For effective manoeuvring of a drone around “clumps” of obstacles, we propose to set the neural net input, $\boldsymbol{\Omega} \left(\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, \phi(\boldsymbol{\theta}_1) \right)$, for the t^{th} drone to merely the sum of all reciprocal Euclidean distances between said drone and all obstacles within a specified radius $R_{detection}$ of the drone. Indeed, this approach has two key benefits: first, the input value increases when there are many obstacles within the drone’s vicinity. Second, the input value becomes larger the closer an obstacle is to the drone. In this way, the input can quantify the level of caution the t^{th} drone should exercise. This is actually critical for the generalization ability of the agent, since we want the agent to not simply learn the coordinates of objects in a particular presentation of a game, but some measure of spatial proximity to objects in general.

Hence we set $\phi(\boldsymbol{\theta}_1) = R_{detection}$ and $\boldsymbol{\Omega} : \left(\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, R_{detection} \right) \rightarrow \mathbf{a}(t)^0 \in \mathbb{R}^{d_0=1}$, that is, a single input node is created for each t^{th} drone with:

$$\boldsymbol{\Omega}(\mathbf{x}_t^{(k)}, \mathbf{o}_j^{(k)}, R_{detection}) = \begin{cases} \sum_{j^*=1}^{J^*} \frac{1}{\|\mathbf{x}_t^{(k)} - \mathbf{o}_{j^*}\| - R_{crash}} & \text{if } J^* > 0, \\ \frac{1}{\min_{1 \leq j \leq J} \|\mathbf{x}_t^{(k)} - \mathbf{o}_j\| - R_{crash}} & \text{if } J^* = 0. \end{cases} \quad (8)$$

where we subtract R_{crash} here as $\|\mathbf{x}_t^{(k)} - \mathbf{o}_{j^*}\| - R_{crash}$ represents the true distance the t^{th} drone may come to an obstacle without crashing. Now, $\|\mathbf{x}_t^{(k)} - \mathbf{o}_{j^*}\|$ denotes the euclidean distance between the t^{th} drone and the j^{*th} obstacle within radius $R_{detection}$ of the drone and there exists $J^* \leq J$ obstacles within the t^{th} drone’s “detection” circle. Furthermore, if $J^* = 0$ (that is, if there are no trees within radius $R_{detection}$ from the t^{th} drone) we return the reciprocal of the minimum euclidean distances between the t^{th} drone and all the J obstacles. We observe that, during the optimization process, if it is estimated that $\hat{R}_{detection} < R_{crash}$, the first case in Equation 8 becomes futile in controlling the movement of non-terminal drones. For example, if $J^* = 1$ (that is, if there is a single obstacle within radius $R_{detection}$ of the t^{th} drone) then the drone would have already collided with that obstacle, rendering it terminal. Consequently, in such scenarios, the only viable control feedback for non-terminal drones is obtained from the second case of Equation 8, which governs behavior when no obstacles are detected within $R_{detection}$ of the t^{th} drone.

We refer to this configuration, with the entirety of Equation 8 dictating the input node, as **model** with input node $\overset{(I)}{a}(t)^0$ and weights $\boldsymbol{\theta}_2^{(I)}$ embedded in $\boldsymbol{\theta}^{(I)}$. Now to ensure a meaningful $R_{detection}$ is estimated

during the optimization process, we introduce a second model, namely **model**^(II), which replaces the second case in Equation 8 with 0 if $J^* = 0$, to create input node $a^{(II)}(t)^0$ with weights $\theta_2^{(II)}$ as a subset of $\theta^{(II)}$.

Lastly, since we assume Gaussian priors for our parameters $\theta \in \mathbb{R}^S$, that is $\theta \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \mathbf{I}_S)$, where for our case we have $\theta = [\theta_1, \theta_2]'$ for which $\theta_1 = \theta_1 \in \mathbb{R}$ and $\theta_2 = \mathbf{w} \in \mathbb{R}^R$, we must ensure $\phi(\theta_1 = \theta_1) = R_{\text{detection}} \geq 0$. Accordingly, we define $\phi = \phi$ as a logistic function with a scaling factor sf to ensure that the radius remains positive while being constrained by an upper bound equal to sf . More formally, define $\phi = \phi$ as a mapping $\phi : \theta_1 \xrightarrow{\frac{sf}{1+\exp(-\theta_1)}} R_{\text{detection}}$, where the upper bound sf is a user-specified parameter determined by the geometry of the problem.

6.1.4 An arbitrary objective

Consider now an arbitrary objective where, for a given parameter configuration $\theta \in \mathbb{R}^S$ and T drones, we record the relative frequency of successes. That is, we evaluate the arbitrary objective:

$$\begin{aligned} \operatorname{argmax}_{\theta} \text{Obj}(\theta) &= \operatorname{argmax}_{\theta} \frac{1}{T} \sum_{t=1}^T \mathbb{I}(s_t^K(\theta) = +1) \\ &= \operatorname{argmax}_{\theta} \frac{1}{T} k(\theta). \end{aligned}$$

where $s_t^K(\theta)$ represents the success status of the t^{th} drone after K steps, dependent on our parameter vector θ . Again, we distinguish this as an arbitrary objective since it is not differentiable with respect to the unknown parameter vector and we're imposing a simple goal-based objective of "win as often as possible", hoping that maximizing said objective corresponds to the behaviour we want to illicit.

By including L2 regularization, the L2 penalized objective becomes:

$$\operatorname{argmax}_{\theta} \left(\frac{1}{T} k(\theta) - \nu \sum_{i=1}^S \theta_i^2 \right). \quad (9)$$

6.1.5 Likelihoods

The subsequent section introduces three distinct likelihood formulations, more accurately described as pseudo-likelihoods as previously explained in Section 5. The former two are loosely motivated by the fact that for count-based objectives (i.e., objectives quantifying the number of successes), it is natural to construct a likelihood based on the binomial distribution, which defines a probability mass function for a fixed success probability and number of trials. However, as discussed in Section 4, it is necessary that these likelihoods be proportional to the objective function, that is, $p(\mathcal{D} \mid \theta) \propto \text{Obj}(\theta)$. Accordingly, we restructure these likelihoods to ensure they align with this notion. Additionally, Section 5 indicates that our likelihoods are tempered; hence, we incorporate a likelihood sharpness parameter $\beta \in \mathbb{R}^+$ to enable control over this.

More broadly, the pseudo-likelihood formulations in this section inherently exhibit differing sharpness (without explicitly inducing sharpness through the parameter β). Therefore, this section aims to facilitate a discussion on how different likelihoods, with inherently different sharpnesses, can induce distinct behaviours within the two-block MCMC framework.

6.1.5.1 Binomial-based likelihood We model the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ as the probability of observing $k(\boldsymbol{\theta}) = \sum_{t=1}^T \mathbb{I}(s_t^K(\boldsymbol{\theta}) = +1) \leq T$ successes for T total drones given a success probability of $p_{\boldsymbol{\theta}} \approx \frac{k(\boldsymbol{\theta})}{T}$, that is $k(\boldsymbol{\theta}) \sim \text{BIN}(T, p_{\boldsymbol{\theta}})$ for $k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$. We note that the number of successes $k(\boldsymbol{\theta})$ is dependent on the specific parameter $\boldsymbol{\theta}$ used, and $p_{\boldsymbol{\theta}}$ is obtained empirically, hence, rendering our likelihood a simulation-based likelihood as follows:

$$\begin{aligned} p(\mathcal{D} \mid \boldsymbol{\theta}) &= \binom{T}{k(\boldsymbol{\theta})} p_{\boldsymbol{\theta}}^{k(\boldsymbol{\theta})} (1 - p_{\boldsymbol{\theta}})^{T - k(\boldsymbol{\theta})} \\ &= \binom{T}{k(\boldsymbol{\theta})} \left(\frac{k(\boldsymbol{\theta})}{T} \right)^{k(\boldsymbol{\theta})} \left(1 - \left(\frac{k(\boldsymbol{\theta})}{T} \right) \right)^{T - k(\boldsymbol{\theta})}. \end{aligned} \quad (10)$$

We emphasize that this is not a classical likelihood function in the strictest sense, as both the observed outcome $k(\boldsymbol{\theta})$ and the estimated success probability $p_{\boldsymbol{\theta}}$ are derived from the same data. This creates a circularity in which the probability of the outcome, $\boldsymbol{\theta}$, is conditioned on a parameter, $p_{\boldsymbol{\theta}}$, that itself depends on the outcome. We view this pseudo-likelihood as a proxy for how well $\boldsymbol{\theta}$ explains the observed outcomes by quantifying the plausibility of $k(\boldsymbol{\theta})$ under a binomial model with $p_{\boldsymbol{\theta}} \approx \frac{k(\boldsymbol{\theta})}{T}$.

Now, one can not assume that $p(\mathcal{D} \mid \boldsymbol{\theta})$ in Equation 10 is monotonic increasing with respect to the number of successes $k(\boldsymbol{\theta})$. In fact, $p(\mathcal{D} \mid \boldsymbol{\theta})$ in Equation 10 exhibits a sole minimum when $k(\boldsymbol{\theta}) = \frac{T}{2}$, Theorem 1 in conjunction with Lemma 1 clarifies this.

Lemma 1. $\frac{1}{n} > \psi^{(1)}(n+1)$ for $n > 0$.

Proof. By Guo, Qi, Zhao, and Luo (2015), we know $\psi^{(1)}(x) < \frac{1}{x+\frac{1}{2}} + \frac{1}{x^2}$ for $x > 0$. Hence $\psi^{(1)}(n+1) < \frac{1}{(n+1)+\frac{1}{2}} + \frac{1}{(n+1)^2} < \frac{1}{n+1} + \frac{1}{(n+1)^2}$ for $n > 0$. We show $\frac{1}{n} > \psi^{(1)}(n+1)$ by contradiction. Consider:

$$\begin{aligned} \frac{1}{n} &\leq \frac{1}{n+1} + \frac{1}{(n+1)^2} \\ \therefore 1 &\leq \frac{n^2 + 2n}{n^2 + 2n + 1} \\ \therefore 1 &\leq 0 \end{aligned}$$

Which is a contradiction, implying $\frac{1}{n} > \frac{1}{n+1} + \frac{1}{(n+1)^2} > \psi^{(1)}(n+1)$ for $n > 0$. $\square$

Theorem 1. $f(x) = \binom{T}{x} \left(\frac{x}{T}\right)^x \left(1 - \frac{x}{T}\right)^{T-x}$ exhibits a sole minimum at $x = \frac{T}{2}$ for $x \in [0, T]$ with $T \in (0, \infty)$, with $f(x)$ being symmetric about $x = \frac{T}{2}$.

Proof. Since $\lim_{x \rightarrow 0^+} f(x) = 1$ and $\lim_{x \rightarrow T^-} f(x) = 1$, we know $f(x)$ is continuous on the closed interval $[0, T]$. Consider:

$$l(x) = \log[f(x)] = \log(T!) - \log(x!) - \log[(T-x)!] + x \log\left(\frac{x}{T}\right) + (T-x) \log\left(\frac{T-x}{T}\right).$$

Using $\log(n!) = \log[\Gamma(n+1)]$ we have:

$$l(x) = \log[\Gamma(T+1)] - \log[\Gamma(x+1)] - \log[\Gamma(T-x+1)] + x \log\left(\frac{x}{T}\right) + (T-x) \log\left(\frac{T-x}{T}\right).$$

Taking the first derivative with respect to x and using the digamma function $\psi(n) = \frac{d}{dn} \log [\Gamma(n)]$:

$$l'(x) = -\psi(x+1) + \psi(T-x+1) + \log\left(\frac{x}{T-x}\right).$$

Since $l'(\frac{T}{2}) = -\psi(\frac{T}{2}+1) + \psi(\frac{T}{2}+1) + \log(1) = 0$, we know $f(x)$ has a critical point at $x = \frac{T}{2}$. We now show $f''(x) > 0$ on the interval $(0, T)$ implying $f(x)$ is convex and has most one minimum on this interval. We use the trigamma function $\frac{d\psi(n)}{dn} = \psi^{(1)}(n)$ and consider:

$$l''(x) = -\psi^{(1)}(x+1) - \psi^{(1)}(T-x+1) + \frac{1}{x} + \frac{1}{T-x}.$$

By Lemma 1, since $\frac{1}{x} > \psi^{(1)}(x+1)$ and $\frac{1}{T-x} > \psi^{(1)}(T-x+1)$, we have $l''(x) > 0$ on $x \in (0, T)$. Additionally, since:

$$\begin{aligned} f(T-x) &= \binom{T}{T-x} \left(\frac{T-x}{T}\right)^{T-x} \left(1 - \frac{T-x}{T}\right)^{T-(T-x)} \\ &= \binom{T}{x} \left(\frac{x}{T}\right)^x \left(1 - \frac{x}{T}\right)^{T-x} \\ &= f(x). \end{aligned}$$

we have shown that $f(x)$ is symmetric about $x = \frac{T}{2}$. $\square$

To ensure a monotonic increasing likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ on interval $k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$, we propose artificially changing the structure of the likelihood given in Equation 10. Since Theorem 1 implies that Equation 10 is monotonic decreasing on $k(\boldsymbol{\theta}) \in \{0, 1, \dots, \frac{T}{2}\}$, after which being monotonic increasing on $k(\boldsymbol{\theta}) \in \{\frac{T}{2}, \frac{T}{2} + 1, \dots, T\}$, to enforce monotonicity over the entire interval, we define a piecewise function $h(x)$ as follows:

$$h(x) = \begin{cases} g(x), & \text{if } x < \frac{T}{2}, \\ f(x), & \text{if } x \geq \frac{T}{2}. \end{cases}$$

where $f(x) = \binom{T}{x} (\frac{x}{T})^x (1 - \frac{x}{T})^{T-x}$ for $x \in \{0, 1, \dots, T\}$ represents the original binomial-based likelihood with $g(x) = ax$ merely being a linear function where scaling factor $a = \frac{2}{T} f(\frac{T}{2})$ has a dual function of ensuring continuity at $x = \frac{T}{2}$ as well as ensuring $g(x) \geq 0$ for $x \in \{0, 1, \dots, \frac{T}{2}\}$. Additionally, one could make the argument that any monotonic increasing function $g(x)$ could be used, satisfying $g(x) \geq 0$ for $x \in \{0, 1, \dots, \frac{T}{2}\}$ as well as the continuity constraint of $g(\frac{T}{2}) = f(\frac{T}{2})$. This notion is further elaborated on in Section 6.1.5.1.1.

To allow for sharper likelihoods, specifically, to increase the rate of change of $h(x)$ over the domain, we introduce a sharpness parameter $\beta \in \mathbb{R}^+$. This parameter modulates the steepness of the likelihood function by exponentiating it directly. That is, we define $[h(x)]^\beta$, where larger values of β yield a more pronounced increase in likelihood as a function of $k(\boldsymbol{\theta})$, effectively sharpening the likelihood surface, further elucidated in Section 6.1.5.1.2. We exponentiate $h(x)$ by β , rather than scale it via $\beta \cdot h(x)$, because our focus is on amplifying the steepness of the log-likelihood, which plays a central role in acceptance probability computations (as seen in Equation 4). Specifically, taking the logarithm of $[h(x)]^\beta$ yields $\beta \cdot \log[h(x)]$, thereby linearly scaling the log-likelihood. Alternatively, the parameter β can be interpreted

as a means of amplifying the likelihood ratio in Equation 3, yielding the modified expression $\left(\frac{p(\mathcal{D}|\boldsymbol{\theta}^*)}{p(\mathcal{D}|\boldsymbol{\theta}^{(j)})}\right)^\beta$. Increasing β makes the Markov chain more inclined to accept proposed solutions $\boldsymbol{\theta}^*$ that yield higher objective values, given the proportionality $p(\mathcal{D} | \boldsymbol{\theta}) \propto \text{Obj}(\boldsymbol{\theta})$, in effect, making the MCMC sampler more likelihood-driven. Thus, the prior ratio plays a reduced role in the acceptance step.

Furthermore, for $\boldsymbol{\theta} \in \mathbb{R}^S$, $k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$ and sharpness $\beta \in \mathbb{R}^+$, we have our new likelihood as:

$$p(\mathcal{D} | \boldsymbol{\theta}) = [h(k(\boldsymbol{\theta}))]^\beta. \quad (11)$$

Clearly, $\text{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|\mathcal{D})$, would now be equivalent to maximizing the L2 penalized objective in Equation 9 due to the monotonic increasing nature of $h(x)$ in Equation 11, as displayed in Figure 3 and compared to the original likelihood of Equation 10, in Figure 4. Additionally, Figure 5 illustrates the scaled log-likelihood, $\beta \cdot \log[h(x)]$, highlighting how increasing the sharpness parameter β amplifies the curvature of the log-likelihood. This results in a steeper surface, thereby enhancing the sensitivity of the likelihood to changes in $x = k(\boldsymbol{\theta})$.

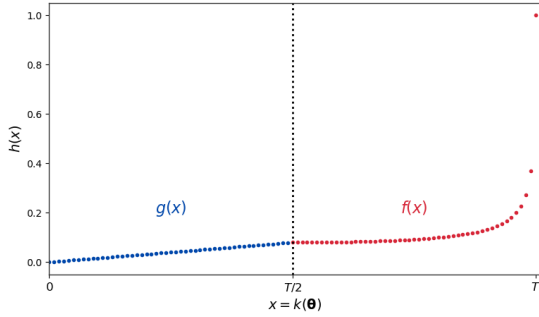


Figure 3: $h(x)$ on interval $[0, 1]$ for $x = k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$.

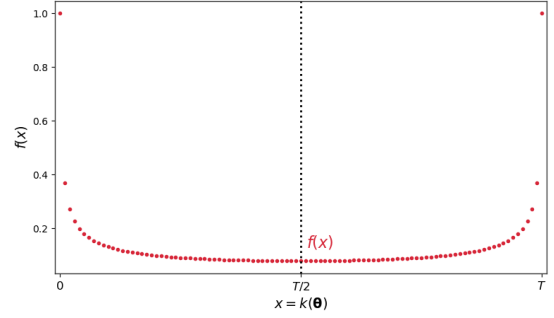


Figure 4: $f(x)$ on interval $[0, 1]$ for $x = k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$

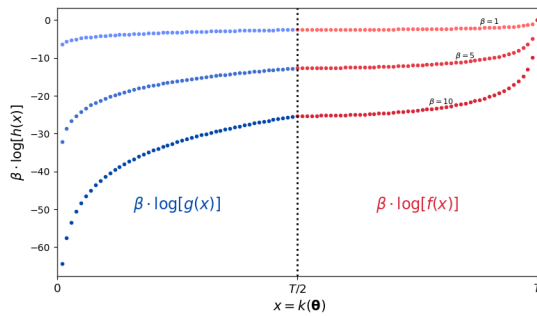


Figure 5: $\beta \cdot \log[h(x)]$ for $x = k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$ for various β .

6.1.5.1.1 Rebuttals against different $g(x)$ forms: It is important to re-emphasize that the modified likelihood consisting of $h(k(\boldsymbol{\theta}))$ is constructed primarily to facilitate efficient sampling in an MCMC context, where the goal is to identify the mode of the conditional posterior, $p(\boldsymbol{\theta} | \sigma_\theta^2, \mathcal{D})$. Since the mode necessarily lies in the high-likelihood (high-valued objective) region (due to $p(\boldsymbol{\theta} | \sigma_\theta^2, \mathcal{D}) \propto p(\mathcal{D} | \boldsymbol{\theta}) \cdot p(\boldsymbol{\theta} | \sigma_\theta^2)$), specifically in the upper half of the domain $k(\boldsymbol{\theta}) \in \{\frac{T}{2}, \frac{T}{2} + 1, \dots, T\}$, where the

original likelihood function $f(k(\boldsymbol{\theta}))$ is already monotonic increasing, our primary interest lies in accurately sampling from this latter half. The lower half $k(\boldsymbol{\theta}) \in \{0, 1, \dots, \frac{T}{2}\}$, where the original likelihood is decreasing, serves primarily as a transitional region that we wish to exit efficiently during sampling. As such, the choice of $g(k(\boldsymbol{\theta}))$ on this interval can be quite flexible: it need only be monotonic increasing and continuous at $k(\boldsymbol{\theta}) = \frac{T}{2}$. For this reason, we adopt a simple linear form $g(k(\boldsymbol{\theta})) = a \cdot k(\boldsymbol{\theta})$, which accelerates the sampler’s movement through low-likelihood regions and thus enhances convergence toward the high-likelihood regions that contribute meaningfully to the conditional posterior mode. This design choice is well justified given that our objective is not full posterior sampling, but rather efficient localization of the posterior mode (achieved by deliberately concentrating samples around it).

6.1.5.1.2 Sharpness $\beta \in \mathbb{R}^+$: In the context of the two-block MCMC framework, wherein a pseudo-likelihood is modulated by a sharpness parameter β , this parameter plays a critical role in shaping the behaviour of the Markov chain by governing the peakedness of the likelihood. Since β exponentiates $p(\boldsymbol{\theta} \mid \mathcal{D})$, not only is the likelihood’s rate of change across the domain increased, but the density itself becomes sharper in the sense that mass is more strongly concentrated around its mode. This is because exponentiating the likelihood amplifies higher values and suppresses lower ones, thereby steepening the posterior landscape. Specifically, a high value of β accentuates the pseudo-likelihood, yielding a sharply peaked conditional posterior $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$, given that $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}) \propto p(\mathcal{D} \mid \boldsymbol{\theta}) \cdot p(\boldsymbol{\theta} \mid \sigma_\theta^2)$, and thus concentrates samples around the mode. This enhances exploitation by focusing the chain on high-likelihood regions. However, in the presence of multimodality, a large β may cause the chain to become trapped in a single dominant mode, hindering exploration of other dominant modes, as proposals that move away from the current region can receive extremely low acceptance probabilities α_θ . Conversely, a low β reduces the sharpness of the likelihood, flattening the conditional posterior and enabling the chain to move more freely across the parameter space, thereby promoting exploration of multiple modes at the expense of slower convergence to high-likelihood regions. The choice of β therefore embodies a trade-off between exploration and exploitation in the MCMC process.

An alternative perspective on the role of the sharpness parameter β is that it controls the number and prominence of modes in the conditional posterior. For example, a low β flattens the likelihood, allowing multiple regions of the parameter space to “compete” for conditional posterior mass and thereby inducing multimodality. In contrast, a high β sharpens the conditional posterior, collapsing it onto dominant modes and potentially suppressing minor alternatives.

6.1.5.2 Beta-based likelihood Invoking the identity $n! = \Gamma(n+1)$ for $n \in \mathbb{W}$, Equation 10 may be written as such:

$$p(\mathcal{D} \mid \boldsymbol{\theta}) = \frac{\Gamma(T+1)}{\Gamma(k(\boldsymbol{\theta})+1) \cdot \Gamma(T-k(\boldsymbol{\theta})+1)} \left(\frac{k(\boldsymbol{\theta})}{T}\right)^{k(\boldsymbol{\theta})} \left(1 - \left(\frac{k(\boldsymbol{\theta})}{T}\right)\right)^{T-k(\boldsymbol{\theta})},$$

for $k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$. Yet, since we would like to model the proportion of successes $\frac{1}{T}k(\boldsymbol{\theta}) \in [0, 1]$, yet retain the constraint of the original likelihood $p(\mathcal{D} \mid \boldsymbol{\theta}) \in [0, 1]$ (better seen as $f(x)$ in Figure 4), we

model the likelihood as such:

$$p(\mathcal{D} \mid \boldsymbol{\theta}) = \frac{\Gamma(2)}{\Gamma(\frac{1}{T}k(\boldsymbol{\theta}) + 1) \cdot \Gamma(2 - \frac{1}{T}k(\boldsymbol{\theta}))} \left(\frac{k(\boldsymbol{\theta})}{T}\right)^{\frac{1}{T}k(\boldsymbol{\theta})} \left(1 - \left(\frac{k(\boldsymbol{\theta})}{T}\right)\right)^{1 - \frac{1}{T}k(\boldsymbol{\theta})}, \quad (12)$$

for $\frac{1}{T}k(\boldsymbol{\theta}) \in \{0, \frac{1}{T}, \frac{2}{T}, \dots, 1\}$. Rescaling Equation 10, using the proportion of successes, $\frac{1}{T}k(\boldsymbol{\theta}) \in [0, 1]$, instead of the number of successes, $k(\boldsymbol{\theta})$, resembles a binomial distribution for a single trial, $T = 1$. Now, as before, since Theorem 1 implies that Equation 12 is monotonic decreasing on $\frac{1}{T}k(\boldsymbol{\theta}) \in [0, \frac{1}{2}]$, after which being monotonic increasing on $\frac{1}{T}k(\boldsymbol{\theta}) \in [\frac{1}{2}, 1]$, to enforce monotonicity over the entire interval, we define a piecewise function $h(x)$ as follows:

$$h(x) = \begin{cases} g(x), & \text{if } x < \frac{1}{2}, \\ f(x), & \text{if } x \geq \frac{1}{2}, \end{cases}$$

where $f(x) = \frac{\Gamma(2)}{\Gamma(x+1)\Gamma(2-x)}(x)^x(1-x)^{1-x}$ for $x \in [0, 1]$ represents the original likelihood in Equation 12. Furthermore, we note that $f(x)$ is proportional to the density function of a Beta($\alpha, 3 - \alpha$) distribution scaled by $\frac{1}{2}$ where $\alpha = x + 1$. Now $g(x) = ax$ is a linear function with scaling factor $a = 2f(\frac{1}{2})$. Furthermore, for $\boldsymbol{\theta} \in \mathbb{R}^S$, $\frac{1}{T}k(\boldsymbol{\theta}) \in [0, 1]$ and sharpness $\beta \in \mathbb{R}^+$, we have our new likelihood as:

$$p(\mathcal{D} \mid \boldsymbol{\theta}) = \left[h\left(\frac{1}{T}k(\boldsymbol{\theta})\right) \right]^\beta. \quad (13)$$

Figure 6 illustrates the re-scaled likelihood function (beta-based) from Equation 13, while Figure 7 displays the original likelihood from Equation 12. Additionally, Figure 8 illustrates the scaled log-likelihood, $\beta \cdot \log[h(x)]$ for various β .

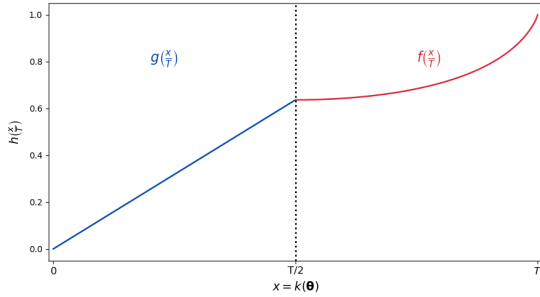


Figure 6: $\beta \cdot h\left(\frac{x}{T}\right)$ on interval $[0, \beta]$ for $x = k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$ for various β .

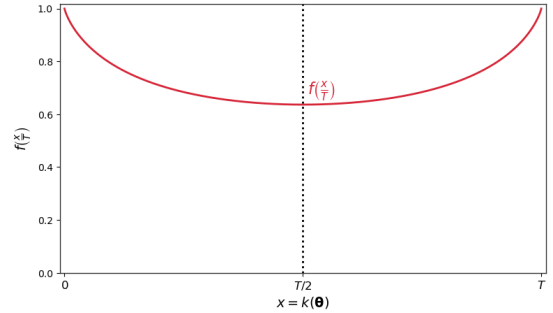


Figure 7: $f\left(\frac{x}{T}\right)$ on interval $[0, 1]$ for $x = k(\boldsymbol{\theta}) \in \{0, 1, \dots, T\}$

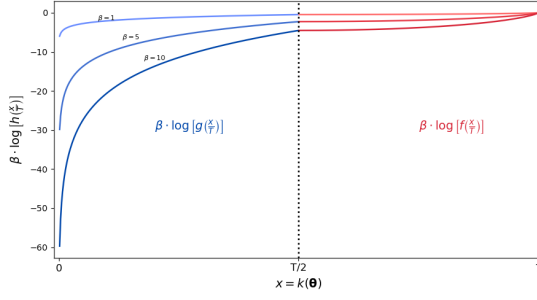


Figure 8: $\beta \cdot \log[h(x)]$ for $x = k(\theta) \in \{0, 1, \dots, T\}$ for various β .

A key distinction between the beta-based likelihood in Equation 13 and the binomial-based likelihood in Equation 11 (see Section 6.1.5.1) lies in the differing rates of change across the support $k(\theta) \in \{0, 1, \dots, T\}$. Specifically, the binomial-based log-likelihood exhibits a relatively flat profile across much of the support, followed by a pronounced increase in steepness at higher values of $k(\theta)$. This behavior is visually evident in Figure 10, where the gradient of the binomial-based log-likelihood clearly exceeds that of the beta-based log-likelihood for large $k(\theta)$. In the context of the MCMC algorithm, such a steep ascent corresponds to a more sharply peaked posterior $p(\theta \mid \sigma_\theta^2, \mathcal{D})$, hence acceptance into higher likelihood regions, $k(\theta) \in \{\frac{T}{2} + 1, \frac{T}{2} + 2, \dots, T\}$, are likely to be accelerated when using the binomial-based likelihood. It is also worthwhile to note that the derivative of both binomial and beta-based log-likelihoods for domain $k(\theta) \in \{0, 1, \dots, \frac{T}{2}\}$ are equal, that is, $\beta \cdot \log[\frac{d}{dx}g^{\text{Binomial}}(x)] = \beta \cdot \log[\frac{d}{dx}g^{\text{Beta}}(\frac{x}{T})] = \beta \cdot \frac{1}{x}$ as shown in Figure 9. This similarity implies that, from an MCMC perspective, transitions out of lower-likelihood regions, $k(\theta) \in \{0, 1, \dots, \frac{T}{2}\}$, into the higher likelihood regions, $k(\theta) \in \{\frac{T}{2} + 1, \frac{T}{2} + 2, \dots, T\}$, are likely to proceed with similar efficiency under both likelihoods. It is important to note, however, that the derivatives depicted reflect infinitesimal changes in the log-likelihood. In contrast, the MH algorithm typically evaluates differences over finite, and often substantial, differences in log-likelihood values. As such, our derivative plots provide interpretive value primarily in scenarios where the proposed θ yield objective function values of comparable magnitude.

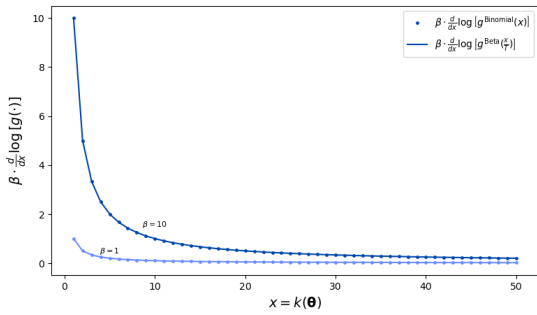


Figure 9: $\beta \cdot \frac{d}{dx} \log[g(\cdot)]$ on interval $[0, \beta]$ for $x = k(\theta) \in \{0, 1, \dots, T\}$ for various β .

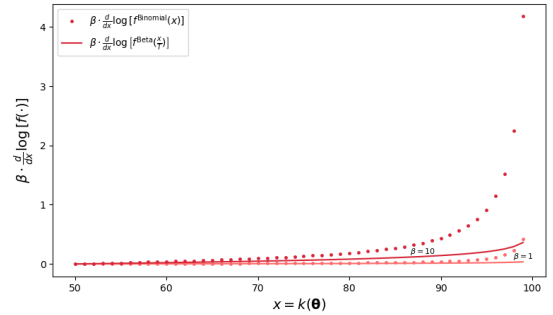


Figure 10: $\beta \cdot \frac{d}{dx} \log[f(\cdot)]$ for $x = k(\theta) \in \{0, 1, \dots, T\}$

6.1.5.3 Exponential-based likelihood Now to reiterate, in the context of this work, our primary objective is not to perform full Bayesian inference via MCMC, but rather to identify the mode of the conditional $p(\theta \mid \sigma_\theta^2, \mathcal{D})$. In such cases, the MCMC algorithm is not employed for its traditional role in posterior sampling, but rather as a stochastic optimization tool that facilitates a guided random search over the parameter space. From this perspective, strict adherence to the exact posterior structure is

unnecessary. It suffices that the proposal mechanism is guided by a function that monotonically increases with the objective of interest, thereby biasing the random walk toward high-likelihood (or high-valued objective) regions. In treating MCMC as a mode-seeking algorithm, the requirement for an explicit, well-defined likelihood linking θ to the data becomes less critical.

Being such, we utilize an exponential function as said monotonic increasing function as a substitute for a well-defined likelihood where we define $h(x) = \exp(x)$ for $x > 0$, analogous to the procedure used in SA. Hence for $\theta \in \mathbb{R}^S$, $\frac{1}{T}k(\theta) \in [0, 1]$ and sharpness $\beta \in \mathbb{R}^+$, we have our new likelihood as:

$$\begin{aligned} p(\mathcal{D} \mid \theta) &= \left[h\left(\frac{1}{T}k(\theta)\right) \right]^\beta \\ &= \exp\left(\beta \cdot \frac{1}{T}k(\theta)\right). \end{aligned} \quad (14)$$

We observe in Figure 11 a comparison between the exponential likelihood defined in Equation 14 and the alternative likelihood formulations given in Equations 11 and 13. Given that the MH algorithm bases proposal acceptance on the ratio of likelihoods, more precisely, the difference in log-likelihoods, the absolute magnitude of the likelihood function is of limited relevance. Instead, the relative rate of change, as illustrated in Figure 12, offers more informative insight into how the Markov chain is guided through the parameter space.

As illustrated in Figure 12, the derivative of the exponential-based log-likelihood, given by $\beta \cdot \frac{1}{T}$, exceeds the gradients of the alternative log-likelihoods over much of the domain $k(\theta) \in \{\frac{T}{2}, \frac{T}{2} + 1, \dots, T\}$. However, this dominance progressively diminishes as $k(\theta)$ increases. In contrast, the derivative of the binomial-based log-likelihood surpasses those of the alternatives at higher values of $k(\theta)$. In this regard, the binomial-based formulation may be considered advantageous, as its steeper gradient in the upper region of the domain can facilitate more dynamic transitions toward higher-valued objective areas, potentially reducing the risk of the Markov chain becoming trapped in local optima.

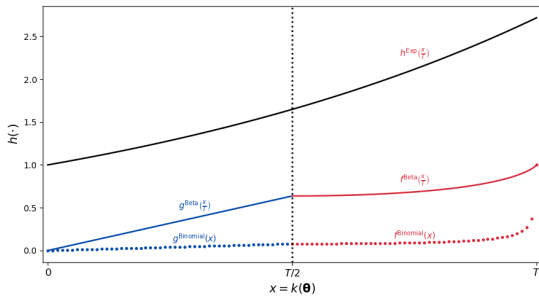


Figure 11: $h^{\text{Binomial}}(x)$, $h^{\text{Beta}}(\frac{x}{T})$ and $h^{\text{Exp}}(\frac{x}{T})$ for $x = k(\theta) \in \{0, 1, \dots, T\}$.

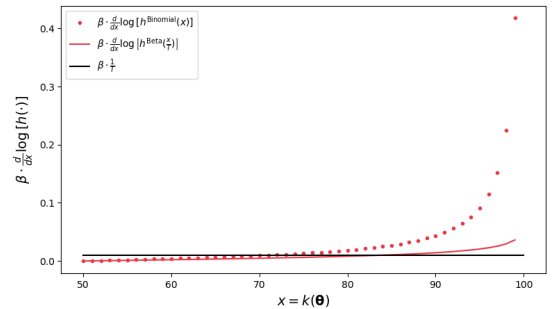


Figure 12: $\beta \cdot \frac{d}{dx} \log[h(\cdot)]$ for $x = k(\theta) \in \{\frac{T}{2}, \frac{T}{2} + 1, \dots, T\}$ for $\beta = 1$.

6.1.6 Results: Effects of regularization

To preliminarily evaluate the effect of regularization on the performance of θ , we assess generalization under a given regularization strength ν . Specifically, we apply the estimator $\hat{\theta}_\nu^{\text{GA}}$, obtained via a GA with L_2 regularization, to a newly initialized environment, defined by a distinct set of J obstacle coordinates at the k^{th} iteration, $\mathbf{o}_j^{(k)}$ for $j = 1, 2, \dots, J$. These coordinates are governed by a new set of angular fre-

quencies ω_j , phase shifts ϕ_j , and orbital radii r_j . Additionally, a new initialization is also characterized by T new initial drone coordinates, $\mathbf{x}_t^{(0)}$ for $t = 1, 2, \dots, T$. We dictate our training initialization by using seed value ω_0^{Train} , with 1000 test initializations governed by seed values $\{\omega_j^{\text{Test}}\}_{j=1}^{1000}$. More formally, our game-updating equation becomes $\mathbf{x}_t^{(k+1)} = \mathbf{x}_t^{(k)} + \mathbf{ct}(\mathbf{x}_t^{(k)}, \hat{\boldsymbol{\theta}}_\nu^{\text{GA}}) \delta_t$ for which $\forall j, k : \mathbf{o}_j^{(k)} \cap \mathbf{o}_j^{(k)\text{Train}} = \emptyset$ and $\forall t : \mathbf{x}_t^{(0)} \cap \mathbf{x}_t^{(0)\text{Train}} = \emptyset$. Figure 13 illustrates the distributions of failures, where the number of failures are defined as $T - k \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA}} \right)$, for 1000 of these initializations against varying regularization strengths ν for both **model** and **model**. Table 1 displays the median and mean of the success distributions, where the number of successes are defined as $k \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA}} \right)$, as well as displaying $\hat{R}_{\text{detection}}$ which is subsumed in $\hat{\boldsymbol{\theta}}_\nu^{\text{GA}}$.

With respect to **model**, Table 1 indicates that improved out-of-sample performance is achieved when the optimization procedure estimates $\hat{R}_{\text{detection}} < R_{\text{crash}} = 0.05$ (see Appendix D for additional specifications). This suggests that only the second case of Equation 8 is necessary to govern the behavior of non-terminal drones in order to enhance out-of-sample performance. In contrast, for **model**, Table 1 suggests that improved out-of-sample performance is attained when the estimated detection radius satisfies $\hat{R}_{\text{detection}} \approx 0.07$. These notions are further supported by the results in Table 2.

Furthermore, no consistent pattern in drone success rates on the out-of-sample set is observed as ν varies, as illustrated in Figure 13. Instead, performance appears to depend primarily on whether $\hat{R}_{\text{detection}}$ is estimated to be greater or less than the crash radius $R_{\text{crash}} = 0.05$ under **model**, or whether $\hat{R}_{\text{detection}} \approx 0.07$ under **model**. The results suggest that there are many plausible solutions that yield high objective values on the in-sample set, although only a subset of these generalize well to the out-of-sample set.

$\nu \times 10^6$	model $\left(\overset{\text{(I)}}{a(t)^0}, \hat{\boldsymbol{\theta}}_{2,\nu}^{\text{GA},\text{(I)}} \right)$				model $\left(\overset{\text{(II)}}{a(t)^0}, \hat{\boldsymbol{\theta}}_{2,\nu}^{\text{GA},\text{(II)}} \right)$			
	In-Sample		Out-of-Sample		In-Sample		Out-of-Sample	
	$k \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA},\text{(I)}} \right)$	$\tilde{k} \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA},\text{(I)}} \right)$	$\bar{k} \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA},\text{(I)}} \right)$	$\hat{R}_{\text{detection}}$	$k \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA},\text{(II)}} \right)$	$\tilde{k} \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA},\text{(II)}} \right)$	$\bar{k} \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA},\text{(II)}} \right)$	$\hat{R}_{\text{detection}}$
0	100	94.00	87.58	0.0350	100	87.00	78.67	0.0726
1	99	14.00	24.86	0.1993	98	54.00	50.61	0.1056
2	99	91.00	75.69	0.0094	100	15.50	29.77	0.1206
3	99	33.00	37.06	0.1238	98	60.00	54.6	0.0983
4	99	75.00	63.61	0.0048	100	84.00	73.90	0.1075
5	100	68.00	59.55	0.0063	99	66.00	58.56	0.0962
6	98	30.00	36.64	0.1481	98	70.50	61.09	0.1016
7	98	33.00	37.29	0.1309	100	86.00	78.47	0.0721
8	100	87.00	79.97	0.0136	100	87.00	79.34	0.0724
9	98	32.00	36.51	0.1249	100	80.00	69.20	0.1161

Table 1: Number of successes $k \left(\hat{\boldsymbol{\theta}}_\nu^{\text{GA}} \right)$ for the in-sample initialization, median ($\tilde{k}$) and mean ($\bar{k}$) for the distributions of successes on the 1000 test initializations and estimated $\hat{R}_{\text{detection}}$, against varying ν for $T = 100$ and $R_{\text{crash}} = 0.05$ using both **model** and **model**.

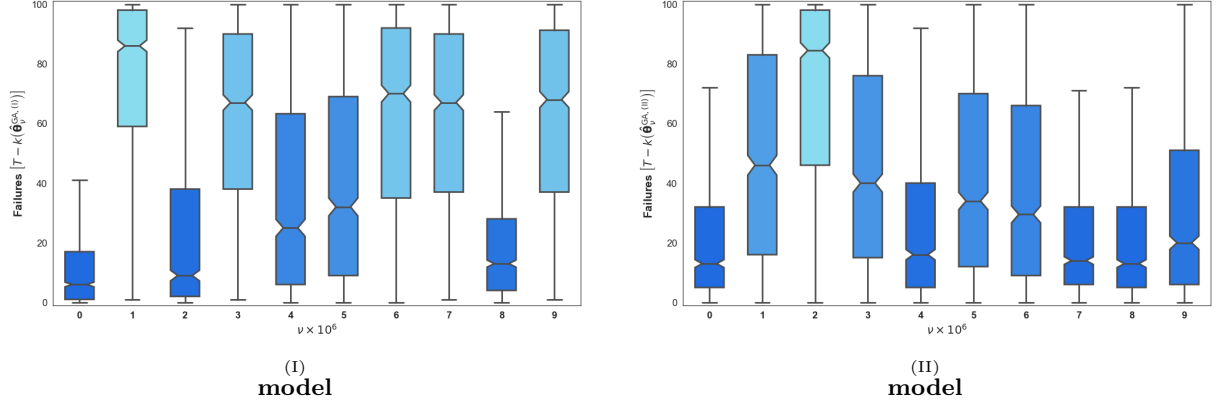


Figure 13: Boxplots illustrating distributions of failures, $T - k(\hat{\theta}_\nu^{\text{GA}})$, against varying ν obtained through 1000 test initializations for $T = 100$ for **model** (left) and **model** (right).

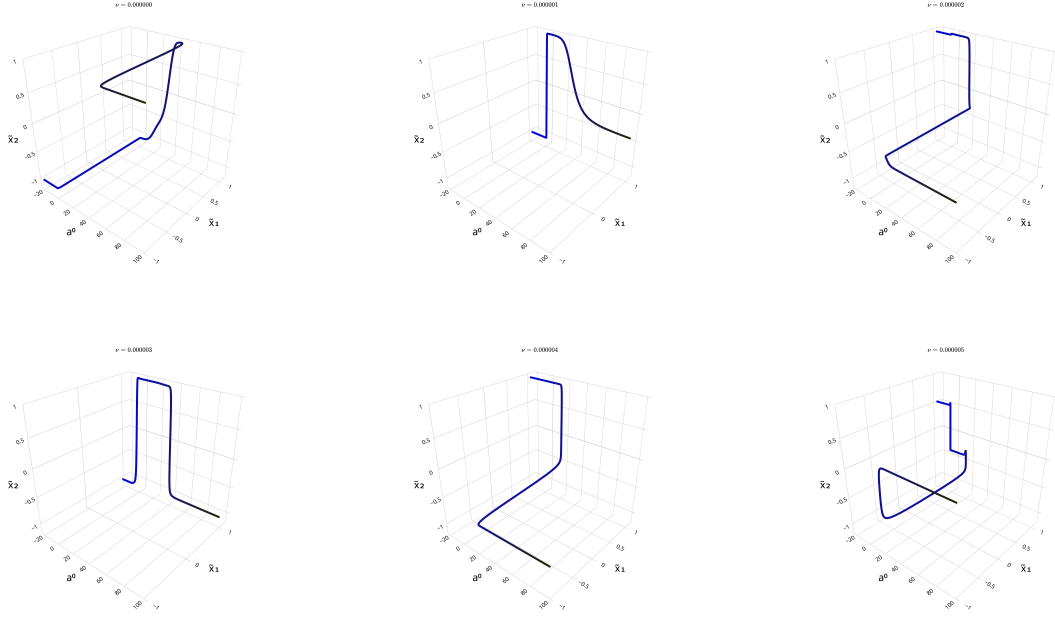
As shown in Table 2, there is a clear trend of decreasing in-sample performance as the regularization strength increases across both models. This behaviour is consistent with underfitting resulting from excessive regularization. That is, the models become overly constrained. Furthermore, the results suggest that the use of the GA may not be essential for achieving improved performance on the in-sample set, as the RS⁸ (used only for **model**) performs comparably to the GA at low values of ν . This observation implies that the fine-tuning capabilities of the GA offer limited benefit when the model is only weakly regularized. Under such conditions, the GA's exploitation mechanisms appear to be inconsequential, with its role in refining existing parent solutions rendered largely unnecessary. Interestingly, at moderate regularization levels, the RS appears to estimate $\hat{R}_{\text{detection}} \approx 0.07$ more effectively than the GA. However, this outcome reinforces the notion that multiple plausible solutions exist which yield high objective values on the in-sample set, and that the RS may have coincidentally identified ones that generalize well. Nevertheless, Table 1 demonstrates that the GA is indeed capable of consistently discovering this advantageous value of $\hat{R}_{\text{detection}}$.

ν	model ^(I) $\left(a(t)^0, \hat{\theta}_{2,\nu}^{\text{GA},(I)} \right)$			model ^(II) $\left(a(t)^0, \hat{\theta}_{2,\nu}^{\text{GA},(II)} \right)$			model ^(II) $\left(a(t)^0, \hat{\theta}_{2,\nu}^{\text{RS},(II)} \right)$		
	In-Sample	OOS	$\hat{R}$	In-Sample	OOS	$\hat{R}$	In-Sample	OOS	$\hat{R}$
	$k(\hat{\theta}_\nu^{\text{GA},(I)})$	$\bar{k}(\hat{\theta}_\nu^{\text{GA},(I)})$		$k(\hat{\theta}_\nu^{\text{GA},(II)})$	$\bar{k}(\hat{\theta}_\nu^{\text{GA},(II)})$		$k(\hat{\theta}_\nu^{\text{RS},(II)})$	$\bar{k}(\hat{\theta}_\nu^{\text{RS},(II)})$	
0.000001	99	24.86	0.1993	98	50.61	0.1056	100	77.51	0.0666
0.00001	98	37.54	0.1362	100	68.29	0.0847	98	79.55	0.0668
0.0001	100	75.04	0.0070	100	42.63	0.2843	96	78.36	0.0779
0.001	96	31.27	0.1463	98	25.30	0.0942	70	18.68	0.0375
0.01	74	18.62	0.5863	74	18.61	0.5406	21	19.45	0.9914
0.1	62	17.63	0.4640	73	18.14	0.4550	8	18.62	0.1608
1	8	12.15	0.4403	49	13.57	0.5855	70	18.68	0.1757

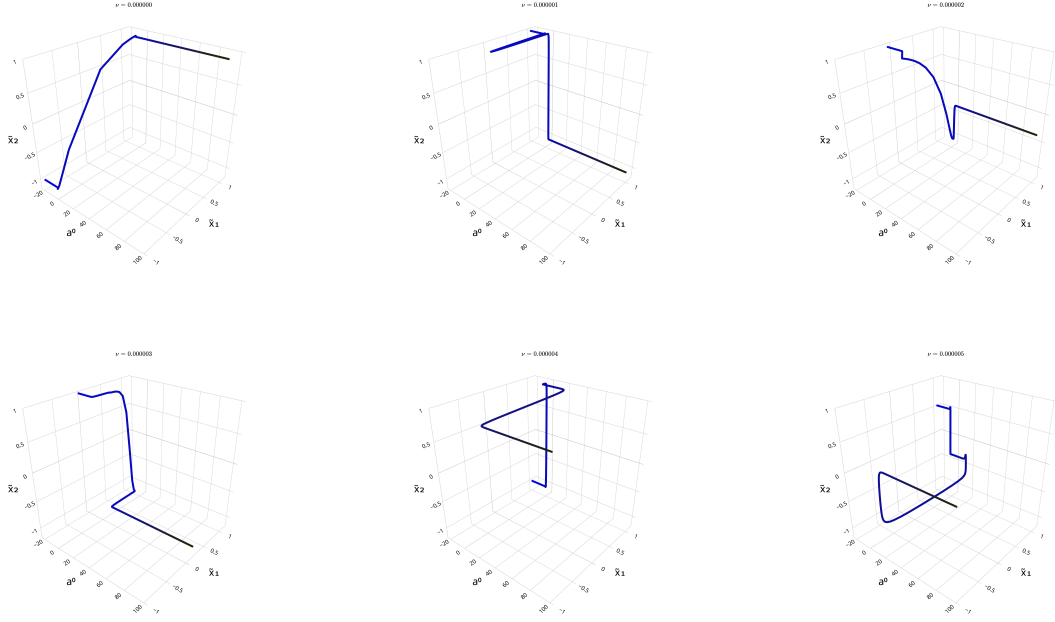
Table 2: Number of successes $k(\hat{\theta}_\nu^{\text{GA}})$ for the in-sample initialization, mean ($\bar{k}$) for the distributions of successes on the 1000 test initializations (OOS) and estimated $\hat{R}_{\text{detection}}$, against varying ν for $T = 100$ and $R_{\text{crash}} = 0.05$ using both **model** and **model**.

⁸Just as the GA utilises $M = 1000$ generations of population $N = 100$, the RS likewise performs $M \cdot N$ iterations.

6.1.6.1 Response curves To further evaluate the role of regularization and to examine how the control vector integrates drone state information and environmental elements to output drone evasive manoeuvres, we analyze the response curves shown in Figure 14. For a given input $a^0 \in \mathbb{R}$, the model yields the output $\tilde{\mathbf{x}} = (\tilde{x}_1, \tilde{x}_2)$ using $\text{model}^{(\text{I})} \left(a^0, \hat{\boldsymbol{\theta}}_{2,\nu}^{\text{GA},(\text{I})} \right)$ and $\text{model}^{(\text{II})} \left(a^0, \hat{\boldsymbol{\theta}}_{2,\nu}^{\text{GA},(\text{II})} \right)$. It is worth noting that values of $a^0 < 0$ are feasible, as such cases arise only when a given drone has entered a terminal state. Furthermore, no distinguishable pattern appears to emerge as ν varies; each response curve exhibits a unique shape, with no visually discernible structure that consistently correlates with improved performance as reflected in Table 1.



(I)
model



(II)
model

Figure 14: Response curves of $\tilde{\mathbf{x}} = (\tilde{x}_1, \tilde{x}_2)$ given a^0 using $\text{model}^{(I)} \left(a^0, \hat{\theta}_{2,\nu}^{\text{GA},(I)} \right)$ (top) and $\tilde{\mathbf{x}} = (\tilde{x}_1, \tilde{x}_2)$ given a^0 using $\text{model}^{(II)} \left(a^0, \hat{\theta}_{2,\nu}^{\text{GA},(II)} \right)$ (bottom) for various ν .

6.1.7 Results: MCMC

We aim to compare the behaviour of the three likelihood formulations discussed previously: the binomial-based likelihood (Equation 10), the beta-based likelihood (Equation 13), and the exponential-based likelihood (Equation 14), in terms of their impact on the MCMC results obtained. To ensure a fair comparison, we evaluate each likelihood using the same fixed value of the sharpness parameter, specifically $\beta = 20$, selected arbitrarily. It is important to note that each pseudo-likelihood induces values of differing magnitudes when evaluated at the same $k(\boldsymbol{\theta})$ (this notion is illustrated in Figure 11). Consequently, we refrain from reporting the corresponding values of the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$ (since $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D}) \propto p(\mathcal{D} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \sigma_{\theta}^2)$ and the scale of the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ varies considerably across formulations). Furthermore, we employ **model**^(II) in this section.

Table 3 presents the results, from which it is evident that the binomial-based likelihood yields the best in-sample solution. However, none of the likelihood formulations appear to generalize well to the out-of-sample set of initializations. This is most likely attributable to the estimation of the parameter $R_{\text{detection}}$. As discussed in Section 6.1.6, the out-of-sample performance of a given solution appears to be highly sensitive to the value estimated for this parameter.

Additionally, the multivariate effective sample sizes (ESS) reported in Table 3 provide evidence of satisfactory mixing. High ESS values indicate that the sampler is generating effectively independent samples, that is, with low autocorrelation, which are representative of the target posterior distribution. All reported ESS values exceed the commonly accepted threshold of 100 (as recommended by Vehtari, Gelman, Simpson, Carpenter, and Bürkner (2021), Section 4), which supports the claim of efficient exploration. This performance is not coincidental: as discussed in Section 3.2, the sampler was deliberately tuned using an adaptive MH scheme to encourage good mixing. Furthermore, Table 3 suggests that the exponential-based likelihood exhibits the best mixing properties, as indicated by the ESS. That said, there is no clear evidence of poor mixing in the other likelihood formulations.

Figure 15 illustrates the poor convergence behaviour of the beta- and exponential-based likelihoods, as evidenced by the plots of $\|\boldsymbol{\theta}^{(j)}\|^2$ across MCMC iterations. The top panel displays all 100,000 MCMC iterations. Their trajectories exhibit an initial phase of high variability and gradual decline, followed by a transition into a region of relative stability. This behavior suggests that the sampler undergoes a prolonged adaptation phase before reaching a region of the parameter space where $\|\boldsymbol{\theta}^{(j)}\|^2$ fluctuates around a stable value. The prolonged descent and subsequent stabilization imply delayed convergence for the beta- and exponential-based likelihood types, necessitating a longer burn-in period to ensure samples are drawn from the stationary distribution. We note, however, that the binomial-based likelihood exhibits relatively good convergence. For the beta- and exponential-based likelihoods, a burn-in of 60,000 iterations was applied, motivated by the point at which $\|\boldsymbol{\theta}^{(j)}\|^2$ appears to stabilize, whereas a shorter burn-in of 20,000 iterations was used for the binomial-based likelihood. The corresponding marginal distributions, $p(\sigma_{\theta}^2 \mid \mathcal{D})$, are shown in the bottom panel of Figure 15.

Despite the different convergence dynamics, the marginal distributions, $p(\sigma_{\theta}^2 \mid \mathcal{D})$, across all likelihood types appear to be consistent with an inverse-gamma form. Given that the conditional posterior $\sigma_{\theta}^2 \mid \boldsymbol{\theta}, \mathcal{D} \sim \text{Inv-Gamma}\left(a + \frac{S}{2}, b + \frac{\|\boldsymbol{\theta}\|^2}{2}\right)$ with $a, b \approx 0$ (known from Section 3.1.1), we expect that if $\|\boldsymbol{\theta}^{(j)}\|^2$

fluctuates around a constant value c , then the marginal distribution $\sigma_\theta^2 \mid \mathcal{D} \sim \text{Inv-Gamma}(a + \frac{S}{2}, b + \frac{c}{2})$. That is, it should also retain the inverse-gamma form with approximately constant shape and scale parameters. Regarding the level of regularization inferred from the training set, where regularization is represented via the dispersion parameter $\sigma_\theta^2 \propto \frac{1}{\nu}$, we may draw meaningful conclusions from the marginals of $\sigma_\theta^2 \mid \mathcal{D}$ shown in Figure 15. Specifically, the variation in these distributions across different likelihood types suggests that each likelihood inherently induces a different degree of regularization, which is subsequently reflected in the MAP estimates of the parameters.

Furthermore, we may attribute the superior in-sample performance of the binomial-based likelihood to its sharper increase at high values of $k(\theta)$, as we previously alluded to in Figure 12. We propose that the likelihood function can be interpreted as a mechanism for modulating the structural complexity of the conditional posterior. Specifically, in the case of flatter likelihoods, the influence of the likelihood on the posterior is weak, resulting in a diffuse posterior landscape that may support multiple regions with comparable acceptance probabilities α_θ . This, in turn, can induce a multimodal posterior structure. In such a regime, the sampler is encouraged to explore broadly, often traversing disconnected or competing modes across the parameter space. However, this extensive exploration may lead the Markov chain to eventually converge to a mode that is not aligned with a high-value region of the objective function, given the presence of multiple modes. In contrast, a more sharply peaked likelihood suppresses minor modes and concentrates posterior mass around a dominant region, thereby sharpening the posterior and promoting unimodality. We posit that this behaviour underlies the relatively better performance observed with the binomial-based likelihood on the in-sample set, in contrast to the beta- and exponential-based alternatives, which likely converge to sampling around one of these minor modes.

Now, to potentially improve the in-sample performance using the beta- and exponential-based likelihoods, one could increase their likelihood sharpness parameter β , thereby inducing a more concentrated conditional posterior. We do not pursue this adjustment in the present analysis however.

Likelihood Type	In-Sample	Out-of-Sample		$\hat{R}_{detection}$	ESS
	$k(\hat{\theta}_\nu^{\text{GA},(\text{II})})$	$\tilde{k}(\hat{\theta}_\nu^{\text{GA},(\text{II})})$	$\bar{k}(\hat{\theta}_\nu^{\text{GA},(\text{II})})$		
Binomial-based	100	13.00	24.45	0.3912	179.8550
Beta-based	50	8.00	12.72	0.5175	170.0205
Exponential-based	52	12.00	17.25	0.5601	317.2764

Table 3: Number of successes $k(\hat{\theta}_\nu^{\text{GA}})$ for the in-sample initialization, summary statistics for the distributions of successes on the 1000 test initializations and estimated $\hat{R}_{detection}$, for the three likelihood types.

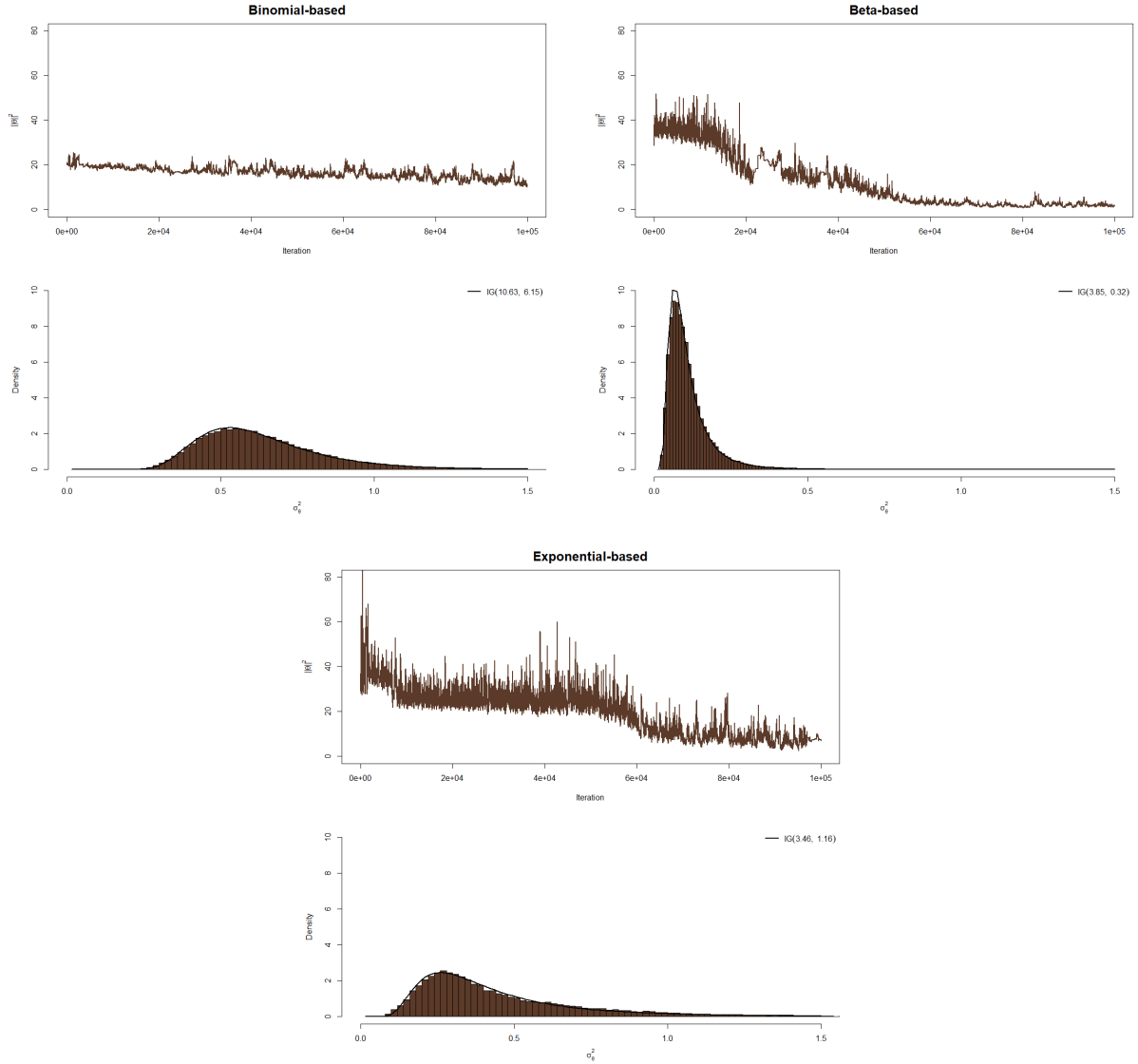


Figure 15: $\|\theta^{(j)}\|^2$ for $j = 1, \dots, 100,000$ (no burn-in) in the top panels, with distribution of marginal $\sigma_\theta^2 \mid \mathcal{D}$ (using 20,000 burn-in for binomial-based and 60,000 burn-in for beta- and exponential-based) for the three likelihood types in the bottom panels (all plots use the same scale).

6.2 The Tic-Tac-Toe problem

In what follows, the study employs a tic-tac-toe reinforcement learning problem to examine how increasing likelihood sharpness (including initial variance σ_{Init}^2) affects both in-sample and out-of-sample performance, as well as the level of regularization inferred. Specifically, a neural network serves as the control policy governing the player’s decisions at each turn, competing against an opponent whose moves are randomly assigned. This environment is deliberately chosen to evaluate MCMC performance in a discrete, stochastic setting with a discrete-valued arbitrary objective.

More specifically, the environment corresponds to the classic tic-tac-toe game: a two-player, deterministic, turn-based game in which the player and opponent alternately place their respective tokens X and O on a 3×3 grid. The objective of the player is to be the first to align three of their tokens consecutively

in a row, column, or diagonal, and likewise for the opponent.

6.2.1 Encoding

Mathematically, the 3×3 game board is represented by a matrix $\mathbf{M}_{3 \times 3}$, where an X token is encoded as -1 , an O token as $+1$, and empty cells as 0 . For convenience, the matrix $\mathbf{M}$ is vectorized by row into a single 9-dimensional vector $\mathbf{m}$, thereby providing a compact representation of the board state. In the configuration considered, the player is assumed to make the first move, assigned as the O player.

Furthermore, we define a game of tic-tac-toe to be complete at time k , where the player places the O token on the grid at discrete times $\tau_k^{(p)} = 1, \dots, T_k^{(p)} \leq 5$ (since the player moves first), and the opponent places the X token on the grid at discrete times $\tau_k^{(o)} = 1, \dots, T_k^{(o)} \leq 4$, while the overall time index for the k^{th} game is given by $\tau_k = 1, \dots, T_k \leq 9$. Hence $\mathbf{m}^{\tau_k-1}$ would be the current board state before the player or opponent places their token at time τ_k . Furthermore, we define the set of opponent's decisions/actions at the conclusion of the k^{th} game in sequence $\mathcal{O}_k = \left[a_{\tau_k^{(o)}=1}, \dots, a_{\tau_k^{(o)}=T_k^{(o)}} \right]$ for any action $a \in \{1, 2, \dots, 9\}$ corresponding to available board positions. Furthermore, we define the set of opponent's actions for K games as matrix $\mathcal{O}_K = \left[\mathcal{O}_1, \dots, \mathcal{O}_K \right]$.

6.2.2 The game's outcome

At each turn of the game, the current state must be evaluated to determine whether the game has reached a terminal condition: namely, a win, loss, or draw, or whether play should continue. This involves computing the sums of each row, column, and diagonal of $\mathbf{M}$ to check for a winning configuration. Specifically, a sum of -3 or 3 indicates a win for the X or O player, respectively.

We may evaluate the game state by transposing and post-multiplying $\mathbf{m}$ by a state-matrix:

$$\mathbf{S} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}_{9 \times 8}$$

hence, if any one of the 8 entries in the game-state evaluation $\mathbf{m}'\mathbf{S}$ is equal to -3 , X has won. Likewise, if any of the entries is equal to $+3$, O has won.

To assess whether a draw has occurred, it is necessary to verify that each of the three rows, three columns, and both diagonals of $\mathbf{M}$ contains at least one X and one O . This ensures that no player can achieve a winning alignment in any direction. To do this, let $\mathbf{m}_+ = [\mathbb{I}(m_i = +1)]_{i=1}^9$ where $\mathbb{I}(\cdot)$ is the indicator function, applied element-wise to the entries of $\mathbf{m}$. Hence, $\mathbf{m}_+$ is a 9-dimensional binary vector with ones at positions where the corresponding entries of $\mathbf{m}$ are equal to $+1$, and zeros elsewhere. Similarly, we define the 9-dimensional binary vector $\mathbf{m}_-$ analogously to indicate the positions of entries

equal to -1 in $\mathbf{m}$. A draw has then occurred if $(\mathbf{v}_+)'_{1 \times 8}(\mathbf{v}_-)_{8 \times 1} = 8$ where 8-dimensional vector $\mathbf{v}_+ = \left[\mathbb{I} \left((\mathbf{m}'_+ \mathbf{S})_i > 0 \right) \right]_{i=1}^8$ and $\mathbf{v}_- = \left[\mathbb{I} \left((\mathbf{m}'_- \mathbf{S})_i > 0 \right) \right]_{i=1}^8$.

We denote ρ_{τ_k} to denote the value of the winning token or zero (for a draw) if the game is terminal after the player or opponent has placed their token at time $\tau_k = 1, 2, \dots, T_k$ for the k^{th} tic-tac-toe game. Hence:

$$\rho_{\tau_k} = \begin{cases} +1, & \text{if } \exists i \in \{1, \dots, 8\} \text{ such that } ((\mathbf{m}^{\tau_k})' \mathbf{S})_i = +3, \\ -1, & \text{if } \exists i \in \{1, \dots, 8\} \text{ such that } ((\mathbf{m}^{\tau_k})' \mathbf{S})_i = -3, \\ 0, & \text{if } (\mathbf{v}_+^{\tau_k})' (\mathbf{v}_-^{\tau_k}) = 8, \\ \text{NULL}, & \text{if game is not terminal.} \end{cases}$$

6.2.3 Control

We control player decisions/actions at time $\tau_k^{(p)} = 1, \dots, T_k^{(p)} \leq 5$ for the k^{th} tic-tac-toe game through the means of the control vector $\mathbf{ct}(\mathbf{m}^{\tau_k-1}, \boldsymbol{\theta}^{\text{Decision}}) \in \mathcal{A}_{\tau_k} \subseteq \{1, 2, \dots, 9\}$, where $\mathcal{A}_{\tau_k}$ represents the subset of available board positions on $\mathbf{m}^{\tau_k-1}$ before the player opts to play at time τ_k for some parameter configuration $\boldsymbol{\theta}^{\text{Decision}} \in \mathbb{R}^R$. Naturally, once a grid position is occupied by a token, it becomes unavailable for subsequent moves by either the player or the opponent. Hence $\mathcal{A}_{\tau_k} = \{i \in \{1, \dots, 9\} : m_i^{\tau_k-1} = 0\}$. The player action selection is probabilistic and derived from a softmax distribution over logits. Hence, for ℓ_a being the logit score for any valid action $a \in \mathcal{A}_{\tau_k}$, the probability of selecting that action is:

$$\sigma_L(a \mid \mathbf{m}^{\tau_k-1}, \boldsymbol{\theta}^{\text{Decision}}) = \frac{\exp(\ell_a)}{\sum_{a' \in \mathcal{A}_{\tau_k}} \exp(\ell_{a'})}.$$

The selected action a^* corresponds to the action with the highest probability, that is, $a^* = \arg\max_{a \in \mathcal{A}_{\tau_k}} \sigma_L(a \mid \mathbf{m}^{\tau_k-1}, \boldsymbol{\theta}^{\text{Decision}})$. Invalid actions (i.e., $a \notin \mathcal{A}_{\tau_k}$) are assigned $\ell_a = -\infty$, ensuring a zero probability is attributed to that specific invalid action. Now the interface between a model and the player action is undergone through this control vector for which $\mathbf{ct} : (\mathbf{m}^{\tau_k-1}, \boldsymbol{\theta}^{\text{Decision}}) \rightarrow \mathbf{model}(\boldsymbol{\Omega}(\mathbf{m}^{\tau_k-1}), \boldsymbol{\theta}^{\text{Decision}}) \xrightarrow{\sigma_L(\cdot)} a^* \in \mathcal{A}_{\tau_k}$ where $\boldsymbol{\theta}^{\text{Decision}}$ is fixed throughout all $k = 1, 2, \dots, K$ tic-tac-toe games for all player turns at times $\tau_k^{(p)} = 1, \dots, T_k^{(p)} \leq 5$, and all player decisions are based on this fixed parameterization $\boldsymbol{\theta}^{\text{Decision}}$. In the framework of using a neural network as our model, we define $\boldsymbol{\Omega} : \mathbf{m}^{\tau_k-1} \rightarrow \mathbf{a}^0 \in \mathbb{R}^{d_0}$ which signifies the vector of input nodes for times $\tau_k^{(p)} = 1, \dots, T_k^{(p)} \leq 5$. Furthermore, $\boldsymbol{\theta}^{\text{Decision}}$ are the weights and biases of the neural network, $\mathbf{w}^{\text{Decision}} \in \mathbb{R}^R$.

6.2.3.1 Feature engineering We construct the feature vector using two inputs: the current board state before the player places a token at time τ_k , $\mathbf{m}^{\tau_k-1}$, as well as the game-state evaluation, $(\mathbf{m}^{\tau_k-1})' \mathbf{S}$. The former encodes the spatial configuration of tokens on the board capturing positional information essential to the learning process. The latter provides a structured summary of token alignments across rows, columns, and diagonals, serving as a higher-level representation that facilitates the identification of a win or loss of the player.

While the raw board configuration, $\mathbf{m}^{\tau_k-1}$, contains all information required to determine the optimal action, relying solely on this representation would require the neural network to implicitly learn the deterministic game logic governing winning, losing, and blocking configurations. This introduces

unnecessary complexity into the learning problem. Hence, to mitigate this, we additionally include the game-state evaluation, $(\mathbf{m}^{\tau_k-1})'\mathbf{S}$, which provides an explicit summary of token alignments across rows, columns, and diagonals. This representation highlights strategically salient patterns, such as imminent wins or threats, thereby reducing the burden on the neural network to rediscover deterministic game logic. Therefore, our 1^{st} set of input nodes are defined as $\mathbf{a}_1^0 = \mathbf{m}^{\tau_k-1}$ which represents the board state at time $\tau_k - 1$ and the 2^{nd} set of input nodes is given by $\mathbf{a}_2^0 = (\mathbf{m}^{\tau_k-1})'\mathbf{S}$ which denotes the game-state evaluation at time $\tau_k - 1$.

6.2.4 The arbitrary objective

Consider an arbitrary objective where, for a given parameter configuration $\boldsymbol{\theta} = \boldsymbol{\theta}^{\text{Decision}} \in \mathbb{R}^R$ and after playing K number of tic-tac-toe games, we count the number of times the player's token O (encoded as $+1$) won the game, denoted as $\sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1) \in \{0, 1, \dots, K\}$. Hence:

$$\operatorname{argmax}_{\boldsymbol{\theta}} \text{Obj}(\boldsymbol{\theta}) = \operatorname{argmax}_{\boldsymbol{\theta}} \frac{1}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1).$$

By including L2 regularization, our L2 penalized objective becomes:

$$\operatorname{argmax}_{\boldsymbol{\theta}} \left(\frac{1}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1) - \nu \|\boldsymbol{\theta}\|^2 \right). \quad (15)$$

Now congruent to Section 4, $\operatorname{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D})$ must be equivalent to maximising the objective function in Equation 15. This is achieved by ensuring the likelihood $p(\mathcal{D} \mid \boldsymbol{\theta})$ is monotonic increasing with respect to $\sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)$, that is, $p(\mathcal{D} \mid \boldsymbol{\theta}) \propto \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)$.

6.2.4.1 Exponential-based likelihood Section 5 elucidated that when MCMC is employed primarily as a mode-seeking algorithm, that is, the mode of the conditional $p(\boldsymbol{\theta} \mid \sigma_{\boldsymbol{\theta}}^2, \mathcal{D})$, rather than for full Bayesian inference, the necessity of an explicit and well-defined likelihood function linking the parameters $\boldsymbol{\theta}$ to the observed data becomes less critical. In such settings, it suffices to employ any monotonically increasing transformation of the objective function to guide the proposal mechanism of the MH algorithm, thereby biasing the random walk toward regions of high-likelihood (high-valued objective) regions to sample around a dominant mode of the conditional.

Accordingly, we adopt an exponential transformation as the chosen monotonic function, serving as a surrogate for the traditional likelihood, to facilitate efficient exploration of high-valued objective regions in the parameter space. Hence for $\boldsymbol{\theta} \in \mathbb{R}^R$, $\sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1) \in \{0, 1, \dots, K\}$ hence $\frac{1}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1) \in [0, 1]$ and sharpness $\beta \in \mathbb{R}^+$, we have our new likelihood as:

$$p(\mathcal{D} \mid \boldsymbol{\theta}) = \exp \left(\beta \cdot \frac{1}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1) \right), \quad (16)$$

with the log of conditional posterior being, noting the prior $\boldsymbol{\theta} \mid \sigma_\theta^2 \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \mathbf{I}_S)$:

$$\begin{aligned} \log(p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})) &\propto \log(p(\mathcal{D} \mid \boldsymbol{\theta})) + \log(p(\boldsymbol{\theta} \mid \sigma_\theta^2)) \\ &\propto \beta \cdot \frac{1}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1) - \frac{1}{2\sigma_\theta^2} \|\boldsymbol{\theta}\|^2 - \frac{S}{2} \log(2\pi\sigma_\theta^2). \end{aligned} \quad (17)$$

6.2.5 Results: Effects of regularization

To train the tic-tac-toe agent, we simulate $K = 100$ tic-tac-toe games in which the model learns to play against an opponent whose behavior is governed by a random decision policy. Specifically, the opponent selects among the available (i.e., unoccupied) grid positions uniformly at random when placing its X token. As a result, the trajectory of each game, and by extension, the opponent’s decision-making process, is contingent on a random seed for which we use seed values $\{\omega_i^{\text{Train}}\}_{i=1}^{100}$ corresponding to the $K = 100$ tic-tac-toe games. Now since the player’s decisions, determined by the solution $\hat{\boldsymbol{\theta}}_\nu$, directly influences the set of random opponent decision sequences $\mathcal{O}_{100}^{\text{Train}}$ for the $K = 100$ games (as the opponent may only place their X token on unoccupied grid positions), we cannot assume that the set of random opponent decision sequences remains fixed across all solutions. For example, the solution $\hat{\boldsymbol{\theta}}_{\nu_1}$ will likely induce a different $\mathcal{O}_{100}^{\text{Train}}$ than $\hat{\boldsymbol{\theta}}_{\nu_2}$ for $\nu_1 \neq \nu_2$, even when each of the $k = 1, \dots, K$ tic-tac-toe games are initialized with the same seed. Consequently, the best we can do to ensure that the random opponent’s behavior is both deterministic and reproducible across different solutions, is to control it via a fixed seed, in an attempt to allow for controlled evaluation and consistent comparison of the agent’s performance across training runs. We may extend this notion further by observing that during the optimization process, each candidate solution dictates the set of random opponent decision sequences. As a result, the effective optimization surface to be maximized is not fixed but changes across iterations under this framework.

Furthermore, we define the test set as consisting of $K = 10,000$ simulated tic-tac-toe games, each initialized by a distinct seed value ω_i^{Test} such that $\omega_i^{\text{Test}} \neq \omega_j^{\text{Train}}$ for all i, j . However, this condition alone does not ensure that all random opponent decision sequences in the test set, $\mathcal{O}_k^{\text{Test}}$ for $k \in \{1, \dots, 10000\}$, are disjoint from those encountered during training, $\mathcal{O}_k^{\text{Train}}$ for $k \in \{1, \dots, 100\}$. Additionally, to prevent the inflation or degradation of performance due to repeated identical random opponent decision sequences in the test set, we enforce that $\mathcal{O}_k^{\text{Test}} \neq \mathcal{O}_l^{\text{Test}}$ for all $k, l \in \{1, \dots, 10000\}$ with $k \neq l$. That is, all random opponent decision sequences used during testing are mutually distinct. Hence, to guarantee that all test games are genuinely out-of-sample (and unique), we iteratively cycle through candidate seed values ω_j^{Test} until we obtain a collection of $K = 10,000$ test games whose random opponent decision sequences are distinct from those observed in the $K = 100$ training games (as well as being distinct from each other). Hence $j \geq 10,000$ for our out-of-sample test seed values ω_j^{Test} . In doing so, we ensure that the training and test environments are disjoint, thereby enabling a valid assessment of the agent’s generalization performance to previously unseen opponent behaviors. As before, however, since the random opponent decision sequences are governed by the player’s decisions, controlled by $\hat{\boldsymbol{\theta}}_\nu$, each $\hat{\boldsymbol{\theta}}_\nu$ would give rise to a different set of random opponent decision sequences $\mathcal{O}_{10,000}^{\text{Test}}$, hence rendering the $K = 10,000$ out-of-sample games to be somewhat different across solutions derived.

To evaluate the impact of the regularization strength ν , we apply the estimator $\hat{\boldsymbol{\theta}}_\nu^{\text{GA}}$ to the test set and assess both in-sample and out-of-sample performance across a range of ν values, as reported in Table 5. We denote by $\hat{\boldsymbol{\theta}}_\nu^{\text{GA},(I)}$ the solution obtained from ^(I)**model**, which solely uses $\mathbf{a}_1^0$ as its feature

vector. Likewise, $\hat{\boldsymbol{\theta}}_{\nu}^{\text{GA},(\text{II})}$ corresponds to $\text{model}^{(\text{II})}$, which incorporates the full feature vector $\mathbf{a}_{(9+8)\times 1}^0 = [(\mathbf{a}_1^0)', (\mathbf{a}_2^0)']'$. Furthermore, we also apply the estimator $\hat{\boldsymbol{\theta}}_{\nu}^{\text{RS}}$ on the test set in order to establish a baseline against which the performance of the GA can be compared (just for $\text{model}^{(\text{II})}$).

As shown in Table 4, there is a clear trend of decreasing performance, both in-sample and out-of-sample, as the regularization strength increases for both models. This behavior is consistent with underfitting due to excessive regularization, that is, the model becomes overly constrained. Moreover, the results suggest that the use of a GA is necessary to achieve improved performance on the in-sample set, as it consistently outperforms RS at low values of ν . However, this pattern does not persist across all regularization strengths: at high regularization levels, RS appears to yield better in-sample solutions than the GA at times. This observation implies that the fine-tuning capability of the GA is most beneficial when the model is not heavily constrained. That is, under such conditions, the GA’s exploitation properties appear to play a critical role in refining existing parent solutions.

Additionally, we observe, that in the absence of regularization ($\nu = 0$), $\text{model}^{(\text{II})}$ slightly outperforms $\text{model}^{(\text{I})}$ in both the in-sample and out-of-sample sets. Nonetheless, we refrain from making general claims regarding comparative performance across varying values of ν between the two models, as the two models differ in complexity with respect to their number of input nodes: $\text{model}^{(\text{I})}$ utilizing $\mathbf{a}_1^0$ as its feature vector and $\text{model}^{(\text{II})}$ utilizing $\mathbf{a}^0$ as its feature vector. Consequently, a given value of ν cannot be interpreted as exerting the same regularization effect (capturing the goal of making a model less complex) across both models, and direct comparisons of regularization magnitudes should be treated with caution. We conjecture, however, that the best out-of-sample performance may be attained at regularization strengths corresponding to $\nu \in [10^{-6}, 10^{-4}]$ for both models, where there seems to be an effective balance between in-sample performance and out-of-sample generalization.

Importantly, Table 4 also demonstrates that both models significantly outperform a baseline agent governed by purely random decision-making in cases where the model is not overly constrained, that is, when ν is not excessively large. This baseline agent (playing as the first mover) also faces a random opponent for $K = 10,000$ tic-tac-toe games all of which have distinct random opponent decision sequences ($\mathcal{O}_k \neq \mathcal{O}_l$ for all $k, l \in \{1, \dots, 10000\}$ with $k \neq l$). The random baseline achieves a normalized win percentage of only 57.16%, which is consistently exceeded by the learned agents under moderate regularization strengths, shown in both Table 4 and 5.

Now to investigate the local sensitivity of model performance to small variations in regularization strength, Table 5 reports in-sample and out-of-sample performance across finely spaced values of ν for both $\text{model}^{(\text{I})}$ and $\text{model}^{(\text{II})}$. The results reveal a non-monotonic relationship between performance and ν : small increases in regularization do not uniformly degrade performance and, in certain cases, even yield improvements, particularly in out-of-sample performance.

ν	$\text{model}^{(I)}(\mathbf{a}_1^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(I)})$		$\text{model}^{(II)}(\mathbf{a}^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(II)})$		$\text{model}^{(II)}(\mathbf{a}^0, \hat{\boldsymbol{\theta}}_\nu^{\text{RS},(II)})$	
	In-Sample	Out-of-Sample	In-Sample	Out-of-Sample	In-Sample	Out-of-Sample
0.000001	97	61.72	99	83.92	94	67.38
0.00001	99	73.78	99	74.84	96	83.16
0.0001	97	82.96	98	66.96	97	62.98
0.001	92	76.28	93	59.96	68	59.46
0.01	82	77.58	85	59.36	45	47.64
0.1	57	45.92	74	69.10	77	46.92
1	44	47.82	52	46.04	67	54.10

Table 4: The normalized number of O wins, as a percentage $\left(\frac{100}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)\right)$ for in-sample ($K = 100$) and out-of-sample ($K = 10,000$) sets across regularization strengths ν using $\text{model}^{(II)}(\mathbf{a}^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(II)})$ and $\text{model}^{(II)}(\mathbf{a}^0, \hat{\boldsymbol{\theta}}_\nu^{\text{RS},(II)})$.

ν	$\text{model}^{(I)}(\mathbf{a}_1^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(I)})$		$\text{model}^{(II)}(\mathbf{a}^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(II)})$	
	In-Sample	Out-of-Sample	In-Sample	Out-of-Sample
0.0000	98	73.62	99	77.86
0.0001	97	82.96	98	66.96
0.0002	94	64.68	97	76.82
0.0003	98	65.32	95	77.46
0.0004	96	77.20	97	73.72
0.0005	95	78.58	95	73.74
0.0006	97	63.00	95	79.50
0.0007	95	58.96	95	72.86
0.0008	95	73.20	91	62.00
0.0009	96	73.74	94	75.40

Table 5: The normalized number of O wins, as a percentage $\left(\frac{100}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)\right)$ for in-sample ($K = 100$) and out-of-sample ($K = 10,000$) sets across regularization strengths ν using $\text{model}^{(I)}(\mathbf{a}_1^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(I)})$ and $\text{model}^{(II)}(\mathbf{a}^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(II)})$.

6.2.6 Results: MCMC

We exclusively employ Equation 16 as the likelihood function in the MH algorithm presented in Section 3.1.1, in this section. Although one could reasonably argue for the use of alternative likelihoods, such as the binomial and beta-based forms presented in Section 6.1.5, we refrain from doing so here as a comprehensive comparison among these three likelihood formulations has already been conducted in Section 6.1.7. The present section is dedicated solely to illustrating how increasing the sharpness of the likelihood (through the parameter $\beta \in \mathbb{R}^+$) may influence the results. Furthermore, we employ $\text{model}^{(II)}$ in this section.

Illustrated in Table 6 are the normalized number of O wins, expressed as a percentage $\left(\frac{100}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)\right)$, computed over 100,000 total MCMC iterations, of which the first 20,000 were discarded as burn-in. We

observe a clear trend: as the sharpness parameter $\beta \in \mathbb{R}^+$ increases, the proportion of in-sample O wins tends to improve. Section 6.1.5.1.2 alluded to this phenomenon: the parameter β can be interpreted as a means of amplifying the likelihood ratio in Equation 3, yielding the modified expression $\left(\frac{p(\mathcal{D}|\boldsymbol{\theta}^*)}{p(\mathcal{D}|\boldsymbol{\theta}^{(j)})}\right)^\beta$. Increasing β makes the Markov chain more inclined to accept proposed solutions $\boldsymbol{\theta}^*$ that yield higher objective values, given the proportionality $p(\mathcal{D} | \boldsymbol{\theta}) \propto \text{Obj}(\boldsymbol{\theta})$, thereby making the MCMC sampler more likelihood-driven.

Furthermore, as elucidated in Section 5, it was justified that the MCMC sampler should be made more likelihood-driven, as failure to do so could result in an indefinite contraction of $\boldsymbol{\theta}$ toward zero. This phenomenon is corroborated by Figure 16, which depicts the trajectory of $\|\boldsymbol{\theta}^{(j)}\|^2$ across the 80,000 post-burn-in iterations. For small values of β , we observe that the norm $\|\boldsymbol{\theta}^{(j)}\|^2$ exhibits a slow, monotonic decline across iterations: a strong indication that the Markov chain remains in its transient phase and has not yet reached stationarity. We posit that, this monotonic decay may reflect more than just delayed convergence. At low values of β , the pseudo-likelihood, and by extension the conditional posterior $p(\boldsymbol{\theta} | \sigma_\theta^2, \mathcal{D})$ in Equation 17, becomes too diffuse to meaningfully constrain the parameter space. As a result, the data exerts minimal influence over the proposed solution $\boldsymbol{\theta}^{(j)}$, and the conditional posterior is effectively dominated by the prior $p(\boldsymbol{\theta} | \sigma_\theta^2)$. In this setting, the likelihood becomes inconsequential, and the sampling dynamics are driven almost entirely by the prior structure. Consequently, the MH acceptance probabilities α_θ favour proposals $\boldsymbol{\theta}^{(j)}$ that reduce the norm $\|\boldsymbol{\theta}^{(j)}\|^2$: as is evident by Equation 17, where a reduction in $\|\boldsymbol{\theta}^{(j)}\|^2$ results in larger conditional posterior values thereby guiding the Markov chain to search in areas where low $\|\boldsymbol{\theta}^{(j)}\|^2$ values are obtained. This behavior is a direct reflection of what the conditional posterior, being flat and prior-dominated, is prescribing. The sampler is “doing its job”: in the absence of strong likelihood information, the proposals $\boldsymbol{\theta}^{(j)}$ are simply contracting toward the origin under the influence of the prior.

Additionally, the multivariate effective sample sizes (ESS) reported in Table 6 provide evidence of satisfactory mixing. All reported ESS values exceed the commonly accepted threshold of 100 (as recommended by Vehtari et al. (2021), Section 4), which supports the claim of efficient exploration. However, it is important to emphasize that ESS is a meaningful diagnostic only after convergence has been attained. In particular, for small values of β , we observe, via the continued drift in $\|\boldsymbol{\theta}^{(j)}\|^2$, that the chain remains in a transient phase, and thus has not yet fully converged to its stationary distribution. Consequently, while we report ESS values at the end of the 80,000 usable MCMC iterations for completeness, we interpret them with caution in the low- β regime.

Furthermore, for low values of β , where the MCMC samples exhibit non-stationary behavior in $\|\boldsymbol{\theta}^{(j)}\|^2$, the resulting marginal distribution of $\sigma_\theta^2 | \mathcal{D}$ deviates from an inverse-gamma form. Since the conditional posterior $\sigma_\theta^2 | \boldsymbol{\theta}, \mathcal{D} \sim \text{Inv-Gamma}(a + \frac{S}{2}, b + \frac{\|\boldsymbol{\theta}\|^2}{2})$ where $a, b \approx 0$, we know that if $\|\boldsymbol{\theta}^{(j)}\|^2$ fluctuates around some constant c , the marginal $\sigma_\theta^2 | \mathcal{D}$ should also be inverse-gamma distributed with constant shape and scale parameters. That is, if $\|\boldsymbol{\theta}^{(j)}\|^2 \approx c$, then $\sigma_\theta^2 | \mathcal{D} \sim \text{Inv-Gamma}(a + \frac{S}{2}, b + \frac{c}{2})$ as illustrated in Figure 16.

Now, Table 6 suggests that, in order to obtain solutions yielding in-sample performance comparable to that achieved by traditional optimization methods such the GA in Section 6.2.5, the likelihood sharpness parameter β must be increased to sufficiently concentrate the conditional posterior $p(\boldsymbol{\theta} | \sigma_\theta^2, \mathcal{D})$. This

ensures that the sampler is more decisively guided by the pseudo-likelihood, that is, made sufficiently likelihood-driven, resulting in proposed solutions $\boldsymbol{\theta}^{(j)}$ whose norms $\|\boldsymbol{\theta}^{(j)}\|^2$ stabilize across MCMC iterations. Such stabilization indicates convergence toward a dominant mode of the conditional $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$. Empirically, for the tic-tac-toe problem studied here, values of $\beta \geq 100$ appear to meet this threshold, yielding both stable posterior behavior (see the top panel of Figure 16) and competitive in-sample performance. However, a practical consideration arises when selecting an appropriate value of β . As β becomes too large, the likelihood, and consequently the conditional posterior, becomes exceedingly sharp, leading to steep gradients around high-valued objective regions. In such cases, the MCMC chain is prone to becoming effectively “trapped” in these narrow peaks, as proposed moves away from the current mode receive vanishingly small acceptance probabilities α_θ . This occurs because the prior no longer exerts sufficient regularizing influence to counterbalance the likelihood’s dominance, unlike in regimes where β is moderate and the posterior retains a broader structure. Hence, while increasing β can give rise to solutions with improved in-sample performance, it must be done judiciously to avoid compromising the chain’s ability to explore alternative dominant modes of the conditional.

Additionally, we note from Table 6, that the maximum of the log of the conditional posterior, $\log(p(\boldsymbol{\theta} \mid \mathcal{D}, \sigma_\theta^2))$, increases as β increases. This behavior is substantiated by Equation 17, where it follows directly that increasing β increases the contribution of the likelihood to the conditional posterior, thereby sharpening the overall posterior landscape.

	$\frac{100}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)$			
Sharpness β	In-Sample	Out-of-Sample	$\max\{\log(p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D}))\}$	ESS
0.1	56	50.08	-254.1965	2332.9560
1	70	59.86	-237.8046	2340.9900
10	63	56.68	-243.4494	4222.4970
50	86	72.98	-200.5137	690.1510
100	98	72.62	-162.3267	927.7915
1000	96	68.58	733.5609	927.4778

Table 6: The normalized number of O wins, as a percentage $\left(\frac{100}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)\right)$ for in-sample ($K = 100$) and out-of-sample ($K = 10,000$) sets across likelihood sharpness $\beta \in \mathbb{R}^+$ using $\mathbf{model}^{(II)}(\mathbf{a}^0, \hat{\boldsymbol{\theta}}_\nu^{\text{GA},(II)})$ and $\sigma_{\text{Init}}^2 = 10$ for 100,000 MCMC iterations and burn-in of 20,000 iterations.

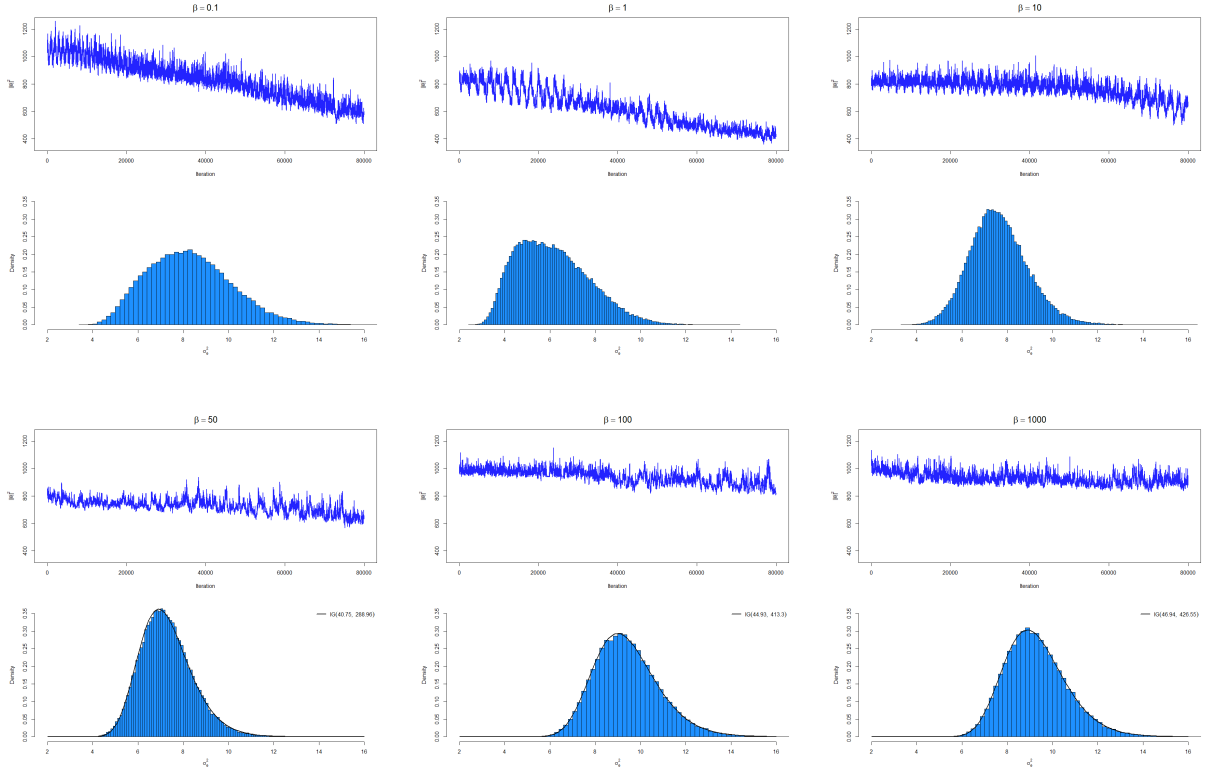


Figure 16: $\|\theta^{(j)}\|^2$ for $j = 1, \dots, 80,000$ (post burn-in) with distribution of marginal $\sigma_\theta^2 \mid \mathcal{D}$ for varying likelihood sharpness β using $\text{model}^{(\text{II})}(\mathbf{a}^0, \hat{\theta}_\nu^{\text{GA}, (\text{II})})$ and $\sigma_{\text{Init}}^2 = 10$.

With regard to the amount of regularization inferred from the training set, where regularization is represented by the dispersion parameter $\sigma_\theta^2 \propto \frac{1}{\nu}$, we may draw meaningful conjectures from the marginal distributions of $\sigma_\theta^2 \mid \mathcal{D}$ shown in Figure 16. Specifically, the variation in these distributions across different values of sharpness parameter β , suggests that different β values inherently induce different degrees of regularization, which are in turn reflected in the MAP estimates of the parameters (as discussed in Section 3.1.1, MAP estimates naturally encode a level of regularization inferred by the training set, since the marginal posterior $p(\theta_i \mid \mathcal{D})$ essentially integrates over both the remaining parameters θ_{-i} and the dispersion parameter σ_θ^2). That is, by examining the marginal distribution of $\sigma_\theta^2 \mid \mathcal{D}$, we gain insight into the implied concentration of the regularization strength ν . Interestingly, we observe that beyond a certain threshold, approximately at $\beta = 100$ for our tic-tac-toe problem, these inverse-gamma marginals appear to converge in shape and scale, as evidenced by the similarity between the distributions for $\beta = 100$ and $\beta = 1000$ in Figure 16. This suggests that the strength of regularization inferred by the training set saturates beyond a certain level of likelihood sharpness.

Even more revealing with respect to the amount of regularization inferred by the training set, is the influence of the initial variance σ_{Init}^2 used to initialize the Markov chain. Recall that the initial proposal $\theta^{(1)}$ is sampled from a multivariate normal distribution, $\mathcal{N}(\mathbf{0}_{S \times 1}, \sigma_{\text{Init}}^2 \mathbf{I}_S)$. Notably, when fixing $\beta = 100$ to ensure stability of $\|\theta^{(j)}\|^2$, the resulting marginal distributions of $\sigma_\theta^2 \mid \mathcal{D}$ remain inverse-gamma distributed with approximately constant shape parameters but exhibit increasing rate parameters as σ_{Init}^2 increases, as shown in Table 7. This trend is visually supported in Figure 17 (note that the axis scales vary across plots), which displays the inverse-gamma marginals $\sigma_\theta^2 \mid \mathcal{D}$ for increasing values of σ_{Init}^2 . It

is evident that both the mean and variance of the resulting distributions shift upward: the distributions move to the right and become more compressed. While this “squashing” effect may not be immediately noticeable without paying attention to the axis scales, it highlights an important insight: since $\sigma_\theta^2 \propto \frac{1}{\nu}$, increasing the initial variance σ_{Init}^2 results in the inference of a weaker regularization strength from the training set. In other words, the amount of regularization implicitly inferred by the training set is not only a function of the likelihood sharpness β , but is also influenced by the choice of initial dispersion σ_{Init}^2 .

	$\frac{100}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)$					
Initial Variance σ_{Init}^2	In-Sample	Out-of-Sample	Shape	Rate	ESS	
0.1	92	71.18	41.7518	3.1350	414.6554	
1	88	65.40	46.2323	41.3117	884.6408	
10	98	72.62	44.9264	413.2980	927.7915	
100	92	68.92	47.28048	5704.8909	488.9163	

Table 7: The normalized number of O wins, as a percentage $\left(\frac{100}{K} \sum_{k=1}^K \mathbb{I}(\rho_{T_k}(\boldsymbol{\theta}) = +1)\right)$ for in-sample ($K = 100$) and out-of-sample ($K = 10,000$) sets across various initial variances σ_{Init}^2 accompanied by shape and rate parameters of marginal $\sigma_\theta^2 \mid \mathcal{D} \sim \text{Inv-Gamma}$ for likelihood sharpness $\beta = 100$.

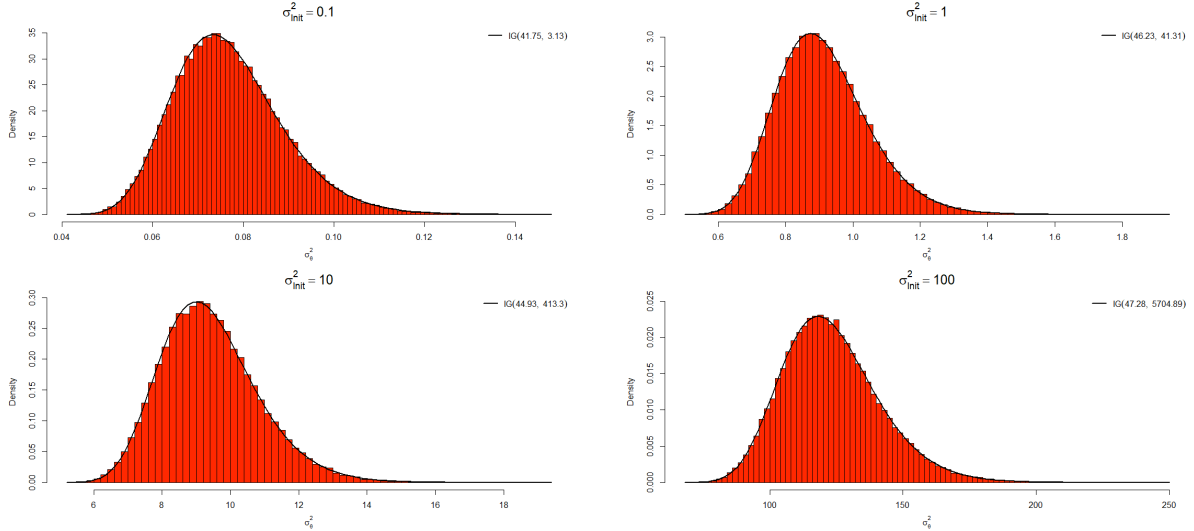


Figure 17: Distribution of marginal $\sigma_\theta^2 \mid \mathcal{D} \sim \text{Inv-Gamma}(\text{Shape}, \text{Rate})$ for varying initial variance σ_{Init}^2 (plots are on different x and y scales).

Now as previously argued in Section 3.1.1, the motivation for adopting a hierarchical Bayesian framework, wherein a dispersion parameter σ_θ^2 is introduced via a prior such that $\sigma_\theta^2 \propto \frac{1}{\nu}$, was to allow the training set to inform the degree of regularization. However, the preceding analysis including Section 6.1.7 reveals a tension in this reasoning. Specifically, the pseudo-likelihood form, the likelihood sharpness β and the initial dispersion σ_{Init}^2 are user-specified hyperparameters that exert a substantial influence on the marginal posterior distribution of $\sigma_\theta^2 \mid \mathcal{D}$. This dependence implies that the extent of regularization is not fully inferred by the training set, but is instead strongly shaped by likelihood and prior design choices: particularly the choices of β and σ_{Init}^2 . As such, we may question whether the hierarchical structure gen-

uinely facilitates data-driven regularization or whether it merely reintroduces user-defined regularization through a more complex inferential route. From this perspective, the use of a Bayesian hierarchical model for the sole purpose of inferring σ_θ^2 from the training set may appear unnecessary, especially when the same effect could be achieved by explicitly fixing σ_θ^2 (and hence regularization strength ν) to a chosen value. In this light, one might argue that the two-block MCMC procedure employed here, through which the Bayesian hierarchical model is implemented, is actually just the *user* inferring a specific regularization strength but with “extra steps”.

The remainder of the study further examines the implications of increased likelihood sharpness. To facilitate this analysis, the first block of the two-block MCMC scheme described in Section 3.1.1 is replaced with an iterative optimization procedure, and this hybrid method is compared to the original algorithm. The comparison is illustrated using three arbitrary objective functions derived from the classic S-17 blackjack game. The environments and the corresponding role of the neural network as a control mechanism are described in the subsequent sections.

6.3 The Blackjack Problem I: Controlling Player Decisions

In this section, we broaden the range of reinforcement learning problem types considered by introducing a stochastic, partially observable environment, specifically in the form of blackjack. Blackjack represents a substantially more complex reinforcement learning environment, characterized by uncertainty, delayed rewards, and a richer decision process than the previous two reinforcement learning tasks considered. As a result, effective learning requires greater model capacity, making the role of regularization both more necessary and more informative. Blackjack is therefore included to ensure that the analysis considers a task in which higher neural network capacity is required, highlighting how two-block MCMC behaves as model complexity increases. Furthermore, in this section, a neural network is used as the control policy to determine the player’s actions at each decision point.

Formally, blackjack is a card game in which a player competes directly against the dealer (rather than against other players at the table). In the S-17 variant, the dealer must stand on all soft 17s (a hand-value of 17 that includes an Ace counted as 11) and may draw additional cards only until reaching this threshold. The player’s objective is to obtain a hand-value closer to 21 than the dealer’s without exceeding 21, in which case the player busts and loses immediately. A natural blackjack (an initial two-card hand-value of 21) pays 3 : 2, while the dealer wins whenever achieving a higher valid hand-value than the player.

6.3.1 Encoding

We define a hand of blackjack as complete at time k , where the player’s cards are dealt at times $\tau_k = 1, \dots, T_k^{(p)}$ and recorded as the vector $\mathbf{c}_k^{(p)} = \left[c_{k,1}^{(p)}, \dots, c_{k,T_k^{(p)}}^{(p)} \right]' \in \{1, 2, \dots, 10\}^{T_k^{(p)}}$. An Ace is encoded as 1 (not to be confused with its potential hand-value of either 1 or 11), face cards (J , Q , and K) are encoded as 10, and all other cards retain their nominal face values. The dealer’s single upcard is denoted by the scalar $c_k^{(d,\text{up})}$. The dealer’s hole card (revealed only after the player’s turn ends) at time $\tau_k = T_k^{(p)} + 1$ is denoted by the scalar $c_k^{(d,\text{hole})}$, and any additional dealer cards drawn at times $\tau_k = T_k^{(p)} + 2, \dots, T_k$ are recorded as the vector $\mathbf{c}_k^{(d)} = \left[c_{k,T_k^{(p)}+2}^{(d)}, \dots, c_{k,T_k}^{(d)} \right]' \in \{1, 2, \dots, 10\}^{T_k - T_k^{(p)} - 1}$.

We treat both the player's and the dealer's turns as two ordered sequences of events, and define the card history for the k^{th} blackjack hand as $\mathcal{H}_k = \left[\left(\mathbf{c}_k^{(p)} \right)', c_k^{(d,up)}, c_k^{(d,hole)}, \left(\mathbf{c}_k^{(d)} \right)' \right]' \in \{1, 2, \dots, 10\}^{T_k}$ where $\mathcal{H}_k$ represents all cards eventually observed by the player in the k^{th} hand. Furthermore, we define the complete card history observed by the player at the end of the k^{th} hand (from the 1st hand of blackjack) as $\mathcal{H}_k = [\mathcal{H}_1', \mathcal{H}_2', \dots, \mathcal{H}_k']'$. Additionally, we define the complete set of the dealer's cards observed in the k^{th} hand as $\mathbf{c}_k^{(d,all)} = \left[c_k^{(d,up)}, c_k^{(d,hole)}, \left(\mathbf{c}_k^{(d)} \right)' \right]' \in \{1, 2, \dots, 10\}^{T_k - T_k^{(p)}}$. Furthermore, we represent the partial player hand during the k^{th} blackjack hand at time τ_k , where $\tau_k \leq T_k^{(p)}$, as $\mathbf{c}_{k,1:\tau_k}^{(p)} = \left[c_{k,1}^{(p)}, \dots, c_{k,\tau_k}^{(p)} \right]' \in \{1, 2, \dots, 10\}^{\tau_k}$. The corresponding partial card state including the dealer's upcard is then given by $\mathbf{c}_{k,\tau_k}^{partial} = \left[\left(\mathbf{c}_{k,1:\tau_k}^{(p)} \right)', c_k^{(d,up)} \right]' \in \{1, 2, \dots, 10\}^{\tau_k + 1}$.

6.3.2 The hand's outcome

We define the hand-value of a player's or dealer's cards, $|\mathbf{c}| = R$ as such:

$$h(\mathbf{c}) = \begin{cases} \sum_{i=1}^R c_i + 10, & \text{if } c_i = 1 \text{ and } \sum_{i=1}^R c_i + 10 \leq 21, \\ \sum_{i=1}^R c_i & \text{otherwise.} \end{cases}$$

which implies that if an Ace is present ($c_i = 1$) and the total hand-value does not exceed 21 when the Ace is treated as 11, then the Ace is valued at 11. Otherwise, it is valued as 1. We record the bet multiplier at the conclusion of the k^{th} hand, s_k , as such:

$$s_k = \begin{cases} +2, & \text{if } h(\mathbf{c}_k^{(p)}) > h(\mathbf{c}_k^{(d,all)}) \text{ and } h(\mathbf{c}_k^{(p)}) \leq 21 \text{ and player chose to double-down where } |\mathbf{c}_k^{(p)}| = 3, \\ +2, & \text{if } h(\mathbf{c}_k^{(p)}) \leq 21 \text{ and } h(\mathbf{c}_k^{(d,all)}) > 21 \text{ and player chose to double-down,} \\ +1.5, & \text{if } h(\mathbf{c}_k^{(p)}) = 21 \text{ for } |\mathbf{c}_k^{(p)}| = 2 \text{ and } h\left(\left[c_k^{(d,up)}, c_k^{(d,hole)} \right]'\right) \neq 21, \\ +1, & \text{if } h(\mathbf{c}_k^{(p)}) > h(\mathbf{c}_k^{(d,all)}) \text{ and } h(\mathbf{c}_k^{(p)}) \leq 21 \text{ and player did not choose to double-down,} \\ +1, & \text{if } h(\mathbf{c}_k^{(p)}) \leq 21 \text{ and } h(\mathbf{c}_k^{(d,all)}) > 21 \text{ and player did not choose to double-down,} \\ -2, & \text{if } h(\mathbf{c}_k^{(p)}) > 21 \text{ and player chose to double-down where } |\mathbf{c}_k^{(p)}| = 3, \\ -1, & \text{if } h(\mathbf{c}_k^{(p)}) > 21 \text{ and player did not choose to double-down,} \\ -1, & \text{if } h(\mathbf{c}_k^{(p)}) < h(\mathbf{c}_k^{(d,all)}) \text{ and } h(\mathbf{c}_k^{(d,all)}) \leq 21, \\ -0.5, & \text{if player surrendered hand,} \\ 0, & \text{otherwise.} \end{cases}$$

6.3.3 Control: Player decision

We control player decisions/actions at time $\tau_k = 1, \dots, T_k^{(p)}$ for the k^{th} hand of blackjack through the means of the control vector $\mathbf{ct}(\mathbf{c}_{k,\tau_k-1}^{partial}, \boldsymbol{\theta}^{Decision}) \in \mathcal{A}_{\tau_k} \subseteq \{\text{Stay, Hit, Split, Surrender, Double-Down}\}$ for some parameter configuration $\boldsymbol{\theta}^{Decision} \in \mathbb{R}^R$. We note that $\mathcal{A}_{\tau_k}$ denotes the subset of admissible actions available at time $\tau_k \leq T_k^{(p)}$, immediately prior to the player's decision. Splitting is allowed only if the player's first two cards are identical $c_{k,1}^{(p)} = c_{k,2}^{(p)}$ and can only occur at $\tau_k = 2$, and surrendering and doubling down can only occur at $\tau_k = 2$. Moreover, the player's decision at time τ_k is conditioned on the

cards observed up to time $\tau_k - 1$. Furthermore, the player action selection is probabilistic and derived from a softmax distribution over logits. Hence, for ℓ_a being the logit score for any valid action $a \in \mathcal{A}_{\tau_k}$, the probability of selecting that action is:

$$\sigma_L(a \mid \mathbf{c}_{k, \tau_k-1}^{\text{partial}}, \boldsymbol{\theta}^{\text{Decision}}) = \frac{\exp(\ell_a)}{\sum_{a' \in \mathcal{A}_{\tau_k}} \exp(\ell_{a'})}.$$

The selected action a^* corresponds to the action with the highest probability, that is, $a^* = \operatorname{argmax}_{a \in \mathcal{A}_{\tau_k}} \sigma_L(a \mid \mathbf{c}_{k, \tau_k-1}^{\text{partial}}, \boldsymbol{\theta})$. Invalid actions (i.e., $a \notin \mathcal{A}_{\tau_k}$) are assigned $\ell_a = -\infty$, ensuring a zero probability is attributed to the invalid action. Now the interface between a model and the player action is undergone through this control vector for which $\mathbf{ct} : (\mathbf{c}_{k, \tau_k-1}^{\text{partial}}, \boldsymbol{\theta}^{\text{Decision}}) \rightarrow \mathbf{model} \left(\boldsymbol{\Omega} \left(\mathbf{c}_{k, \tau_k-1}^{\text{partial}} \right), \boldsymbol{\theta}^{\text{Decision}} \right) \xrightarrow{\sigma_L(\cdot)} a^* \in \mathcal{A}_{\tau_k}$ where $\boldsymbol{\theta}^{\text{Decision}}$ is fixed throughout all $k = 1, 2, \dots, K$ blackjack hands for all time $\tau_k \leq T_k^{(p)}$, and all player decisions/actions are based on this fixed parameterization $\boldsymbol{\theta}^{\text{Decision}}$. In the framework of using a neural network as our model, we define $\boldsymbol{\Omega} : \mathbf{c}_{k, \tau_k-1}^{\text{partial}} \rightarrow \mathbf{a}^0 \in \mathbb{R}^{d_0}$ which signifies the vector of input nodes for time $\tau_k \leq T_k^{(p)}$. Furthermore, $\boldsymbol{\theta}^{\text{Decision}}$ are the weights and biases of the neural network, $\mathbf{w}^{\text{Decision}} \in \mathbb{R}^R$.

6.3.3.1 Feature engineering We construct the feature vector using three inputs: the player's hand-value at time $\tau_k - 1$, the dealer's visible upcard for the k^{th} hand, and a binary indicator denoting the presence of a usable ace in the player's hand (defined as an ace valued at 11 rather than 1). This representation aligns with the standard structure of established blackjack strategy tables, which prescribe optimal actions based on this triplet of information. By adopting this input configuration, we leverage a format that has been extensively validated through decades of empirical and theoretical research in blackjack literature.

Hence, our 1^{st} input node is defined as $a_1^0 = \frac{h(\mathbf{c}_{k, 1: (\tau_k-1)}^{(p)})}{21}$ which represents the player's hand-value at time $\tau_k - 1$, before the player makes a decision at time τ_k , normalized by 21. The 2^{nd} input node is given by $a_2^0 = \frac{h(c_k^{(d, \text{up})})}{10}$ which encodes the dealer's upcard value as a fraction of 10. Our 3^{rd} input node is a binary indicator: $a_3^0 = \mathbb{I} \left(1 \in \mathbf{c}_{k, 1: (\tau_k-1)}^{(p)} \cap \left(\sum_{c \in \mathbf{c}_{k, 1: (\tau_k-1)}^{(p)}} c + 10 \right) \leq 21 \right)$ which denotes the presence of a usable ace in the player's hand up to time $\tau_k - 1$.

6.3.4 The arbitrary objective

Consider an arbitrary objective where, for a given parameter configuration $\boldsymbol{\theta} = \boldsymbol{\theta}^{\text{Decision}} \in \mathbb{R}^R$ and after playing K number of blackjack hands, we record the ROI, where the reward for the k^{th} blackjack hand is denoted as $s_k(\boldsymbol{\theta}) \cdot \text{bet}_k$ ($\text{bet}_k = 1$ is the initial bet-size for each k^{th} blackjack hand for simplicity, and scales accordingly for split and double-down actions). Hence:

$$\begin{aligned} \operatorname{argmax}_{\boldsymbol{\theta}} \text{Obj}(\boldsymbol{\theta}) &= \operatorname{argmax}_{\boldsymbol{\theta}} \frac{\sum_{k=1}^K (s_k(\boldsymbol{\theta}) \cdot \text{bet}_k)}{\sum_{k=1}^K \text{bet}_k} \\ &= \operatorname{argmax}_{\boldsymbol{\theta}} \text{ROI}(\boldsymbol{\theta}). \end{aligned}$$

By including L2 regularization, our L2 penalized objective becomes:

$$\operatorname{argmax}_{\boldsymbol{\theta}} (\text{ROI}(\boldsymbol{\theta}) - \nu \|\boldsymbol{\theta}\|^2).$$

6.3.5 Results: Effects of regularization

To train the blackjack agent, we simulate a sequence of $K = 1000$ hands of blackjack, termed a “night”. The training set is fully determined by an initial seed ω_0^{Train} that governs the shuffle of the D_0 -deck shoe at the start of the training night. Formally, we define a sequence of the D_0 deck shoe as $\mathcal{S}(\omega)$: indicating a specific permutation of the D_0 -deck shoe, where ω is a random seed. The initial training shoe is then $\mathcal{S}(\omega_0^{\text{Train}})$. Our shoe utilizes a reshuffle threshold (deck penetration) of 50%, hence the shoe is reshuffled whenever the number of remaining cards falls below $\frac{1}{2}52 \cdot D_0$. Each reshuffle occurs by advancing the random seed deterministically using a known rule (for example, $\omega_{i+1} = f(\omega_i)$, for some deterministic function f), hence ensuring reproducibility across runs. Thus, while each reshuffle produces a distinct card ordering, the sequence of reshuffles is deterministic and reproducible given the initial seed ω_0 .

Now, we define our test set as 10,000 nights where each night is initialized with a distinct shoe shuffle $\mathcal{S}(\omega_{0,n}^{\text{Test}})$ for night n where $\omega_{0,n}^{\text{Test}} \neq \omega_0^{\text{Train}}$ for all $n = \{1, \dots, 10000\}$. The blackjack rules still enforce a deck penetration of 50%, with reshuffling triggered accordingly, hence we define ω_i^{Train} and $\omega_{i,n}^{\text{Test}}$ to be the seed values used for reshuffling in the training night and test nights respectively. Hence to ensure no shoe permutations in the test nights overlap with or is derived from those in training night, we ensure $\omega_i^{\text{Train}} \neq \omega_{i,n}^{\text{Test}}$ implying $\mathcal{S}(\omega_i^{\text{Train}}) \neq \mathcal{S}(\omega_{i,n}^{\text{Test}}) \forall i, n$, ensuring disjoint training and test environments.

Furthermore, to evaluate the impact of the regularization strength ν , we apply the estimator $\hat{\theta}_\nu^{\text{GA}}$ to the test set and assess both in-sample and out-of-sample hit-rates, as reported in Table 8, across various values of ν . Additionally, the mean $\mu_{ROI\%}$ and standard deviation $\sigma_{ROI\%}$ of ROI distributions (normally distributed as illustrated in Figure 19) are reported for the 10,000 out-of-sample nights. The corresponding player decision tables are presented in Figure 18, from which we observe that the regularization strengths yielding the highest hit-rates, on the test set, tend to produce decision policies that closely resemble the S17 strategy, or more broadly, strategies anchored around a hand-value threshold of 17. Nevertheless, none of the tested regularization strengths result in a hit-rate that surpasses those achieved by the standard S17, H17, or Basic Strategy policies reported in Table 9. Furthermore, no consistent or interpretable pattern emerges in the player decision tables as the regularization strength ν is varied; the resulting policies appear to change irregularly across different values of ν . Notably, Table 8 reveals a positive association between higher hit rates and improved $\mu_{ROI\%}$ (despite all policies exhibiting negative $\mu_{ROI\%}$). A notably striking observation is that the standard deviations of the ROI distributions, $\sigma_{ROI\%}$, for all solutions in Table 8 are approximately the same, around 3.1%. This being the case except for $\hat{\theta}_{\nu=0.0009}^{\text{GA}}$, which exhibits a slightly higher $\sigma_{ROI\%}$ and, coincidentally, the lowest $\mu_{ROI\%}$. The decision tables suggests this may be due to it being the only solution that recommended doubling-down on certain hands.

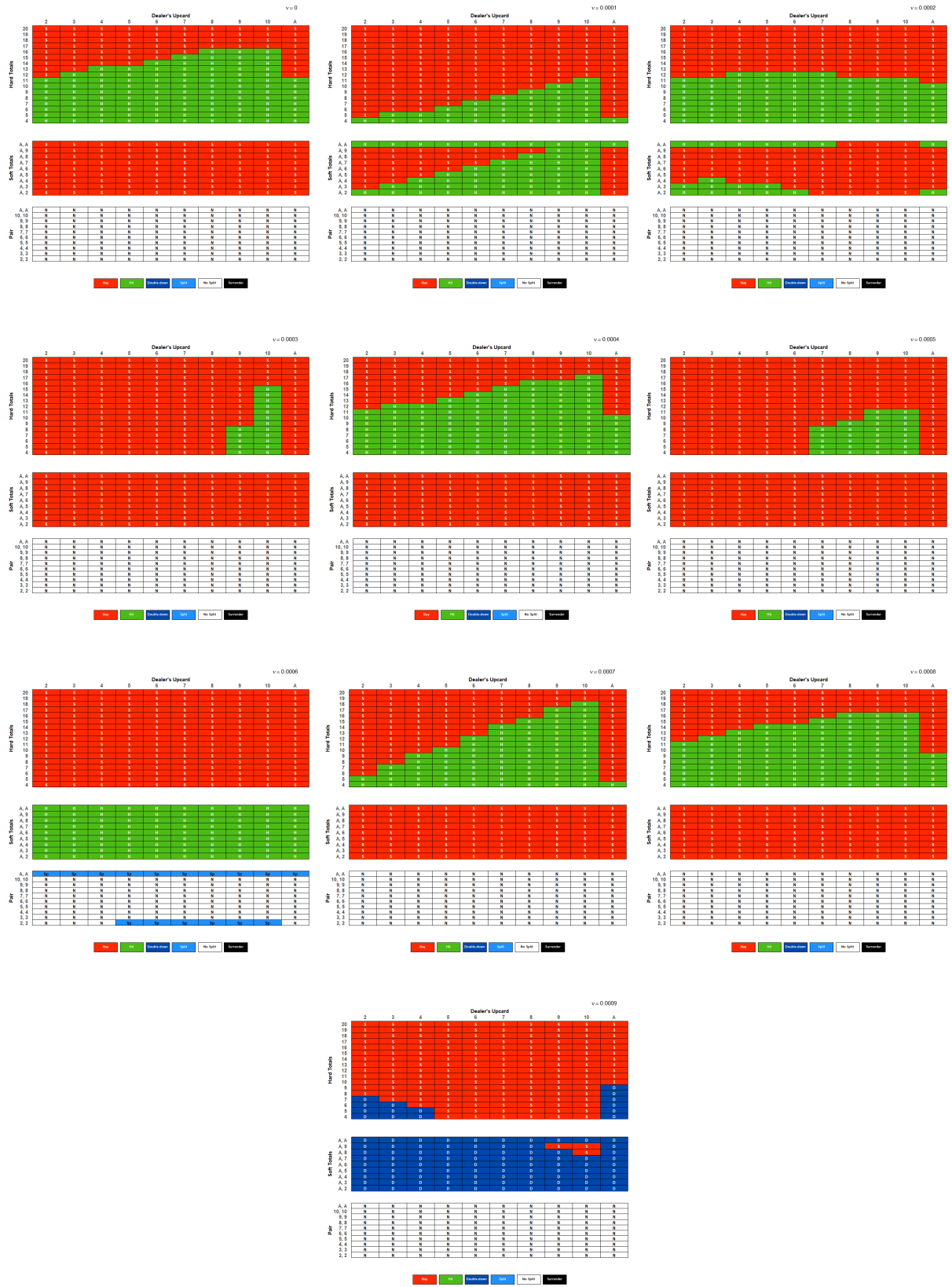


Figure 18: Player decision tables for varying ν (removal of surrender table seeing as, given the range of ν , player never opts to surrender).

Regularization Strength ν	In-Sample		Out-of-Sample		
	Hit-Rate	ROI %	Hit-Rate	$\mu_{ROI\%}$	$\sigma_{ROI\%}$
0.0000	0.4868	-0.1500	0.4575	-5.5064	3.1067
0.0001	0.4708	-2.8000	0.4245	-11.9999	3.1051
0.0002	0.4840	-0.6500	0.4464	-7.7526	3.1170
0.0003	0.4397	-9.1000	0.4222	-12.3205	3.0765
0.0004	0.4695	-3.1500	0.4535	-6.2797	3.1213
0.0005	0.4542	-5.9500	0.4215	-12.5486	3.1114
0.0006	0.4274	-11.7500	0.4058	-15.8433	3.1615
0.0007	0.4490	-7.5500	0.4342	-9.9631	3.1131
0.0008	0.4836	-0.6500	0.4543	-6.0970	3.1112
0.0009	0.4343	-8.6500	0.4098	-16.5464	3.5682

Table 8: Regularization strength ν vs. hit-rate (in- and out-of-sample).

Decision Policy	In-Sample		Out-of-Sample		
	Hit-Rate	ROI %	Hit-Rate	$\mu_{ROI\%}$	$\sigma_{ROI\%}$
Purely Random	0.2704	-37.85	0.2597	-41.8837	3.1961
Random Stay/Hit	0.3442	-27.35	0.3331	-29.6443	3.0478
S17	0.4266	-11.05	0.4578	-5.3242	3.0711
H17	0.4190	-12.35	0.4596	-5.0095	3.0877
Basic Strategy	0.4430	-4.25	0.4630	-0.6716	3.5657

Table 9: Hit-rate and ROI performance of common blackjack decision policies.

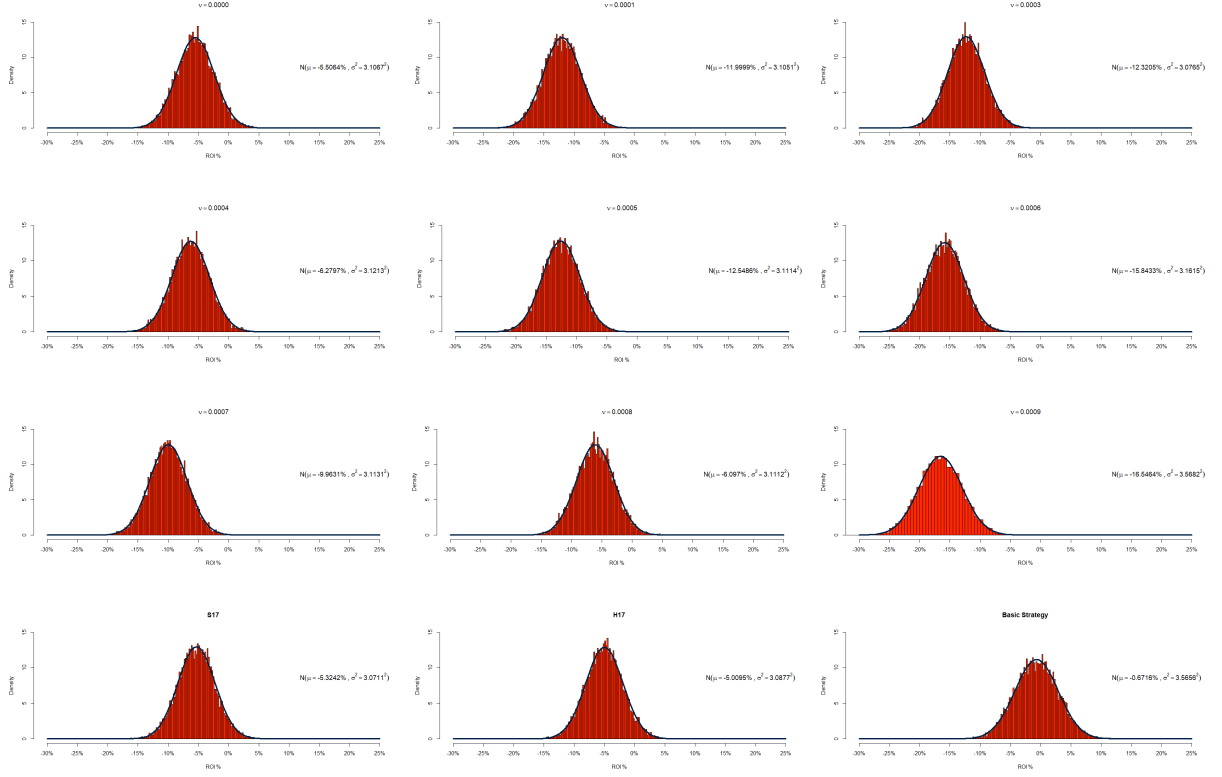


Figure 19: Out-of-sample ROI distributions for varying ν , corresponding to Table 8 and ROI distributions of common blackjack decision policies corresponding to Table 9.

6.4 The Blackjack Problem II: Controlling Bet Size

In this section, we focus exclusively on controlling the bet size (placed by the player before any cards are drawn), without influencing the player’s in-game decisions as in Section 6.3. That is to say, a neural network is used as a control mechanism to determine player bet sizing in the same stochastic, partially observable blackjack environment used previously. Accordingly, player decisions are assumed to follow Basic Strategy⁹, meaning the bet multiplier s_k is determined solely by Basic Strategy recommendations, rather than being governed by θ^{Decision} as in Section 6.3.

6.4.1 Control: Bet size

We control the amount to bet for the k^{th} blackjack hand (which occurs before any cards are dealt for the k^{th} hand), through the means of the control vector $\text{ct}(\mathcal{H}_{k-1}, \theta^{\text{Bet}}) \in [0, 1]$ for some parameter configuration $\theta^{\text{Bet}} \in \mathbb{R}^S$ (note the index for the card history vector $\mathcal{H}$ is $k-1$ to indicate that betting occurs before any cards are observed for the k^{th} blackjack hand). Now the interface between a model and the bet sizing is undergone through this control vector for which $\text{ct} : (\mathcal{H}_{k-1}, \theta^{\text{Bet}}) \rightarrow \text{model}(\Omega(\mathcal{H}_{k-1}), \theta^{\text{Bet}}) \xrightarrow{\sigma_L(\cdot)} [0, 1]$ where θ^{Bet} is fixed throughout all $k = 1, 2, \dots, K$ blackjack hands, and betting decisions at each k^{th} hand are based on this fixed parameterization θ^{Bet} . In the framework of using a neural network as our model, we define $\Omega : \mathcal{H}_{k-1} \rightarrow \mathbf{a}^0 \in \mathbb{R}^{d_0}$ which signifies the vector of input nodes for the t^{th} hand. Furthermore, θ^{Bet} are the weights and biases of the neural network, $\mathbf{w}^{\text{Bet}} \in \mathbb{R}^S$, and $\sigma_L(\cdot)$ denotes a sigmoid activation function.

⁹See Appendix D for Basic Strategy rules.

6.4.1.1 Feature engineering To encode the current state of the game in a manner conducive to learning, we propose a feature vector that captures both (i) the true count of the remaining deck as formally defined in Appendix F and (ii) a weighted summary of the distribution of unseen cards. The true count serves as a classical card-counting statistic reflecting the favourability of the shoe, while the weighted distribution provides fine-grained information about the residual card composition.

Specifically, we define the first input node of the neural network as the scaled true count $a_1^0 = \frac{TC_{k-1}}{3}$, where TC_{k-1} is the true count at time $k-1$ normalized by a constant (here, 3) to map the feature into a compact numerical range. To complement this, we construct additional features that reflect the remaining proportion of each card value in the shoe, weighted by their nominal face value. For cards $i = 2, \dots, 9$ and $i = 11$ we define $a_i^0 = \frac{4iD_0 - i \sum_{c \in \mathcal{H}_{k-1}} \mathbb{I}(c=i)}{4iD_0}$. This expression measures the remaining total “value-weighted mass” of card i , relative to its initial value-weighted mass across the full shoe. For ten-valued cards (10, J, Q, K), which occur with higher frequency (16 per deck), we define $a_{10}^0 = \frac{16iD_0 - 10 \sum_{c \in \mathcal{H}_{k-1}} \mathbb{I}(c=10)}{16iD_0}$. Furthermore, we define two distinct models that differ in the number of input nodes utilized. Now $\text{model}^{(I)}(a_1^0, \theta^{(I)})$ relies solely on the true count to create the sole input node a_1^0 . In contrast, $\text{model}^{(II)}(\mathbf{a}^0, \theta^{(II)})$ incorporates all 11 previously specified input nodes, which offers a richer characterization of card composition than the true count alone.

Hence, in the proposed neural network architecture, the input layer comprises of $d_0 = 11$ nodes, which collectively processes $\mathcal{H}_{k-1}$ to produce a singular output, denoted as the betting propensity, $bet_k \in [0, 1]$. This represents the relative confidence in wagering, where the actual bet size is $\tilde{bet}_k = 1 + 9 \times bet_k \in [1, 10]$, ensuring that a positive, nonzero amount is wagered on each hand (where 1 and 10 may be viewed as the table minimum and maximum bets, respectively). Additionally, in scenarios where the true count satisfies $TC_{k-1} = 0$ and the card history is empty, $|\mathcal{H}_{k-1}| = 0$ (indicating a reshuffled shoe), the betting strategy defaults to $bet_k = 0$, hence $\tilde{bet}_k = 1$. In such cases, the historical card information $\mathcal{H}_{k-1}$ is excluded from the control vector $\mathbf{ct}$, thereby bypassing its integration into the bet-size decision process.

6.4.2 The arbitrary objective

Consider an arbitrary objective where, for a given parameter configuration $\theta = \theta^{\text{Bet}} \in \mathbb{R}^S$ and after playing K number of blackjack hands, we record the ROI, where the reward for the k^{th} blackjack hand is denoted as $s_k \cdot \tilde{bet}_k(\theta)$ (where $\tilde{bet}_k(\theta)$ scales accordingly to split and double-down actions). Hence:

$$\begin{aligned} \arg\max_{\theta} \text{Obj}(\theta) &= \arg\max_{\theta} \frac{\sum_{k=1}^K (s_k \cdot \tilde{bet}_k(\theta))}{\sum_{k=1}^K \tilde{bet}_k(\theta)} \\ &= \arg\max_{\theta} \text{ROI}(\theta). \end{aligned}$$

By including L2 regularization, our L2 penalized objective becomes:

$$\arg\max_{\theta} (\text{ROI}(\theta) - \nu \|\theta\|^2).$$

6.4.3 Results: Effects of regularization

To evaluate the effect of varying regularization strengths ν on the learned betting behavior bet_k , we conduct a response curve analysis. For visualization purposes, we employ **model**^(I) that uses a single input node a_1^0 , representing the scaled true count of the shoe denoted in the previous section. The resulting response function is given by **model**^(I) $\left(a_1^0, \hat{\theta}_\nu^{\text{GA},(I)}\right) \rightarrow bet_k \in [0, 1]$ and is depicted in Figure 20.

As established in the blackjack literature (see also Appendix F), exploiting the game advantageously requires betting in proportion to the true count. Figure 20 reveals that the solutions $\hat{\theta}_\nu^{\text{GA},(I)}$, across most ν values considered, tend to produce elevated betting propensities bet_k predominantly at highly negative true counts. While a few values of ν do give rise to solutions which elicit high betting behavior at large positive true counts, as theoretically desirable, this behavior is not consistently observed for all solutions.

Now this behaviour is further demonstrated to be undesirable when examining Table 11, where the notion of betting proportional to the true count of the shoe is illustrated. Notably, an inconsistency arises: in-sample ROI performance of these theoretical-based strategies is surprisingly poor (particularly when compared to the performance of the random betting policy). This observation suggests that the training set may not be conducive to the effective training of $\hat{\theta}_\nu^{\text{GA},(I)}$. That is, this training set may not provide a sufficiently informative environment such that $\hat{\theta}_\nu^{\text{GA},(I)}$ learns to bet in accordance to the true count. The theoretical betting strategy’s superior performance is substantiated, however, when evaluating out-of-sample data, as presented in Table 11. Specifically, the mean ROI ($\mu_{ROI\%}$) values for the theory-driven strategies demonstrate a marked improvement, approaching break-even levels with greater consistency compared to learnt betting in Table 10.

Table 10 reveals that both **model**^(I) and **model**^(II) exhibit substantial underperformance on the out-of-sample set. On a few occasions, the solutions $\hat{\theta}_\nu^{\text{GA}}$ yield mean ROI values ($\mu_{ROI\%}$) that are only marginally higher than $\mu_{ROI\%}$ obtained from the purely random betting policy presented in Table 11. Unsurprisingly, none of the betting configurations derived from $\hat{\theta}_\nu^{\text{GA}}$ solutions outperform any of the true count-based betting strategies on the out-of-sample set, as reported in Table 11. These findings are consistent with the earlier observation that the training set may be ill-suited for facilitating effective learning based on established blackjack theory. Furthermore, we refrain from making general claims regarding comparative performance across varying values of ν between the two models, as the two models differ in complexity with respect to their number of input nodes: **model**^(I) utilizing a_1^0 as its feature vector and **model**^(II) utilizing $\mathbf{a}^0$ as its feature vector.

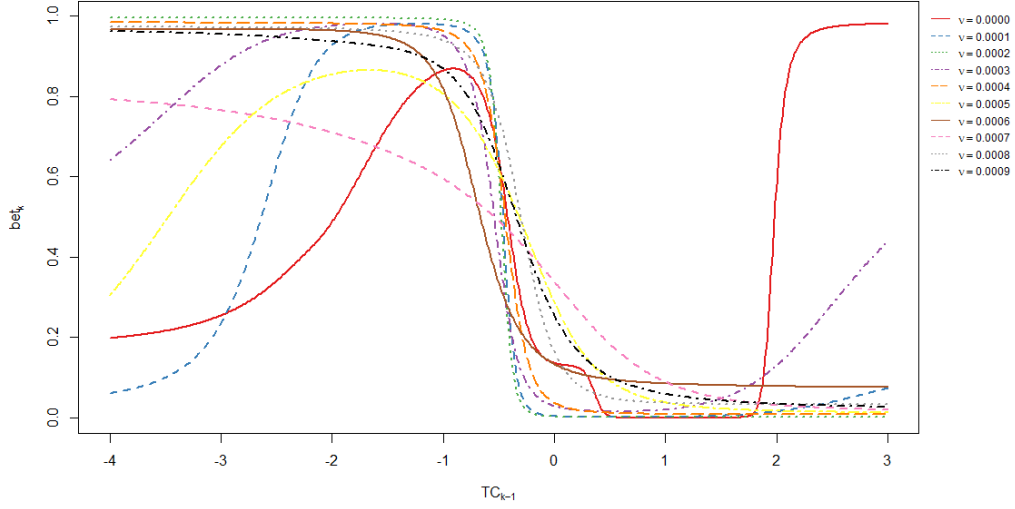


Figure 20: Betting propensity, bet_k , vs. the true count, TC_{k-1} , for various ν using $\text{model}^{(I)}(a_1^0, \hat{\theta}_\nu^{\text{GA},(I)})$.

Regularization Strength ν	$\text{model}^{(I)}(a_1^0, \hat{\theta}_\nu^{\text{GA},(I)})$			$\text{model}^{(II)}(\mathbf{a}^0, \hat{\theta}_\nu^{\text{GA},(II)})$		
	In-Sample	Out-of-Sample		In-Sample	Out-of-Sample	
	ROI %	$\mu_{ROI\%}$	$\sigma_{ROI\%}$	ROI %	$\mu_{ROI\%}$	$\sigma_{ROI\%}$
0.0000	0.7440	-0.7650	4.4523	14.6180	-0.9887	5.5638
0.0001	0.7670	-1.1536	5.1288	11.4806	-0.9207	5.2493
0.0002	-0.5578	-1.3047	5.1894	8.1637	-0.8404	4.7393
0.0003	-0.3340	-1.1145	4.7978	6.0955	-0.8418	5.0195
0.0004	-0.9778	-1.2743	4.9373	-2.0912	-0.8334	3.9489
0.0005	-0.9952	-1.1020	4.3136	-0.3641	-1.1610	4.4662
0.0006	-1.9658	-1.1819	4.5306	-3.9218	-0.6722	3.6046
0.0007	-2.6602	-1.0656	4.0409	-2.2319	-0.4890	4.1832
0.0008	-1.4639	-1.1926	4.5497	8.74736	-0.7584	4.8677
0.0009	-1.8479	-1.1604	4.3676	-3.3625	-0.7375	3.6303

Table 10: ROI in-sample and out-of-sample performance across regularization strengths ν using $\text{model}^{(I)}(a_1^0, \hat{\theta}_\nu^{\text{GA},(I)})$ and $\text{model}^{(II)}(\mathbf{a}^0, \hat{\theta}_\nu^{\text{GA},(II)})$.

Betting Policy	In-Sample	Out-of-Sample	
	ROI %	$\mu_{ROI\%}$	$\sigma_{ROI\%}$
Purely Random	-0.2202	-0.6909	4.0185
$TC_{k-1} > 0$	-5.8488	-0.0012	5.0031
$TC_{k-1} > 1$	-4.4912	0.0511	5.5327
$TC_{k-1} > 2$	-1.8258	-0.0233	6.0224
$TC_{k-1} > 3$	-5.9491	-0.2950	5.7936

Table 11: ROI in-sample and out-of-sample performance of betting policies whose bet sizes are linearly proportional to TC_{k-1} ($bet_k = \frac{TC_{k-1}}{3} \cdot \mathbb{I}(TC_{k-1} > x)$ for $x = 0, 1, 2, 3$).

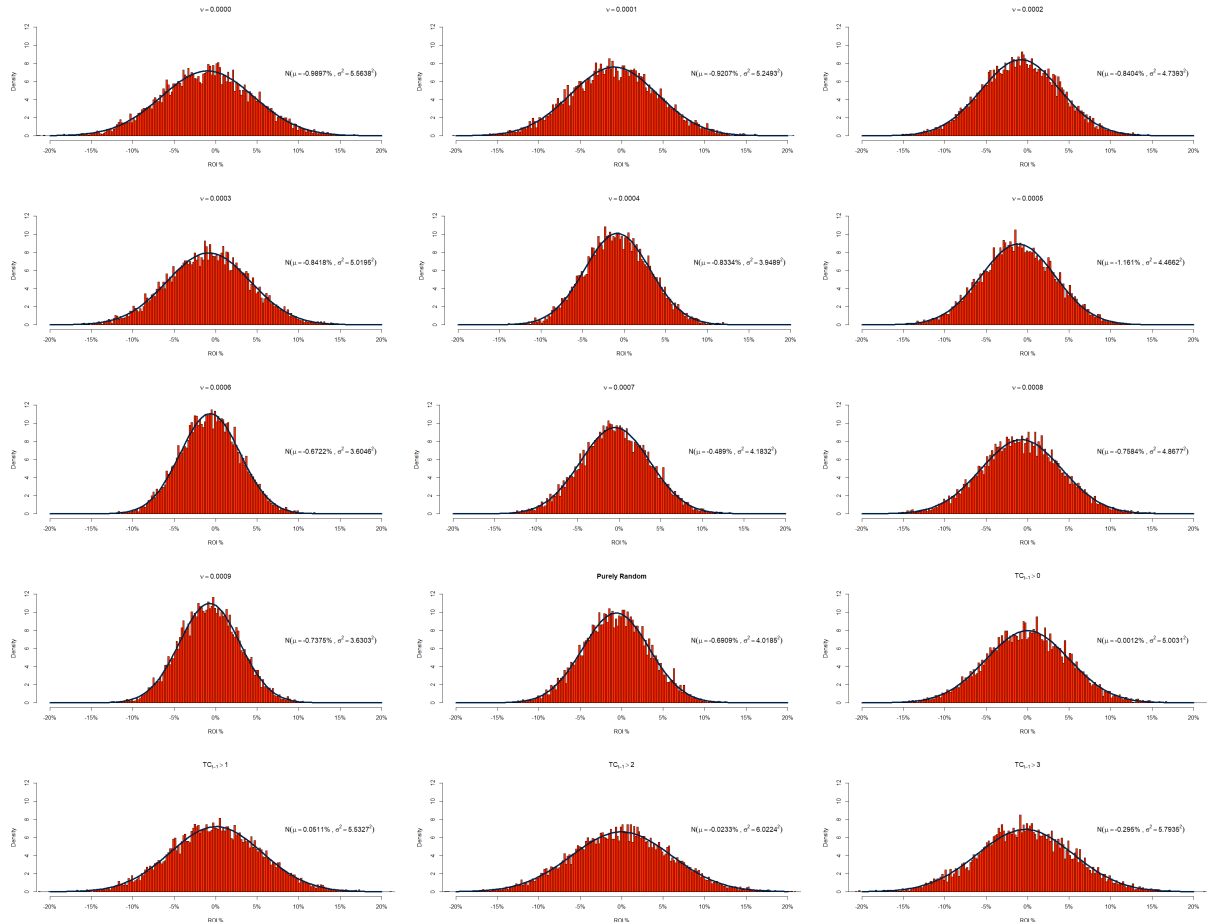
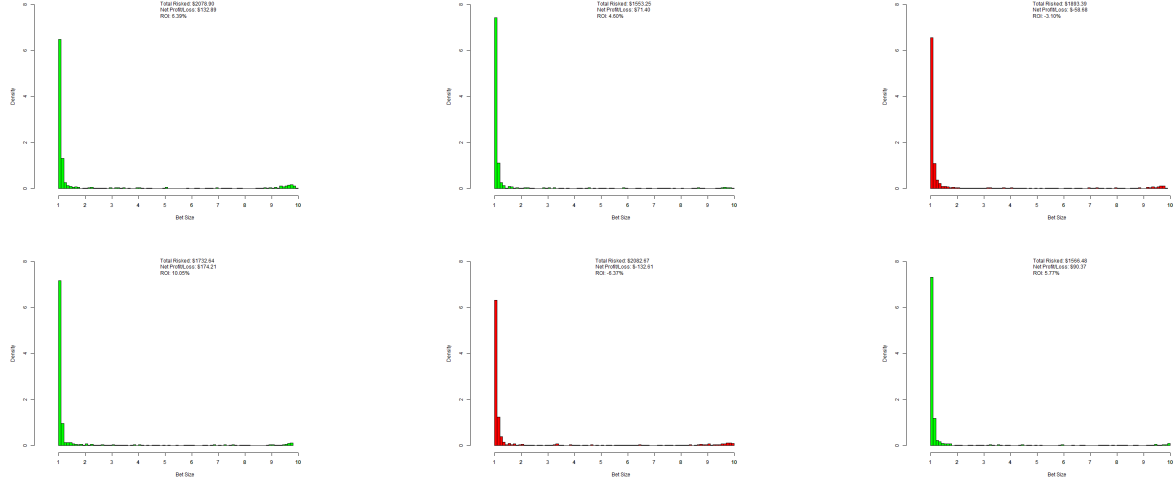


Figure 21: Out-of-sample ROI distributions for varying ν using $\text{model}^{(II)}(\mathbf{a}^0, \hat{\theta}_\nu^{\text{GA},(II)})$, corresponding to Table 10 and ROI distributions of common blackjack betting policies, corresponding to Table 11.

Now, Figure 22 illustrates the distributions of bet sizes, $\tilde{bet}_k \in [1, 10]$, throughout a night using $\text{model}^{(II)}(\mathbf{a}^0, \hat{\theta}_\nu^{\text{GA},(II)})$, for 6 nights, where the net profit or loss is computed as the cumulative gain or loss at the end of each night (the 1,000 hands of blackjack played). The optimization process consistently yields solutions that exhibit sparse betting patterns: the player seems to be consistently opted to bet close to the table minimum, given solutions with these specific ν .



$$\nu = 0.000$$

$$\nu = 0.0005$$

$$\nu = 0.0007$$

Figure 22: Distributions of bet size $\tilde{bet}_k$ over 6 nights for various ν .

7 Hybrid Algorithms

Within the two-block MCMC framework described in Section 3.1.1, the incorporation of pseudo-likelihoods introduces a sharpness parameter $\beta \in \mathbb{R}^+$ that governs the peakedness of the pseudo-likelihood. Consequently, this parameter also influences the shape of the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$, as increased values of β lead the pseudo-likelihood to dominate the conditional, resulting in a more sharply peaked distribution (an effect discussed in greater detail in Section 6.1.5.1.2). An increase in β renders the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$ more sensitive to variations in the objective function, thereby producing a sharply peaked distribution concentrated around a dominant mode. In other words, the heightened peakedness intensifies the concentration of samples in the vicinity of this dominant mode.

Now, to leverage this property, it is proposed to replace the MH sampling step in Section 3.1.1, which samples $\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D}$, with an iterative optimization method. Given the sharply peaked nature of the conditional posterior, induced by a large β , the latter stages of an optimization method converge to the mode of $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$, given the optimization method explicitly seeks $\operatorname{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$. These iterates can be regarded as samples from the approximate modal region of the conditional posterior, particularly when the posterior is highly concentrated around a dominant mode.

Additionally, Section 6.1.7 and Section 6.2.6 highlighted that variations in likelihood sharpness, the initial dispersion σ_{init}^2 , and the particular pseudo-likelihood formulation employed during MCMC, substantially influenced the dispersion parameter σ_{θ}^2 , and by extension, the degree of regularization inferred (given that $\sigma_{\theta}^2 \propto \frac{1}{\nu}$). This raised a critical concern: does the hierarchical Bayesian structure (implemented via two-block MCMC) truly enable data-driven regularization, or does it instead reintroduce user-specified regularization? After all, the user must still choose the pseudo-likelihood formulation (and its associated sharpness parameter β), as well as the initial dispersion σ_{init}^2 . In this light, it becomes necessary to reconsider the idea that the training data determines a meaningful level of regularization embedded within the MAP estimates.

Considering the iterative optimization procedure is to be employed rather than MH sampling in Block 1 of the two-block MCMC framework, we are inclined to reinterpret the sampling of $\sigma_{\theta}^2 \mid \boldsymbol{\theta}, \mathcal{D}$ in Block 2 as an auxiliary mechanism for enhancing exploration within the optimization process. Specifically, since the objective function being maximized is the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$, the act of sampling $\sigma_{\theta}^2 \mid \boldsymbol{\theta}, \mathcal{D}$ at each iteration induces a fluctuating optimization landscape. This variability results in a “wobbly” optimization trajectory, wherein the shape of the objective function changes slightly from one iteration to the next. Such non-static behaviour naturally encourages broader exploration of the parameter space before the algorithm converges toward a dominant mode of the conditional distribution. Furthermore, it must be noted that the sampling of $\sigma_{\theta}^2 \mid \boldsymbol{\theta}, \mathcal{D}$ per iteration continues to allow the training data to influence the inferred degree of regularization applied to the MAP estimates, although, as before, this influence remains substantially shaped by the user.

7.1 Genetic algorithm hybrid

Under the framework of a GA, where our fitness scores reflect that of maximizing the conditional $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$, we can view each generation as a pursuit to refine $\boldsymbol{\theta} \in \mathbb{R}^S$ such that it closely reflects $\operatorname{argmax}_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$, that is, the mode of $p(\boldsymbol{\theta} \mid \mathcal{D}, \sigma_{\theta}^2)$. Furthermore, we may propose the notion that a GA can be

viewed as a means of drawing θ from an approximate modal region of $p(\theta \mid \sigma_\theta^2, \mathcal{D})$, where each n^{th} individual per m^{th} generation, denoted as $\theta^{(n,m)}$, may be viewed as a draw.

Now the fitness values for $\theta^{(n,m)}$:

$$\begin{aligned} f_{n,m} &= \max \left\{ p \left(\theta^{(n,m-1)} \mid \sigma_{\theta^{(n,m-1)}}^2, \mathcal{D} \right) \right\} \\ &= \max \left\{ p \left(\mathcal{D} \mid \theta^{(n,m-1)} \right) p \left(\theta^{(n,m-1)} \mid \sigma_{\theta^{(n,m-1)}}^2 \right) \right\} \\ &= \max \left\{ p \left(\mathcal{D} \mid \theta^{(n,m-1)} \right) \cdot \frac{1}{\sqrt{(2\pi\sigma_{\theta^{(n,m-1)}}^2)^S}} \exp \left(-\frac{1}{2\sigma_{\theta^{(n,m-1)}}^2} \|\theta^{(n,m-1)}\|^2 \right) \right\} \\ &= \max \left\{ \log \left(p \left(\mathcal{D} \mid \theta^{(n,m-1)} \right) \right) - \frac{1}{2\sigma_{\theta^{(n,m-1)}}^2} \|\theta^{(n,m-1)}\|^2 - \frac{S}{2} \log (2\pi\sigma_{\theta^{(n,m-1)}}^2) \right\}. \end{aligned}$$

Afterwhich we conduct the sampling step in Block 2 in Section 3.1.1, where $(\sigma_\theta^2)^{(n,m)} \mid \theta^{(n,m)}, \mathcal{D} \sim \text{Inv-Gamma}(a + \frac{S}{2}, b + \frac{\|\theta^{(n,m)}\|^2}{2})$.

7.2 Gradient descent hybrid for classification

Furthermore, for non-arbitrary objectives, that is for problems whose cost functions can be directly related to input-output pairs (for example, in supervised learning), which give rise to data-driven likelihoods (and not pseudo-likelihoods as we've been employing in this study), we may employ gradient-based optimization methods in order to maximize the conditional $p(\theta \mid \sigma_\theta^2, \mathcal{D})$. We propose that we may adopt the same approach as before, by postulating that each iteration of gradient descent (GD) may be viewed as sampling from the approximate modal region of the conditional $p(\theta \mid \sigma_\theta^2, \mathcal{D})$. Hence, the m^{th} iteration of θ for $m = 1, 2, \dots, M$ with step-size h is updated as:

$$\theta^{(m)} = \theta^{(m-1)} - h \cdot \nabla \text{Obj} \left(\theta^{(m-1)}, (\sigma_\theta^2)^{(m-1)} \right). \quad (18)$$

As gradient descent seeks to minimize an objective function (Obj), we may express our optimization problem, $\text{argmax}_\theta p(\theta \mid \sigma_\theta^2, \mathcal{D})$, as:

$$\text{argmin}_\theta \text{Obj}(\theta) = \text{argmin}_\theta \left\{ -\log [p(\theta \mid \sigma_\theta^2, \mathcal{D})] \right\}.$$

Now,

$$\begin{aligned} -\log [p(\theta \mid \sigma_\theta^2, \mathcal{D})] &\propto -\log [p(\mathcal{D} \mid \theta)] - \log [p(\theta \mid \sigma_\theta^2)] \\ &\propto -\log [p(\mathcal{D} \mid \theta)] + \frac{1}{2\sigma_\theta^2} \|\theta\|^2 + \frac{S}{2} \log (2\pi\sigma_\theta^2). \end{aligned} \quad (19)$$

In the context of K -class classification, we know a neural network outputs logits $z_{i,1}, z_{i,2}, \dots, z_{i,d_L}$ for observations $i = 1, 2, \dots, N$ and $k = 1, 2, \dots, d_L = K$ classes. These are transformed into probabilities using a softmax activation function. Hence, for a given observation i , the predicted probability of class k is:

$$p_{i,k} = \sigma_L(z_{i,k}) = a_k(i)^L = \frac{\exp(z_{i,k})}{\sum_{j=1}^{d_L} \exp(z_{i,j})}.$$

So for a one-hot-encoded y_i , $p(y_i) = \prod_{k=1}^{d_L} (a_k(i)^L)^{y_{i,k}}$. Now for N independent observations, the likelihood is:

$$p(\mathcal{D} | \boldsymbol{\theta}) = \prod_{i=1}^N \prod_{k=1}^{d_L} (a_k(i)^L)^{y_{i,k}}. \quad (20)$$

Being such, we substitute this likelihood into Equation 19 to obtain our cost function:

$$\begin{aligned} \mathcal{L}(\boldsymbol{\theta}, \sigma_\theta^2) &= -\log [p(\boldsymbol{\theta} | \sigma_\theta^2, \mathcal{D})] \propto \underbrace{\left(-\sum_{i=1}^N \sum_{k=1}^{d_L} y_{i,k} \log (a_k(i)^L) + \frac{1}{2\sigma_\theta^2} \|\boldsymbol{\theta}\|^2 \right)}_{\mathcal{L}_{\text{cross-entropy}}(\boldsymbol{\theta}) + \text{L2 penalty}} + \frac{S}{2} \log (2\pi\sigma_\theta^2) \\ &= \mathcal{L}_{\text{cross-entropy}}^*(\boldsymbol{\theta}, \sigma_\theta^2) + \frac{S}{2} \log (2\pi\sigma_\theta^2). \end{aligned} \quad (21)$$

Gradient descent proceeds via backpropagation, with the gradient of our cost function satisfying $\nabla \mathcal{L}(\boldsymbol{\theta}, \sigma_\theta^2) = \nabla \mathcal{L}_{\text{cross-entropy}}^*(\boldsymbol{\theta}, \sigma_\theta^2)$ since the additional term in Equation 21 does not depend on $\boldsymbol{\theta}$ seeing as $\nabla = \frac{\partial(\cdot)}{\partial \boldsymbol{\theta}}$. Now after sampling $\boldsymbol{\theta}^{(m)}$ from Equation 18, we sample our dispersion parameter as $(\sigma_\theta^2)^{(m)} | \boldsymbol{\theta}^{(m)}, \mathcal{D} \sim \text{Inv-Gamma} \left(a + \frac{S}{2}, b + \frac{\|\boldsymbol{\theta}^{(m)}\|^2}{2} \right)$ as before.

Now under the framework of GD, for a fixed σ_θ^2 , our cost function (Equation 21) is static, and GD follows a smooth, deterministic trajectory toward a minimum. By sampling σ_θ^2 per iteration, we create a “wobbly” optimization path: the gradient direction shifts not just due to the current $\boldsymbol{\theta}$, but also because the regularization term’s influence (that is, the influence of σ_θ^2) fluctuates. With fixed σ_θ^2 , GD descends a static trough (a single, well-defined valley) to its lowest point. Varying σ_θ^2 reshapes this trough each iteration: large σ_θ^2 (low ν) widens and shallows it, letting the algorithm wander; small σ_θ^2 (high ν) narrows and deepens it near the origin, tugging inward. The descent becomes a pursuit of a shifting bottom, possibly broadening the exploration before homing in.

7.3 A toy example: 3-class classification

We apply the three previously described optimization methods: namely, two-block MCMC, GD Hybrid and GA Hybrid, to the 3-class classification problem introduced in Appendix G, using the likelihood formulation given in Equation 20. To ensure comparability across methods, we initialize all algorithms with the same starting solution, denoted as $\boldsymbol{\theta}^{(1)}$. For the GA Hybrid approach, every individual in the initial population is set to this same value, that is $\boldsymbol{\theta}^{(n,1)} = \boldsymbol{\theta}^{(1)}$ for all n individuals. Each method, however, comes with specific caveats that influence its behavior. For the GA Hybrid method, several hyperparameters significantly affect the optimization dynamics. In particular, the choice of lower and upper bounds constraining the search space directly impacts the magnitude of $\|\boldsymbol{\theta}\|^2$. Additionally, the mutation probability applied to a particular gene in a parent solution governs the extent of variability of $\|\boldsymbol{\theta}\|^2$. With respect to two-block MCMC, the effect of the initial dispersion parameter σ_{init}^2 on performance has already been examined in Section 6.2.6. For the GD Hybrid approach, variation in the step size h primarily influences the convergence rate.

Figure 23 illustrates that the different optimization methods give rise to varying concentrations in the marginal distributions of the dispersion parameter σ_θ^2 , which we attribute to differences in the degree of regularization implicitly induced by each method. This variation stems from the distinct convergence

behaviors of $\|\boldsymbol{\theta}^{(j)}\|^2$ across iterations. Recall that $\sigma_\theta^2 \mid \boldsymbol{\theta}, \mathcal{D} \sim \text{Inv-Gamma}\left(a + \frac{S}{2}, b + \frac{\|\boldsymbol{\theta}\|^2}{2}\right)$, with $a, b \approx 0$. Hence it follows that if $\|\boldsymbol{\theta}^{(j)}\|^2$ fluctuates around a constant value c , then the marginal distribution of $\sigma_\theta^2 \mid \mathcal{D}$ should also follow an inverse-gamma distribution with approximately constant shape and scale parameters as such: $\sigma_\theta^2 \mid \mathcal{D} \sim \text{Inv-Gamma}\left(a + \frac{S}{2}, b + \frac{c}{2}\right)$. Notably, the GD Hybrid method exhibits the least variability in $\|\boldsymbol{\theta}^{(j)}\|^2$, followed by MCMC, both of which result in marginal distributions of $\sigma_\theta^2 \mid \mathcal{D}$ that closely follow an inverse-gamma form. Among the optimization methods considered, the GA Hybrid approach is the only one that results in a clearly non-inverse-gamma marginal of $\sigma_\theta^2 \mid \mathcal{D}$, which we attribute to the greater variability in $\|\boldsymbol{\theta}^{(j)}\|^2$ across iterations. This variation suggests that the GA explores more diverse regions of the parameter space, which in turn leads to a more dispersed marginal of $\sigma_\theta^2 \mid \mathcal{D}$ (as noted earlier, this variability can be modulated through the mutation probability).

Figure 24 presents the distribution of one of the estimated parameters, specifically $\hat{\theta}_2$. The top panel displays the results on a common scale to highlight the extent to which the different optimization methods produce distinct solutions, $\hat{\boldsymbol{\theta}}$, allowing for a direct comparison between them. In contrast, the bottom panel uses individual scales to better visualize the shape of each distribution of $\hat{\theta}_2$. This latter view illustrates that both hybrid methods can function as approximate sampling techniques, all yielding unimodal distributions for the estimated parameter, $\hat{\theta}_2$ (albeit with differing spreads). While MCMC serves as the baseline for comparison, since it directly samples from the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_\theta^2, \mathcal{D})$ rather than sampling from the approximate modal region of the posterior, multiple runs of the sampler often yield noticeably different parameter solutions. This behavior is also observed in the other two optimization methods as well. Nevertheless, these disparate solutions tend to produce comparable in-sample performance, suggesting the existence of a multi-modal posterior landscape. Accordingly, one cannot conclude that the methods are fundamentally dissimilar solely on the basis of differences in the estimated solutions $\hat{\boldsymbol{\theta}}$, as variation in solutions arises even within a single optimization method across multiple runs.

Finally, Figure 25 in conjunction with Table 12, illustrates the performance of the three optimization methods. The response curves in Figure 25 reveal substantial similarity across methods, with each successfully delineating distinct classification regions for the three particle types. Moreover, Table 12 reports comparable in- and out-of-sample performance across the three approaches, further supporting the notion that all methods achieve a similar level of predictive accuracy.

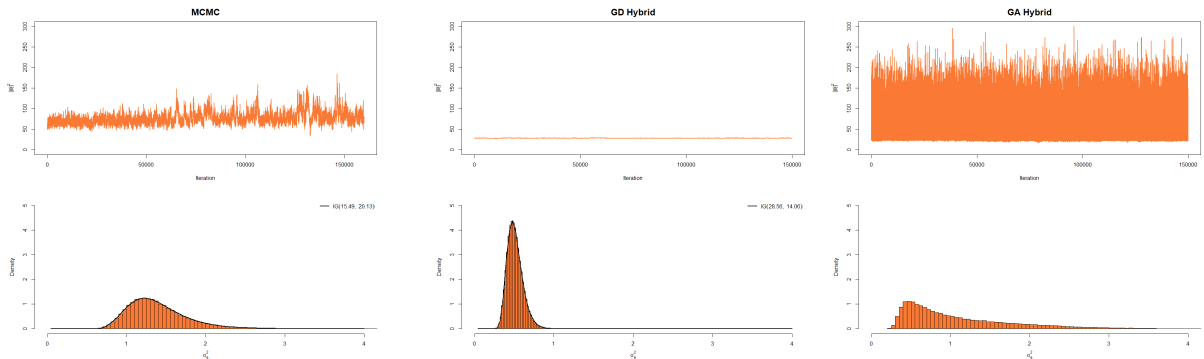


Figure 23: $\|\boldsymbol{\theta}^{(j)}\|^2$ for $j = 1, \dots, 150,000$ (post burn-in) with distribution of marginal of $\sigma_\theta^2 \mid \mathcal{D}$.

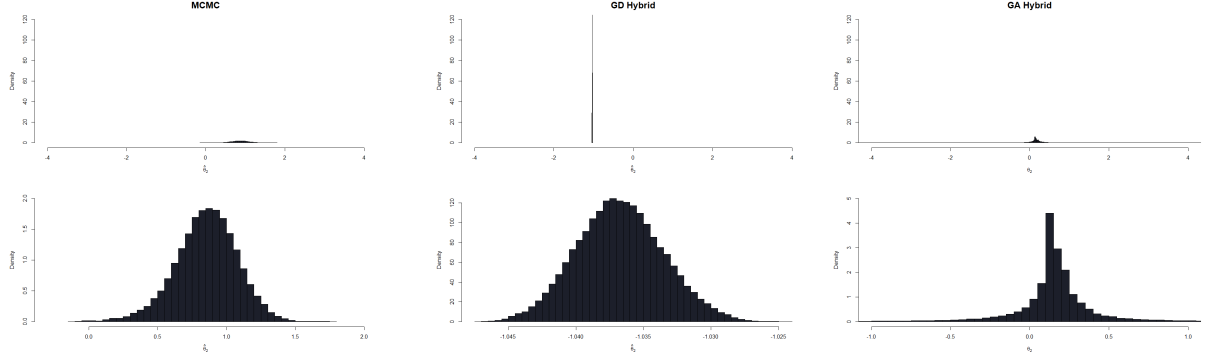


Figure 24: Distribution of $\hat{\theta}_2$. The top panel uses a common scale across all three optimization methods to facilitate direct comparison, while the bottom panel employs different scales across methods.

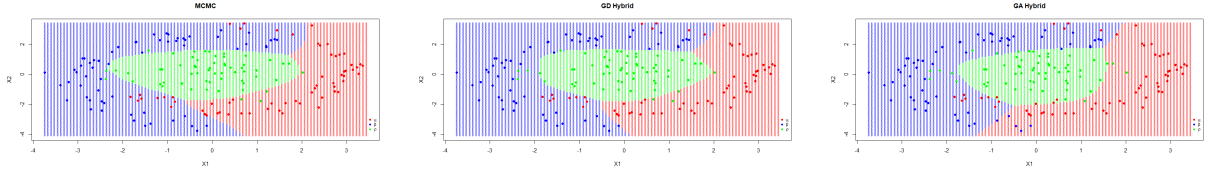


Figure 25: Response curves for different optimization methods, represented as a cross-sectional heat map of particle classification regions (circles represent the out-of-sample particles).

Optimization Method	In-sample	Out-of-sample
MCMC	86.67	86.67
GD Hybrid	85.55	87.22
GA Hybrid	82.78	81.11

Table 12: Accuracy for in-sample and out-of-sample sets using two-block MCMC and two hybrid optimization techniques.

It is important to recall that such hybrid optimization methods are comparable to that of two-block MCMC, only in settings where the conditional posterior $p(\boldsymbol{\theta} \mid \sigma_{\theta}^2, \mathcal{D})$ is highly peaked. The similarity in results across the three methods in this case may therefore indicate that, for the data under consideration, the conditional posterior is indeed sharply concentrated. However, this observation is likely data-specific and should not be taken as evidence that such comparability will generalize across all datasets. In the following section, we intentionally increase the sharpness of the likelihood function, and by extension the conditional posterior, to investigate the behaviour of the optimization methods under a sharpened conditional.

7.4 Hybrid for Blackjack Problem I, II and III

This section primarily examines the similarities and differences between solutions derived from the two-block MCMC framework (as outlined in Section 3.1.1) and the GA Hybrid method previously introduced, when applied to Blackjack Problems I and II (using **model** for II). In addition, we introduce a third variant, referred to as Blackjack Problem III, in which both the decision-making and bet-sizing parameter sets are embedded into a unified parameter vector: $\boldsymbol{\theta} = \left[\boldsymbol{\theta}^{\text{Bet}}, \boldsymbol{\theta}^{\text{Decision}} \right] \in \mathbb{R}^{S+R}$. In this formulation, $\boldsymbol{\theta}$

contains the weights and biases governing two distinct neural networks: one dedicated to decision-making and the other to bet-sizing (noting we use **model** for the bet-sizing network). The overarching objective for Blackjack Problem III remains unchanged, namely, to maximize the ROI achieved at the conclusion of the training night, such that:

$$\begin{aligned} \operatorname{argmax}_{\boldsymbol{\theta}} \operatorname{Obj}(\boldsymbol{\theta}) &= \operatorname{argmax}_{\boldsymbol{\theta}} \frac{\sum_{k=1}^K \left(s_k(\boldsymbol{\theta}) \cdot \tilde{bet}_k(\boldsymbol{\theta}) \right)}{\sum_{k=1}^K \tilde{bet}_k(\boldsymbol{\theta})} \\ &= \operatorname{argmax}_{\boldsymbol{\theta}} \operatorname{ROI}(\boldsymbol{\theta}). \end{aligned}$$

We employ the exponential likelihood across all three blackjack problems, albeit with differing sharpness parameters, β , selected to yield the most favourable MCMC convergence. For fairness in comparison, the corresponding GA Hybrid method implementations employ identical β values. Accordingly, the pseudo-likelihood is given by $p(\mathcal{D} \mid \boldsymbol{\theta}) = \exp(\beta \cdot \operatorname{ROI}(\boldsymbol{\theta}))$. Now, given that our objective function is arbitrary, we restrict our analysis to the GA Hybrid method described in Section 7.1, and do not employ the GD Hybrid method from Section 7.2, as the objective function in question is non-differentiable with respect to $\boldsymbol{\theta}$.

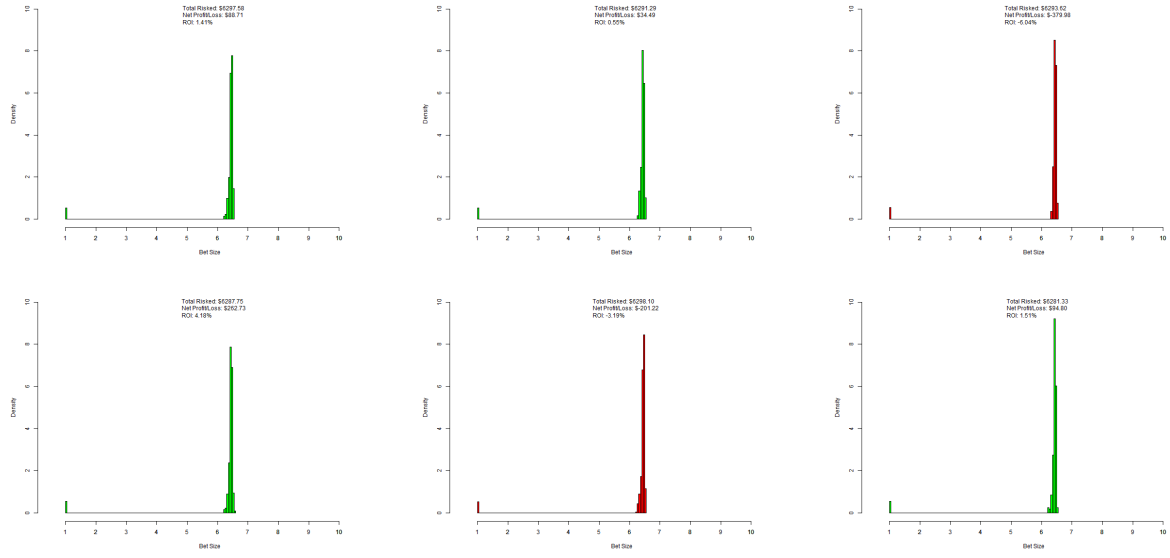
Table 13 presents the results of the two-block MCMC optimization method alongside its corresponding GA Hybrid method for each of the three blackjack problems. In the case of Blackjack Problem I, both methods favour the stand action for all soft and hard totals, and never opt to surrender or split pairs, which explains why we obtain identical in- and out- of sample performance.

For Blackjack Problem II, we observe near-identical in-sample and out-of-sample performance. As illustrated in Figure 26, this similarity appears to stem from both methods producing agents with comparable betting behaviour: specifically, agents that concentrate their bet-size to approximately 6 to 8 units per night.

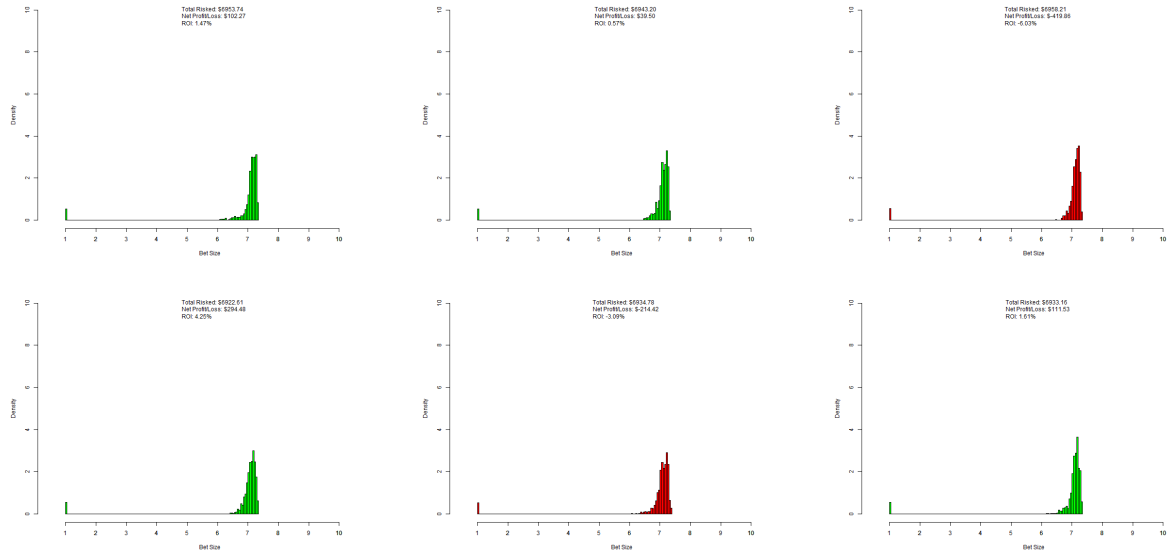
For Blackjack Problem III, two-block MCMC produces an agent that opts to stand on all hard totals but hit on all soft totals, whereas the Hybrid method selects to stand for both hard and soft totals. In both cases, neither method chooses to surrender or split pairs. Additionally, both methods yield betting agents whose bet-sizes are concentrated to approximately 3 to 5 units per night (as illustrated in Figure 27). Despite these differences, both methods produce solutions which achieve comparable in-sample ROI %, suggesting the possibility that multiple combinations of decision-making strategies and bet-sizing schemes can lead to similar in-sample ROI %.

	MCMC			Hybrid		
	In-Sample	Out-of-Sample		In-Sample	Out-of-Sample	
Blackjack Problem	ROI %	$\mu_{ROI\%}$	$\sigma_{ROI\%}$	ROI %	$\mu_{ROI\%}$	$\sigma_{ROI\%}$
I	-14.8500	-16.0162	3.0645	-14.8500	-16.0162	3.0645
II	-3.9011	-0.6765	3.6044	-3.9154	-0.6841	3.6069
III	-11.9441	-15.7276	3.1629	-12.1605	-14.4514	2.8351

Table 13: ROI in-sample and out-of-sample performance for the three blackjack problems using $\beta = 50$ for Problem I, $\beta = 250$ for Problem II and $\beta = 50$ for Problem III.

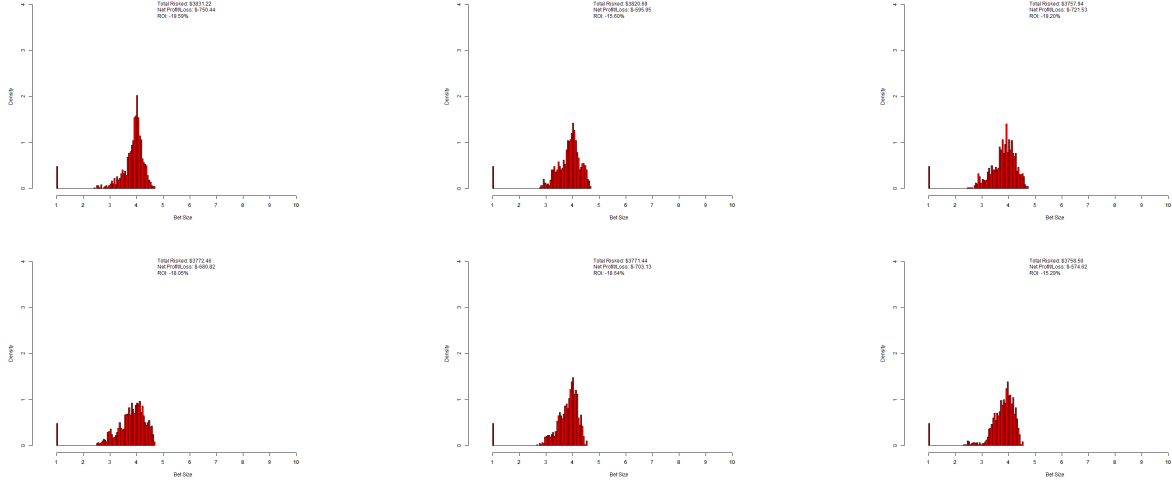


Bet-size ($\tilde{bet}_k(\hat{\theta}^{\text{MCMC}, (\text{II})})$) distributions per night according to solution derived from MCMC.

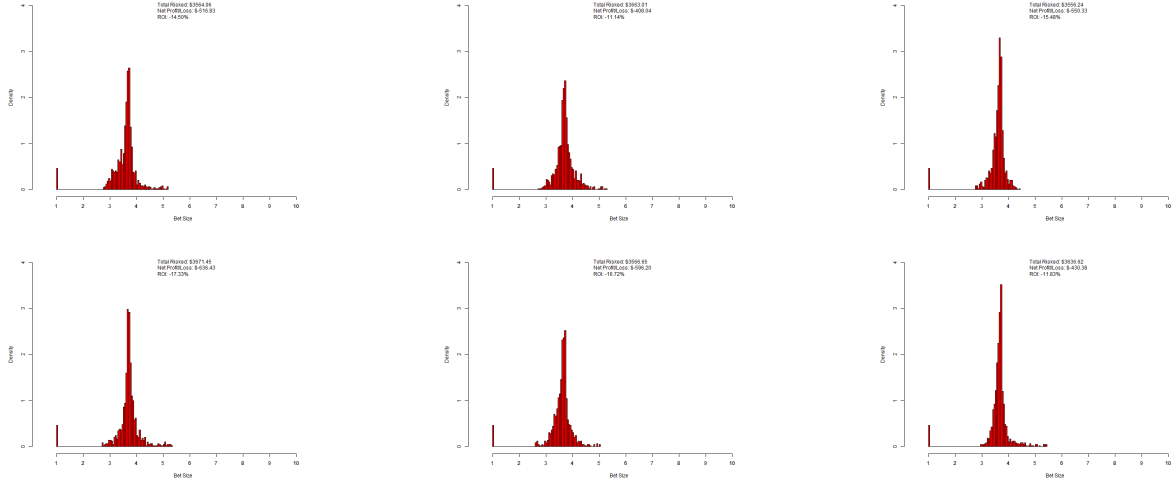


Bet-size ($\tilde{bet}_k(\hat{\theta}^{\text{Hybrid}, (\text{II})})$) distributions per night according to solution derived from GA Hybrid.

Figure 26: Bet-size distributions per night (over 6 nights) for Blackjack Problem II.



Bet-size ($\tilde{bet}_k(\hat{\theta}^{\text{MCMC}})$) distributions per night according to solution derived from MCMC.



Bet-size ($\tilde{bet}_k(\hat{\theta}^{\text{Hybrid}})$) distributions per night according to solution derived from GA Hybrid.

Figure 27: Bet-size distributions per night (over 6 nights) for Blackjack Problem III.

Furthermore, this section illustrates that, when the conditional posterior $p(\theta \mid \sigma_\theta^2, \mathcal{D})$ is sufficiently sharp (achieved via an increased β), both the two-block MCMC framework and the Hybrid method tend to optimize solutions which converge toward the same dominant mode of the conditional posterior (recalling from earlier that increasing β suppresses minor modes and accentuates dominant modes). This is consistent to what was articulated by Kirkpatrick et al. (1983) with regards to SA; where a decreased temperature parameter (inverse to β) concentrates samples around global minima of the cost function. This results in comparable in-sample performance between the aforementioned methods. Such evidence supports the earlier postulate that an iterative optimization procedure could serve as a viable replacement for the MH sampling in Block 1 of the two-block MCMC framework in Section 3.1.1, provided the posterior is sufficiently sharp.

8 Conclusion

The study aimed to illustrate the shortcomings of two-block MCMC, which is often employed to allow the training data to infer a level of regularization by incorporating the sampling of the dispersion parameter, σ_θ^2 , into the algorithm. The study showed that the pseudo-likelihood form, the likelihood sharpness parameter β , and the initial dispersion σ_{init}^2 are in fact user-specified hyperparameters that exert a substantial influence on the degree of regularization inferred. As such, the use of a Bayesian hierarchical model in this context does not genuinely infer regularization from the training data; rather, it is the *user* who determines the effective strength of regularization, albeit with additional steps.

Furthermore, the study demonstrated that if one were to increase likelihood sharpness to an extreme, one may effectively reduce the two-block MCMC to a hybrid approach in which the first block is replaced by an iterative optimization procedure, yielding nearly identical in-sample performance to the original scheme (as solutions from both methods converge toward the same dominant mode of the conditional posterior). In this sense, the sampling of the dispersion parameter at each iteration functions primarily as a mechanism to add exploration to the search process, rather than as a means to infer regularization.

Taken together, the results of this study directly address a gap implicit in the existing literature on MCMC-based optimization. While prior work has cautioned that applying MCMC to arbitrary objectives blurs the distinction between posterior inference and stochastic optimization, these concerns have largely remained conceptual. By explicitly examining two-block MCMC in settings where the likelihood is deliberately constructed to be proportional to the arbitrary objective function, this study provides a concrete empirical demonstration of those concerns. In particular, it shows that claims of “data-driven” regularization in such frameworks are fragile, and that the effective strength of regularization is instead governed by user-specified choices, most notably likelihood sharpness and initial dispersion. In doing so, the study clarifies that two-block MCMC in arbitrary objective settings should generally be interpreted as a form of stochastic, temperature-controlled optimization rather than Bayesian posterior inference, thereby making explicit what has previously been only implicitly acknowledged in the literature.

Future work should explore treating the sharpness parameter in analogy to SA, whereby β is increased according to a cooling schedule within the two-block MCMC framework, rather than being fixed *a priori*. As expounded upon Section 2.4, parallel tempering, introduced by Geyer (1991), operates by running multiple Markov chains in parallel at different temperatures, each targeting a tempered version of the objective-induced distribution. While this may appear to offer a natural extension of two-block MCMC, this study’s results indicate that running a Markov Chain at different temperatures fundamentally alters the degree of regularization inferred by the sampler. Consequently, the direct application of tempering-based strategies to two-block MCMC is unlikely to yield meaningful results.

A worthwhile avenue for future research, however, would be to explore simulated tempering, introduced by Marinari and Parisi (1992) (see Section 2.4 of the current study), which treats the temperature, and hence the sharpness parameter β , as a stochastic variable rather than a fixed tuning parameter. Additionally, and more generally, one could investigate how pre-specified temperature schedules, as formalized in annealed MCMC schemes by Geyer and Thompson (1995) (again, see Section 2.4), influence the behavior of two-block MCMC results. Nonetheless, caution is warranted with these approaches: continually increasing sharpness may substantially degrade the mixing properties of the Markov chain, and

the mechanisms proposed in this study to mitigate such issues may not directly extend to these settings.

References

- Shun-ichi Amari. Backpropagation and stochastic gradient descent method. *Neurocomputing*, 5(4-5): 185–196, 1993.
- Blackjack Apprenticeship. Blackjack strategy charts - how to play perfect blackjack. <https://www.blackjackapprenticeship.com/blackjack-strategy-charts/>, 2025. Accessed on 13 November 2025.
- Sudipto Banerjee, Bradley Carlin, and Alan Gelfand. *Hierarchical Modeling and Analysis for Spatial Data*. CRC Press, 2014.
- James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(Feb):281–305, 2012.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- Annette J Dobson and Adrian G Barnett. *An introduction to generalized linear models*. Chapman and Hall/CRC, 2018.
- Meire Fortunato et al. Noisy networks for exploration. *Proceedings of the International Conference on Learning Representations*, 2018.
- Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. *Proceedings of the International Conference on Machine Learning*, 2016.
- Andrew Gelman. Prior distributions for variance parameters in hierarchical models. *Bayesian Analysis*, 1(3):515–533, 2006.
- Stuart Geman and Donald Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6(6):721–741, 1984.
- Edward George and Robert McCulloch. Approaches for bayesian variable selection. *Statistica Sinica*, pages 339–373, 1997.
- Charles J. Geyer. Markov chain monte carlo maximum likelihood. *Computing Science and Statistics*, 23: 156–163, 1991.
- Charles J. Geyer and Elizabeth A. Thompson. Annealing markov chain monte carlo with applications to ancestral inference. *Journal of the American Statistical Association*, 90(431):909–920, 1995.
- Bai-Ni Guo, Feng Qi, Jiao-Lian Zhao, and Qiu-Ming Luo. Sharp inequalities for polygamma functions. *Mathematica Slovaca*, 65(1):103–120, 2015.
- Heikki Haario, Eero Saksman, and Johanna Tamminen. An adaptive metropolis algorithm. *Bernoulli*, 7(2):223–242, 2001. doi: 10.2307/3318737.
- Nick Hansen, Hao Zhou, Maruan Al-Shedivat, Jacob Walker, Chelsea Finn, and Sergey Levine. Towards generalization in reinforcement learning: A survey. *arXiv preprint arXiv:2111.09794*, 2021.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009. ISBN 978-0-387-84857-0. URL <https://web.stanford.edu/~hastie/ElemStatLearn/>.

- W. Keith Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. doi: 10.1093/biomet/57.1.97.
- Peter Henderson et al. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, 1975.
- Jari Kaipio and Erkki Somersalo. *Statistical and Computational Inverse Problems*. Springer, 2005.
- S. Kirkpatrick, C.D. Gelatt, and M.P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598): 671–680, 1983. doi: 10.1126/science.220.4598.671.
- Enzo Marinari and Giorgio Parisi. Simulated tempering: a new monte carlo scheme. *Europhysics Letters*, 19(6):451–458, 1992.
- Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The journal of chemical physics*, 21(6):1087–1092, 1953.
- Volodymyr Mnih et al. Asynchronous methods for deep reinforcement learning. *Proceedings of the International Conference on Machine Learning*, 2016.
- Radford M. Neal. Annealed importance sampling. *Statistics and Computing*, 11(2):125–139, 2001. doi: 10.1023/A:1008923215028.
- Michael Painter and Emma Brunskill. Cs234 notes - lecture 1: Introduction to reinforcement learning, 2018. URL https://web.stanford.edu/class/archive/cs/cs234/cs234.1224/CS234Win2019/slides/1notes_intro.pdf. Accessed: 2025-02-10.
- Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. *Proceedings of the IEEE International Conference on Robotics and Automation*, 2018.
- Etienne Pienaar. Evolutionary learning and arbitrary objectives, October 2024. URL [Unpublishedlecturematerial](#). Lecture slides.
- Christian Robert and George Casella. *The Bayesian Choice*. Springer, 2007.
- Christian P. Robert and George Casella. *Monte Carlo Statistical Methods*. Springer, New York, 2 edition, 2004.
- Gareth O Roberts and Jeffrey S Rosenthal. Coupling and ergodicity of adaptive markov chain monte carlo algorithms. *Journal of Applied Probability*, 44(2):458–475, 2007.
- Gareth O Roberts, Andrew Gelman, and Walter R Gilks. Weak convergence and optimal scaling of random walk metropolis algorithms. *The Annals of Applied Probability*, 7(1):110–120, 1997.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897. PMLR, 2015.

- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, Cambridge, 2014.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- Joshua Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017.
- Vladimir Vapnik. Principles of risk minimization for learning theory. *Advances in neural information processing systems*, 4, 1991.
- Vladimir N. Vapnik. *Statistical Learning Theory*. Wiley, New York, 1998.
- Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of mcmc. *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.
- Ronald J. Williams. Function optimization using connectionist reinforcement learning algorithms. *Neural Networks*, 4(2):229–256, 1991.

Appendix A Joint Metropolis-Hastings

We define $\mathbf{\Lambda} = [\boldsymbol{\theta}', \sigma_\theta^2]' \in \mathbb{R}^{S+1}$ and re-derive our MH algorithm to include σ_θ^2 . Hence, given the current state of $\mathbf{\Lambda}$, that is $\mathbf{\Lambda}^{(j)}$, the MH algorithm proposes a new value $\mathbf{\Lambda}^*$ obtained from $\mathbf{\Lambda}^* = \mathbf{\Lambda}^{(j)} + \mathbf{Q}$. Subsequently, $\mathbf{\Lambda}^*$ is accepted as the new value in the Markov chain under the following acceptance criterion:

$$\mathbf{\Lambda}^{(j+1)} = \begin{cases} \mathbf{\Lambda}^*, & \text{if } U < \alpha, \\ \mathbf{\Lambda}^{(j)}, & \text{otherwise.} \end{cases}$$

Where the vector $\mathbf{Q} = [\mathbf{Q}_\theta', Q_{\sigma_\theta^2}]' \in \mathbb{R}^{S+1}$ denotes drawn values from proposal densities: $\mathbf{Q}_\theta \sim \mathcal{N}(\mathbf{0}, \sigma_{\mathbf{Q}_\theta}^2 \mathbf{I}_S)$ and $Q_{\sigma_\theta^2} \sim \text{Inv-Gamma}(a_Q, b_Q)$. Now, α is given by:

$$\begin{aligned} \alpha &= \min \left(\frac{p(\mathbf{\Lambda}^*|\mathcal{D})}{p(\mathbf{\Lambda}^{(j)}|\mathcal{D})} \cdot \frac{Q(\mathbf{\Lambda}^{(j)}|\mathbf{\Lambda}^*)}{Q(\mathbf{\Lambda}^*|\mathbf{\Lambda}^{(j)})}, 1 \right) \\ &= \min \left(\frac{p(\mathcal{D}|\mathbf{\Lambda}^*)p(\mathbf{\Lambda}^*)}{p(\mathcal{D}|\mathbf{\Lambda}^{(j)})p(\mathbf{\Lambda}^{(j)})} \cdot \frac{Q(\boldsymbol{\theta}^{(j)}, \sigma_{\theta^{(j)}}^2|\boldsymbol{\theta}^*, \sigma_{\theta^*}^2)}{Q(\boldsymbol{\theta}^*, \sigma_{\theta^*}^2|\boldsymbol{\theta}^{(j)}, \sigma_{\theta^{(j)}}^2)}, 1 \right) \\ &= \min \left(\frac{p(\mathcal{D}|\boldsymbol{\theta}^*)p(\boldsymbol{\theta}^*, \sigma_{\theta^*}^2)}{p(\mathcal{D}|\boldsymbol{\theta}^{(j)})p(\boldsymbol{\theta}^{(j)}, \sigma_{\theta^{(j)}}^2)} \cdot \frac{Q_\theta(\boldsymbol{\theta}^{(j)}|\boldsymbol{\theta}^*)Q_{\sigma_\theta^2}(\sigma_{\theta^{(j)}}^2|\sigma_{\theta^*}^2)}{Q_\theta(\boldsymbol{\theta}^*|\boldsymbol{\theta}^{(j)})Q_{\sigma_\theta^2}(\sigma_{\theta^*}^2|\sigma_{\theta^{(j)}}^2)}, 1 \right) && \text{Likelihood not dependent on } \sigma_\theta^2 \\ &= \min \left(\frac{p(\mathcal{D}|\boldsymbol{\theta}^*)p(\boldsymbol{\theta}^*|\sigma_{\theta^*}^2)p(\sigma_{\theta^*}^2)}{p(\mathcal{D}|\boldsymbol{\theta}^{(j)})p(\boldsymbol{\theta}^{(j)}|\sigma_{\theta^{(j)}}^2)p(\sigma_{\theta^{(j)}}^2)} \cdot \frac{Q_{\sigma_\theta^2}(\sigma_{\theta^{(j)}}^2|\sigma_{\theta^*}^2)}{Q_{\sigma_\theta^2}(\sigma_{\theta^*}^2|\sigma_{\theta^{(j)}}^2)}, 1 \right). && \text{Symmetry of } Q_\theta \end{aligned} \tag{22}$$

Assuming $\sigma_\theta^2 \sim \text{Inv-Gamma}(a, b)$ and still assuming $\boldsymbol{\theta}|\sigma_\theta^2 \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \mathbf{I}_S)$, Equation 22 simplifies to:

$$\alpha = \min \left(\frac{h(k(\boldsymbol{\theta}^*)) \cdot \frac{1}{\sqrt{(2\pi\sigma_{\theta^*}^2)^S}} \exp\left(-\frac{1}{2\sigma_{\theta^*}^2} \|\boldsymbol{\theta}^*\|^2\right) \cdot \frac{1}{(\sigma_{\theta^*}^2)^{a+1}} \exp\left(-\frac{b}{\sigma_{\theta^*}^2}\right) \cdot \frac{1}{(\sigma_{\theta^{(j)}}^2)^{a_Q+1}} \exp\left(-\frac{b_Q}{\sigma_{\theta^{(j)}}^2}\right)}{h(k(\boldsymbol{\theta}^{(j)})) \cdot \frac{1}{\sqrt{(2\pi\sigma_{\theta^{(j)}}^2)^S}} \exp\left(-\frac{1}{2\sigma_{\theta^{(j)}}^2} \|\boldsymbol{\theta}^{(j)}\|^2\right) \cdot \frac{1}{(\sigma_{\theta^{(j)}}^2)^{a+1}} \exp\left(-\frac{b}{\sigma_{\theta^{(j)}}^2}\right) \cdot \frac{1}{(\sigma_{\theta^*}^2)^{a_Q+1}} \exp\left(-\frac{b_Q}{\sigma_{\theta^*}^2}\right)}, 1 \right),$$

and by taking the log:

$$\begin{aligned} \log(\alpha) &= \min \left(\log(h(k(\boldsymbol{\theta}^*))) - \frac{S}{2} \log(2\pi\sigma_{\theta^*}^2) - \frac{\|\boldsymbol{\theta}^*\|^2}{2\sigma_{\theta^*}^2} - (a+1) \log(\sigma_{\theta^*}^2) - \frac{b}{\sigma_{\theta^*}^2} \right. \\ &\quad \left. - (a_Q+1) \log(\sigma_{\theta^{(j)}}^2) - \frac{b_Q}{\sigma_{\theta^{(j)}}^2} - \log(h(k(\boldsymbol{\theta}^{(j)}))) + \frac{S}{2} \log(2\pi\sigma_{\theta^{(j)}}^2) \right. \\ &\quad \left. + \frac{\|\boldsymbol{\theta}^{(j)}\|^2}{2\sigma_{\theta^{(j)}}^2} + (a+1) \log(\sigma_{\theta^{(j)}}^2) + \frac{b}{\sigma_{\theta^{(j)}}^2} + (a_Q+1) \log(\sigma_{\theta^*}^2) + \frac{b_Q}{\sigma_{\theta^*}^2}, 0 \right). \end{aligned}$$

Appendix B Neural Network Architecture

We define the feed-forward recursive relation in scalar form, hence the j^{th} node on the l^{th} layer for the i^{th} observation is given as:

$$a_j^l(i) = \sigma_l \left(\sum_{k=1}^{d_{l-1}} a_k^{l-1}(i) w_{kj}^l + b_j^l \right),$$

for $l = 1, 2, \dots, L; j = 1, 2, \dots, d_l; i = 1, 2, \dots, N$. Now, $\sigma_l(\cdot)$ denotes the activation function on layer l , d_{l-1} denotes the number of nodes in layer $l-1$, w_{kj}^l denotes the kj^{th} weight linking the k^{th} node in layer $l-1$ and the j^{th} node in layer l with b_j^l denoting the j^{th} bias in layer l . The equation is evaluated subject to the initial conditions $a_j^{(0)} = x_{ij}$ for all j at the i^{th} training example.

Appendix C Genetic Algorithm

Define $\theta^{(n,m)} \in \mathbb{R}^p$ to be the n^{th} solution (individual) from the m^{th} generation such that $n = \{1, \dots, N\}$ and $m = \{1, \dots, M\}$. We present the N -size population as $\Theta^{(m)} = \left[\theta^{(1,m)}, \theta^{(2,m)}, \dots, \theta^{(N,m)} \right]_{p \times N}$ for the m^{th} generation. We initialize by setting $\Theta^{(0)}$ where $\theta^{(n,0)} \sim \mathcal{U}_p$ for $n = \{1, \dots, N\}$. At the termination of the algorithm, we return the solution $\theta^{(n^*, m^*)}$ that achieved the highest objective value across all individuals and generations:

$$\theta^{(n^*, m^*)} = \arg \max_{n \in \{1, \dots, N\}, m \in \{1, \dots, M\}} \text{Obj} \left(\theta^{(n,m)} \right).$$

Hence for generation $m = 1, \dots, M$:

Fitness

For $n = 1, \dots, N$, we compute the fitness for each n^{th} individual of the m^{th} generation as $f_{n,m} = \text{Obj} \left(\theta^{(n,m-1)} \right)$.

Selection (Roulette Wheel)

1. Compute selection probabilities:

$$p_{n,m} = \frac{f_{n,m}}{\sum_{i=1}^N f_{i,m}}, \quad n = 1, \dots, N.$$

2. Compute cumulative probabilities:

$$C_{n,m} = \sum_{i=1}^n p_{i,m}, \quad n = 1, \dots, N.$$

3. For each selection $i = 1, \dots, N$:

- (a) Sample $r \sim \mathcal{U}(0, 1)$.
- (b) Find the smallest n such that $C_{n,m} \geq r$.
- (c) Select parent $\theta^{(n,m)} = \tilde{\theta}^{(n,m)}$.

4. To form the mating pool $\mathcal{M}^{(m)} = \left[\tilde{\theta}^{(1,m)}, \tilde{\theta}^{(2,m)}, \dots, \tilde{\theta}^{(N,m)} \right]_{p \times N}$.

Recombination (Blend- α crossover)

Select two parents $\tilde{\theta}^{(i^*, m)}$ and $\tilde{\theta}^{(j^*, m)}$ from the mating pool $\mathcal{M}^{(m)}$, to create offspring (with a fixed α):

1. Repeat until N offspring are created:

For each gene $r = 1, 2, \dots, p$:

i. Sample $u_r \sim \mathcal{U}(0, 1)$.

ii. Compute blend weight:

$$v_r = (1 + 2\alpha)u_r - \alpha.$$

iii. Generate offspring gene:

$$\hat{\theta}_r^{(n,m)} = v_r \cdot \tilde{\theta}_r^{(i^*,m)} + (1 - v_r) \cdot \tilde{\theta}_r^{(j^*,m-1)}.$$

$$\text{to create offspring } \mathcal{O}^{(m)} = \left[\hat{\theta}^{(1,m)}, \hat{\theta}^{(2,m)}, \dots, \hat{\theta}^{(N,m)} \right]_{p \times N}.$$

Mutation (Gaussian)

Randomly perturb a subset of the newly created offspring in $\mathcal{O}^{(m)}$:

1. Sample a subset of indices $\mathcal{K} \subset \{1, \dots, N\}$ uniformly at random.

2. For each $k \in \mathcal{K}$, apply a Gaussian perturbation:

$$\hat{\theta}^{(k,m)} \leftarrow \hat{\theta}^{(k,m)} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}).$$

Replacement (Elitism)

Preserve the best-performing individuals from generation m and newly created offspring:

1. Compute $2N$ fitness values $f_{i,m} = \text{Obj}(\Theta^{(m)}, \mathcal{O}^{(m)})$ for $i = 1, \dots, 2N$.

2. Identify the index set of the top N individuals from $\{\Theta^{(m)}, \mathcal{O}^{(m)}\}$:

$$\mathcal{E} = \{i \in \{1, \dots, 2N\} : f_{i,m} \text{ is among the top } N \text{ values in } \{f_{1,m}, \dots, f_{2N,m}\}\}.$$

3. The $(m+1)^{th}$ generation is defined as $\Theta^{(m+1)} = \left[\theta^{(1,m+1)}, \theta^{(2,m+1)}, \dots, \theta^{(N,m+1)} \right]_{p \times N}$ where every $\theta^{(n,m+1)}$ is such that $n \in \mathcal{E}$.

Appendix D Specifications

A navigation problem

Parameter	Value
R_{inner}	0.25
R_{outer}	1
R_{crash}	0.05
K	250
J	50
T	100
δ	0.01
P_{lower}	$2K$
P_{upper}	$3K$
sf	1
ω_0^{Train}	2024
$\{\omega_j^{\text{Test}}\}_{j=1}^{1000}$	$\{1, 2, \dots, 1000\}$

Table 14: Specifications for the navigation problem.

The blackjack problems

The simulations are conducted under standard S-17 blackjack rules, with an eight-deck shoe used for card dealing. The variant considered allows early surrender and double-after-split (DAS), but does not permit re-splitting or surrender after a split. The Basic Strategy rules used in this study follow the standard charts available in Apprenticeship (2025).

Overall specifications

Parameter	Value
Neural Network	
L (# layers in NN excluding input layer)	3
d_1, d_2 (# nodes in each hidden layer)	3
$\sigma_1(\cdot), \sigma_2(\cdot)$ (activation functions for both hidden layers)	$\tanh(\cdot)$
Metropolis-Hastings	
σ_{Init}^2 (unless otherwise stated)	1
δ	1000
Δ	100
$(s^2)^{(0)}$	1
κ	0.6
GA & RS	
M	1000
N	100
# RS iterations	MN

Table 15: Overall specifications of the algorithms used across all three reinforcement learning problems.

Appendix E Random Search

Define the objective function $\text{Obj}(\theta)$, search space $\theta \in \Theta \subset \mathbb{R}^p$ and number of samples/iterations S .

1. Initialize by setting best score $f^* \leftarrow -\infty$, best parameter $\theta^* \leftarrow \text{null}$.
2. For $s = 1, \dots, S$:

- (a) Sample $\theta^{(s)} \sim \mathcal{U}_p(\Theta)$.
- (b) Evaluate objective:

$$f_s = \text{Obj}(\theta^{(s)}).$$

- (c) If $f_s > f^*$, then update:

$$f^* \leftarrow f_s, \quad \theta^* \leftarrow \theta^{(s)}.$$

3. Our solution is the best found parameter θ^* with corresponding score f^* .

Appendix F The True Count

In card counting (e.g., Hi-Lo system), the running count is the sum of values assigned to seen cards (e.g., +1 for 2-6, 0 for 7-9, -1 for 10-A). The true count adjusts this for the number of decks remaining. We define the running count at time k as $RC_k = \sum_{c \in \mathcal{H}_k} \rho(c)$, where $\rho(c)$ is the count value of card c under the counting system. Assuming a $D_0 = 8$ -deck shoe, the true count is defined as $TC_k = \frac{RC_k}{52D_0 - |\mathcal{H}_k|}$. A high true count implies a greater proportion of high cards (10s and Aces) remaining in the shoe, which statistically favours the player by increasing the likelihood of blackjack or beating the dealer, and improving the effectiveness of doubling and splitting actions.

Appendix G 3-Class Classification Particle Data

Table 16 provides an overview of the variables used in the particle classification task. The dataset consists of a two-dimensional spatial cross-section, represented by x and y coordinates, of three distinct sub-atomic particle types, with a total of 360 observations. The dataset is available in the project files, and the corresponding training and test set allocations are defined within the accompanying codebase on [GitHub](#).

Variable	Description
$X1$	First coordinate in cross-section
$Y2$	Second coordinate in cross-section
$Yi1$	Response: 1 if code- α , 0 otherwise.
$Yi2$	Response: 1 if code- β , 0 otherwise.
$Yi3$	Response: 1 if code- ρ , 0 otherwise.

Table 16: Variable descriptions for the particle classification dataset.

Appendix H Code Snippets

This appendix provides selected code excerpts illustrating key algorithms and implementations used throughout the study. These snippets are intended to clarify specific methodological steps and computational procedures discussed in the main text. The complete codebase for all three reinforcement learning problems, including full implementations and routines, is available on [GitHub](#).

Neural network implementation

This subsection presents the R implementation of the neural network used in the navigation problem (an R function), with specifications as described in Appendix D. The same general architecture is employed across the other reinforcement learning tasks considered in this study. The primary distinction lies in the choice of activation function in the output layer,

```
1 model = function(X,theta,nodes)
2 {
3   # Infer dimensions:
4   N = dim(X)[1]
5   p = dim(X)[2]
6   q = 2
7   dims = c(p,nodes,q)
8
9   # Populate weight and bias matrices:
10  index = 1:(dims[1]*dims[2])
11  W1 = matrix(theta[index],dims[1],dims[2])
12  index = max(index)+1:(dims[2]*dims[3])
13  W2 = matrix(theta[index],dims[2],dims[3])
14  index = max(index)+1:(dims[3]*dims[4])
15  W3 = matrix(theta[index],dims[3],dims[4])
16
17  index = max(index)+1:(dims[2])
18  b1 = matrix(theta[index],dims[2],1)
```

```

19 index = max(index)+1:(dims[3])
20 b2 = matrix(theta[index],dims[3],1)
21 index = max(index)+1:(dims[4])
22 b3 = matrix(theta[index],dims[4],1)
23
24 ones = matrix(1,1,N)
25 a0 = t(X)
26
27 # Evaluate the updating equation in matrix form
28 a1 = tanh(t(W1)%*%a0+b1%*%ones)
29 a2 = tanh(t(W2)%*%a1+b2%*%ones)
30 a3 = tanh(t(W3)%*%a2+b3%*%ones)
31
32 # Return a list of relevant objects:
33 return(list(a3 = t(a3)))
34 }

```

The Metropolis-Hastings algorithm

This section describes the two-block MCMC algorithm, incorporating the adaptive methods, used for the navigation problem as outlined in Section 3.1.1. In particular, we present the version of the algorithm tailored to the navigation problem, which accounts for the presence of different pseudo-likelihood forms.

```

1 library("invgamma")
2 library('MASS')
3 a = 1e-3
4 b = 1e-3
5 S = npars+1
6 prop_var = 1
7 theta = rnorm(S, mean = 0, sd = sqrt(prop_var))
8 epsilon0 <- 1e-6
9 cov_prop = diag(S)*prop_var
10 window_size = 100
11 stride = 1000
12 kappa = 0.6
13 target_accept = 0.234
14 s = 1
15 # Pick sharpness
16 beta = 100
17
18 sig_theta2 = rinvgamma(n = 1, shape = a + S/2, rate = b + sum(theta^2)/2)
19 runs = 1e5
20 t = nrow(xt)
21 theta_store = matrix(NA, nrow = S, ncol = runs+1)
22 theta_accept = matrix(NA, nrow = S, ncol = runs)
23 sig_theta2_store = matrix(NA, nrow = 1, ncol = runs+1)
24 kstore = c()
25 theta_store[, 1] = theta
26 theta_accept[, 1] = theta
27 sig_theta2_store[, 1] = sig_theta2
28
29 alphastore = c()
30 k_theta_store = c()

```

```

31 k_theta_new_store = c()
32
33 # Binomial-Based
34 f= function(k){
35   choose(t, k)*(k/t)^(k*(1-k/t)^(t - k)
36 }
37 h = function(k){
38   2/t* f(t/2)*k*(k < t/2) + f(k)*(k >= t/2)
39 }
40 lik = function(beta, k){
41   h(k)^beta
42 }
43
44 # Beta-Based
45 f = function(x){
46   1/(gamma(x+1)*gamma(2-x))*(x^x)*((1-x)^(1-x))
47 }
48 h = function(x){
49   2* f(1/2)*x*(x < 1/2) + f(x)*(x >= 1/2)
50 }
51 lik = function(beta, x){
52   h(x)^beta
53 }
54
55 # Exponential-Based
56 lik = function(beta, x){
57   exp(beta*x)
58 }
59
60 acpt_cnt = 0
61 k_theta_store = c()
62 p_theta = play_a_game(theta)
63 k_theta = p_theta*t
64 k_theta_store[1] = k_theta
65 post_store = matrix(NA, nrow = 1, ncol = runs)
66 post_store[, 1] =log(lik(beta, k_theta)) - sum(theta^2)/(2*sig_theta2)
67
68 for (i in 2:runs){
69   # Block 1
70   theta_new <- mvrnorm(1, theta, cov_prop)
71
72   p_theta = play_a_game(theta)
73   p_theta_new = play_a_game(theta_new)
74   k_theta = p_theta*t
75   k_theta_store[i] = k_theta
76   k_theta_new = p_theta_new*t
77   k_theta_new_store[i] = k_theta_new
78
79   if (k_theta_new > 0){
80     # Binomial-Based
81     alpha = exp(min(log(lik(beta, k_theta_new)) - sum(theta_new^2)/(2*sig_theta2) -
82       ↪ log(lik(beta, k_theta)) + sum(theta^2)/(2*sig_theta2), 0))
83     # Beta-based
84     # alpha = exp(min(log(lik(beta, p_theta_new)) - sum(theta_new^2)/(2*sig_theta2) -
85       ↪ log(lik(beta, p_theta)) + sum(theta^2)/(2*sig_theta2), 0))

```

```

84     # Exp-based
85     # alpha = exp(min(log(lik(beta, p_theta_new)) - sum(theta_new^2)/(2*sig_theta2) -
86     ↪ log(lik(beta, p_theta)) + sum(theta^2)/(2*sig_theta2), 0))
87 }else {alpha = 0}
88 if (runif(1, 0, 1) < alpha){
89     theta = theta_new
90     acpt_cnt = acpt_cnt +1
91     post_store[, i] = log(lik(beta, k_theta_new)) - sum(theta^2)/(2*sig_theta2)
92 }else{
93     post_store[, i] = log(lik(beta, k_theta))- sum(theta^2)/(2*sig_theta2)
94 }
95 theta_store[, i] = theta
96 alphastore[i] = alpha
97
98 #Block 2
99 sig_theta2 = rinvgamma(n = 1, shape = a + S/2, rate = b + sum(theta^2)/2)
100 sig_theta2_store[, i] = sig_theta2
101
102 if (i < stride)
103 {
104     gamma_n <- 1 / (i^kappa)
105     s <- s * exp(gamma_n * (alpha - target_accept))
106     cov_emp <- cov(t(theta_store[,1:i]))
107     cov_prop <- s*(cov_emp + epsilon0 * diag(SS))
108 }
109
110 if (i > stride)
111 {
112     gamma_n <- 1 / (i^kappa)
113     s <- s * exp(gamma_n * (alpha - target_accept))
114     idx_recent = seq(from = i - floor(i/stride)*stride, to = i, by = stride)
115     theta_recent <- theta_store[, idx_recent]
116     cov_emp <- cov(t(theta_recent))
117     cov_prop <- s*( cov_emp + epsilon0 * diag(S))
118 }
119 }

```

The hybrid algorithm (for the blackjack problems)

This subsection details the hybrid algorithm employed for the three blackjack problems examined in the study. The function `play_a_game` used here merely evaluates the arbitrary objective function value (which differs across the three blackjack problems).

```

1  library("invgamma")
2  library('MASS')
3  libraryr('GA')
4
5  log_cond = function(theta, sigma_theta2){
6      beta* play_a_game(theta) - 1/(2*sigma_theta2)*sum(theta^2) - S/2 * log(2*pi *sigma_theta2)
7  }
8
9  popSize =100
10 n_gens = 100

```

```

11  beta= 100
12  S = npars
13  a = 1e-3
14  b = 1e-3
15  sigma_theta2_store = matrix(NA, nrow = popSize, ncol = n_gens)
16  pop_store = array(NA, dim = c(popSize, S, n_gens))
17  fitness_store = matrix(NA, nrow = popSize, ncol = n_gens)
18
19  monitor_func <- function(ga_obj) {
20    current_gen <- ga_obj@iter
21    sigma_theta2_store[, current_gen] <- apply(ga_obj@population, 1, function(theta) {
22      rinvgamma(1, shape = a + S/2, rate = b + sum(theta^2)/2)
23    })
24    fitness_store[, current_gen] <- ga_obj@fitness
25    pop_store[, , current_gen] <- (ga_obj@population)
26    gaMonitor(ga_obj)
27  }
28
29  best_log_cond <- -Inf
30  best_theta <- NULL
31  best_sigma_theta2 <- NULL
32  obj = function(theta){
33    sigma_theta2 = rinvgamma(1, shape = a + S/2, rate = b + sum(theta^2)/2)
34    current_log_cond = log_cond(theta, sigma_theta2)
35    if (current_log_cond > best_log_cond) {
36      best_log_cond <- current_log_cond
37      best_theta <- theta
38      best_sigma_theta2 <- sigma_theta2
39    }
40    return(current_log_cond)
41  }
42
43  GA <- ga(type = "real-valued", fitness = obj, lower = rep(-10, S), upper = rep(10, S), popSize =
  ↪ popSize, maxiter = n_gens,
44    keepBest = TRUE, monitor = monitor_func, pmutation = 0.8)
45
46
47  param_store = matrix(NA, nrow = S, ncol = popSize*n_gens)
48  for (j in 1:S){
49    onestore =c()
50    for (i in 1:n_gens){
51      onestore = c(onestore, pop_store[1:popSize,j, i])
52    }
53    param_store[j, ] = onestore
54  }

```

The S-17 blackjack environment

This section details the entire S-17 blackjack environment used in the study, with this specific variant allowing early surrender and double-after-split (DAS), but not permitting re-splitting or surrendering after a split.

```

1 blackjack_env <- function(n_decks = 8, reshuffle_threshold = 4*52) {
2   env <- list()
3   state <- new.env()
4
5   init_shoe <- function() {
6     shoe <- rep(c(1:9, 10, 10, 10, 10), 4 * n_decks)
7     sample(shoe) # Shuffle the shoe
8   }
9
10  update_count <- function(card) {
11    if (card >= 2 && card <= 6) state$running_count <- state$running_count + 1
12    else if (card == 1 || card == 10) state$running_count <- state$running_count - 1
13  }
14
15  update_at_hist <- function(card) {
16    state$at_hist <- c(state$at_hist, card)
17  }
18
19  draw_card <- function(count = TRUE) {
20    if (length(state$deck) == 0) {
21      state$deck <- init_shoe()
22      state$running_count <- 0
23    }
24    card <- state$deck[1]
25    state$deck <- state$deck[-1]
26    if (count) {
27      update_count(card)
28      update_at_hist(card)
29    }
30    return(card)
31  }
32
33  hand_value <- function(hand) {
34    val <- sum(hand)
35    if (1 %in% hand && val + 10 <= 21) val <- val + 10
36    val
37  }
38
39  usable_ace <- function(hand) {
40    1 %in% hand && (sum(hand) + 10 <= 21)
41  }
42
43  env$reset <- function(bet_size = 1) {
44    if (is.null(state$deck) || length(state$deck) < reshuffle_threshold) {
45      state$deck <- init_shoe()
46      state$running_count <- 0
47      state$at_hist <- c()
48    }
49
50    state$player <- c(draw_card(), draw_card())
51    dealer_up <- draw_card()
52    dealer_hole <- draw_card(count = FALSE)
53    state$dealer <- c(dealer_up, dealer_hole)
54    state$done <- FALSE

```

```

55     state$bet_size <- bet_size
56     state$history <- list(
57       player = list(state$player),
58       dealer = list(dealer_up)
59     )
60     state$can_split <- (state$player[1] == state$player[2])
61     state$split <- FALSE
62
63     if (hand_value(state$player) == 21) {
64       state$done <- TRUE
65       update_count(dealer_hole)
66       update_at_hist(dealer_hole)
67       state$history$dealer[[length(state$history$dealer) + 1]] <- state$dealer
68       reward <- if (hand_value(state$dealer) == 21) 0 else 1.5
69       return(list(obs = env$get_obs(), reward = reward * state$bet_size, done = TRUE))
70     }
71
72     return(list(obs = env$get_obs(), reward = 0, done = FALSE))
73   }
74
75   env$get_obs <- function() {
76     list(
77       player_sum = hand_value(state$player),
78       dealer_show = state$dealer[1],
79       usable_ace = usable_ace(state$player),
80       running_count = state$running_count,
81       true_count = env$true_count(),
82       done = state$done,
83       bet_size = state$bet_size,
84       history = state$history,
85       at_history = state$at_hist,
86       can_split = state$can_split && !state$split && length(state$player) == 2,
87       can_surrender = length(state$player) == 2 && !state$split,
88       can_doubledown = length(state$player) == 2
89     )
90   }
91
92   env$step <- function(action) {
93     if (state$done) stop("Game is over. Please reset.")
94
95     if (action == 1) {
96       # Hit
97       new_card <- draw_card()
98       state$player <- c(state$player, new_card)
99       state$history$player[[length(state$history$player) + 1]] <- state$player
100       if (state$split) {
101         state$player_hands[[state$current_hand_index]] <- state$player
102       }
103
104       if (hand_value(state$player) == 21){
105         return(env$step(action = 0))
106       }
107
108       if (hand_value(state$player) > 21) {
109         if (state$split) {

```

```

110     state$hand_rewards[state$current_hand_index] <- -1 *
      ↪ state$bet_sizes[state$current_hand_index]
111     # Move to next hand if there is one
112     if (state$current_hand_index < length(state$player_hands)) {
113         state$current_hand_index <- state$current_hand_index + 1
114         state$player <- state$player_hands[[state$current_hand_index]]
115         return(list(obs = env$get_obs(), reward = 0, done = FALSE))
116     } else {
117         state$done <- TRUE
118         return(list(obs = env$get_obs(), reward = sum(state$hand_rewards), done = TRUE))
119     }
120 }else{
121     state$done <- TRUE
122     return(list(obs = env$get_obs(), reward = -1*state$bet_size, done = TRUE))
123 }
124 }else{
125     return(list(obs = env$get_obs(), reward = 0, done = FALSE))
126 }
127 }
128 if (action == 0) {
129     # Stand
130     if (state$split && state$current_hand_index < length(state$player_hands)) {
131         # Move to next split hand
132         state$current_hand_index <- state$current_hand_index + 1
133         state$player <- state$player_hands[[state$current_hand_index]]
134         # state$history$player <- list(state$player)
135         return(list(obs = env$get_obs(), reward = 0, done = FALSE))
136     }
137
138     # Reveal hole card
139     update_count(state$dealer[2])
140     update_at_hist(state$dealer[2])
141     state$history$dealer[[length(state$history$dealer) + 1]] <- state$dealer
142
143     while (hand_value(state$dealer) < 17) {
144         new_card <- draw_card()
145         state$dealer <- c(state$dealer, new_card)
146         state$history$dealer[[length(state$history$dealer) + 1]] <- state$dealer
147     }
148
149     # Evaluate rewards for each hand
150     if (state$split) {
151         for (i in seq_along(state$player_hands)) {
152             hand <- state$player_hands[[i]]
153             player_val <- hand_value(hand)
154             dealer_val <- hand_value(state$dealer)
155             if (player_val > 21) {
156                 state$hand_rewards[i] <- -1 * state$bet_sizes[i]
157             }else{
158                 state$hand_rewards[i] <- if (dealer_val > 21 || player_val > dealer_val) {
159                     1 * state$bet_sizes[i]
160                 } else if (player_val < dealer_val) {
161                     -1 * state$bet_sizes[i]
162                 } else {
163                     0

```

```

164     }
165   }
166 }
167 state$done <- TRUE
168 return(list(obs = env$get_obs(), reward = sum(state$hand_rewards), done = TRUE))
169 }else {
170   player_val <- hand_value(state$player)
171   dealer_val <- hand_value(state$dealer)
172   if (player_val > 21)
173   {
174     reward = -1*state$bet_size
175   }else
176   {
177     reward <- if (dealer_val > 21 || player_val > dealer_val) {
178       1 * state$bet_size
179     } else if (player_val < dealer_val) {
180       -1 * state$bet_size
181     } else {
182       0
183     }
184   }
185   state$done <- TRUE
186   return(list(obs = env$get_obs(), reward = reward, done = TRUE))
187 }
188 }
189
190 if (action == 2 && state$can_split && !state$split && length(state$player) == 2) {
191   # Perform split
192   card1 <- state$player[1]
193   card2 <- state$player[2]
194
195   state$player_hands <- list(
196     c(card1, draw_card()),
197     c(card2, draw_card())
198   )
199   state$current_hand_index <- 1
200   state$split <- TRUE
201   state$hand_rewards <- c(0, 0)
202   state$bet_sizes <- rep(state$bet_size, 2)
203   state$player <- state$player_hands[[1]]
204   # state$history$player <- list(state$player)
205   state$history$player <- state$player_hands
206   return(list(obs = env$get_obs(), reward = 0, done = FALSE))
207 }
208 if (action == 3) {
209   # Surrender (only allowed if no hit/split has occurred)
210   if (length(state$player) == 2 && !state$split) {
211     state$done <- TRUE
212     return(list(obs = env$get_obs(), reward = -0.5 * state$bet_size, done = TRUE))
213   }
214 }
215 if (action == 4) {
216   # Double down: allowed if current hand has exactly 2 cards and hasn't been hit yet
217   if (length(state$player) == 2) {
218     new_card <- draw_card()

```

```

219     state$player <- c(state$player, new_card)
220     state$history$player[[length(state$history$player) + 1]] <- state$player
221
222     # Handle bet increase
223     if (state$split) {
224         state$bet_sizes[state$current_hand_index] <- 2 *
225         ↪ state$bet_sizes[state$current_hand_index]
226     } else {
227         state$bet_size <- 2 * state$bet_size
228     }
229
230     if (hand_value(state$player) > 21) {
231         if (state$split) {
232             state$hand_rewards[state$current_hand_index] <- -1 *
233             ↪ state$bet_sizes[state$current_hand_index]
234             # Move to next hand if there is one
235             if (state$current_hand_index < length(state$player_hands)) {
236                 state$current_hand_index <- state$current_hand_index + 1
237                 state$player <- state$player_hands[[state$current_hand_index]]
238                 return(list(obs = env$get_obs(), reward = 0, done = FALSE))
239             } else {
240                 state$done <- TRUE
241                 return(list(obs = env$get_obs(), reward = sum(state$hand_rewards), done = TRUE))
242             }
243         } else {
244             state$done <- TRUE
245             return(list(obs = env$get_obs(), reward = -2*state$bet_size, done = TRUE))
246         }
247     } else {
248         return(env$step(action = 0))
249     }
250 }
251 }
252
253 env$true_count <- function() {
254     remaining_decks <- max(1, length(state$deck) / 52)
255     round(state$running_count / remaining_decks, 2)
256 }
257
258 env$get_state <- function() state
259
260 return(env)
261 }

```
